INPT
المعهد الوطني للبريد والمواصلات
ⵜⴰⵙⴷⴰⵡⵜ ⵏ ⵜⵓⵔⵉⵜ ⴷ ⵜⵙⵡⵉⵍⵜ
Institut National des Postes et Télécommunications

SCIENTIAE

End-of-Study Project Report

To obtain the Engineering Degree

Major: Embedded Systems and Digital Services

Knowledge Graph Exploration and Querying Platform with Retrieval-Augmented Generation

---

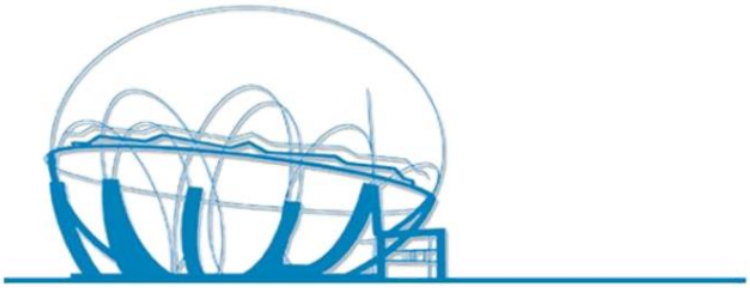
Project carried out at SCIENTIAE

Authored by:

Mr. Anouar EL IDRISSI

Defended on July 27, 2026, before the jury composed of:

Ms. Safae DAHMANI	INPT	- Examiner
Mr. Mouhcine CHAMI	INPT	- Examiner
Ms. Assia EL HADBI	INPT	- Internal Supervisor
Mr. Issam AMROUSS	SCIENTIAE	- External Supervisor



Agence Nationale de Réglementation des Télécommunications

Institut National des Postes et Télécommunications

Promotion: 2025/2026

Dedication

This report is dedicated, first and foremost, to my mother, whose unconditional love, endless sacrifices, and unwavering support have been the foundation of every achievement in my life. Her strength, patience, and encouragement have inspired me throughout my academic journey.
I also dedicate this work to my brother and sisters, whose affection, confidence, and constant encouragement have always motivated me to strive for excellence and persevere through challenges.
To my entire family, thank you for your trust, understanding, and support during every stage of my studies. Your belief in me has been a constant source of strength and determination.
Finally, I dedicate this work to all those who have contributed, directly or indirectly, to my education and personal growth, and who have nurtured my appreciation for scientific rigor, technical discipline, and the pursuit of knowledge.

Acknowledgements

I would like to express my sincere gratitude to everyone who contributed to the successful completion of this end-of-study project.

I would first like to thank **Ms. ASSIA EL HADBI**, my internal supervisor at INPT, for her guidance, rigorous follow-up, and constructive feedback, which helped maintain a demanding academic approach throughout the project.

I also extend my sincere thanks to **Mr. Amrouss Issam** and **Mohammed EL AMRANI** for their technical support, availability, and trust during this internship. Their feedback was essential in connecting architectural decisions to the real needs of the platform.

I thank the entire **SCIENTIAE** team for their welcome, the quality of our exchanges, and the working environment provided during this project. This experience offered a valuable context for deepening key topics in data engineering, natural language processing, and knowledge graphs.

Finally, I thank my teachers and the entire INPT pedagogical staff for the training provided during the engineering program, which supplied the scientific and methodological foundations necessary for completing this report.

Abstract

This report presents the design and implementation of a multimodal Graph-RAG platform entitled *Knowledge Graph Exploration and Querying Platform with RAG*. Developed in the context of political intelligence, the system combines autonomous web crawling, dense semantic search, structured traversal of a Neo4j knowledge graph, and grounded answer synthesis supported by the Mistral language model. Its objective is to transform heterogeneous political information into a traceable and exploitable knowledge layer.

The project addresses the limitations of classical retrieval-augmented generation when applied to relational political data. Standard RAG can retrieve relevant passages, but it struggles with multi-hop questions, entity disambiguation, temporal interpretation, and explicit relationship tracking. By integrating vector retrieval with graph-structured evidence, the proposed platform improves the ability to answer complex analytical questions while preserving traceability to source documents.

The implemented architecture includes a three-level crawler, an administrative scheduler, natural language processing extraction services, deterministic entity resolution, graph persistence, document storage, local vector retrieval, and a React interface for exploration and consultation. The crawler monitors curated RSS feeds, performs query-driven Google News discovery, and parses PDF reports. The platform was evaluated on factual, relational, temporal, and comparative questions using RAGAS-inspired metrics, showing the value of combining retrieval, graph evidence, and reflexive reasoning.

Keywords: Graph-RAG, knowledge graph, Neo4j, Mistral, autonomous crawling, political intelligence, natural language processing, relation extraction, retrieval-augmented generation, distributed systems.

Résumé

Ce rapport présente la conception et la mise en œuvre d'une plateforme *Graph-RAG* multimodale intitulée *Knowledge Graph Exploration and Querying Platform with RAG*. Développé dans un contexte d'intelligence politique, le système combine un crawl web autonome, une recherche sémantique dense, l'exploration structurée d'un graphe de connaissances **Neo4j** et une synthèse de réponses fondée sur les preuves à l'aide du modèle **Mistral**. Son objectif est de transformer des informations politiques hétérogènes en une couche de connaissance traçable et exploitable.

Le projet répond aux limites de la génération augmentée par récupération classique lorsqu'elle est appliquée à des données politiques relationnelles. Le RAG standard peut retrouver des passages pertinents, mais il peine sur les questions multi-sauts, la désambiguïsation d'entités, l'interprétation temporelle et le suivi explicite des relations. En intégrant la récupération vectorielle à des preuves structurées sous forme de graphe, la plateforme proposée améliore la capacité à répondre à des questions analytiques complexes tout en préservant la traçabilité vers les documents sources.

L'architecture réalisée comprend un crawler à trois niveaux, un planificateur d'administration, des services d'extraction NLP, une résolution déterministe des entités, la persistance du graphe, le stockage documentaire, la recherche vectorielle locale et une interface **React** d'exploration et de consultation. Le crawler surveille des flux **RSS** sélectionnés, effectue une découverte d'actualités via **Google News** et analyse des rapports **PDF**. La plateforme a été évaluée sur des questions factuelles, relationnelles, temporelles et comparatives à l'aide de métriques inspirées de **RAGAS**, montrant l'intérêt de combiner récupération, preuves graphiques et raisonnement réflexif.

Mots-clés : Graph-RAG, graphe de connaissances, Neo4j, Mistral, crawl autonome, intelligence politique, traitement du langage naturel, extraction de relations, génération augmentée par récupération, systèmes distribués.

ملخص

يقدم هذا التقرير تصميم وتنفيذ منصة *Graph-RAG* متعددة الوسائط بعنوان *Knowledge Graph Exploration and Querying Platform with RAG*. طورت المنصة في سياق التحليل السياسي، وتجمع بين الزحف الذاتي للويب، والبحث الدلالي الكثيف، والاستكشاف المنظم لرسم بياني معرفي في **Neo4j**، وتوليد إجابات موثقة بمساعدة نموذج **Mistral**. وتهدف إلى تحويل المعلومات السياسية المتنوعة إلى طبقة معرفية قابلة للتتبع والاستغلال.

يعالج المشروع محدوديات الجيل المعزز بالاستخلاص الكلاسيكي عند تطبيقه على المعطيات السياسية ذات البعد العلائقي. يمكن للاستخلاص التقليدي إيجاد مقاطع ذات صلة، لكنه يواجه صعوبة في الأسئلة متعددة القفزات، وإزالة لبس الكيانات، والتفسير الزمني، وتتبع العلاقات بشكل صريح. ومن خلال دمج الاستخلاص الشعاعي مع أدلة منظمة على شكل رسم بياني، تحسن المنصة المقترحة القدرة على الإجابة عن أسئلة تحليلية معقدة مع الحفاظ على إمكانية الرجوع إلى الوثائق المصدرية.

تتضمن المعمارية المنجزة زاحفا بثلاثة مستويات، ومجدولا إداريا، وخدمات لاستخراج المعلومات بالمعالجة اللغوية، وحلا حتميا للكيانات، وتخزينا للرسم البياني والوثائق، واستخلاصا شعاعيا محليا، وواجهة **React** للاستكشاف والاستشارة. يراقب الزاحف تدفقات **RSS** مختارة، ويكتشف الأخبار عبر Google News، ويحلل تقارير PDF. وقد قيمت المنصة على أسئلة واقعية وعلائقية وزمنية ومقارنة باستعمال مقاييس مستوحاة من **RAGAS**، وأظهرت النتائج فائدة الجمع بين الاستخلاص، وأدلة الرسم البياني، والتفكير الانعكاسي.

الكلمات المفتاحية: Graph-RAG، رسم بياني معرفي، Neo4j، Mistral، زحف ذاتي، تحليل سياسي، معالجة اللغة الطبيعية، استخراج العلاقات، توليد معزز بالاستخلاص، أنظمة موزعة.

Contents

Acknowledgements **2**

Abstract **3**

Résumé **4**

Arabic Abstract **5**

Table of Contents **6**

List of Figures **7**

List of Tables **8**

List of Acronyms **9**

General Introduction **1**

1 Project Context and Requirements **4**

1.1 Host Organization 4

1.1.1 Company Profile and Activities 4

1.1.2 Research Need 6

1.2 Problem Definition 6

1.2.1 Core Problem Statement 6

1.2.2 Scope and Constraints 7

1.2.3 Target Domain Assumptions 7

1.3 Proposed Solution and Objectives 8

1.3.1 Solution Overview 8

1.3.2 Project Objectives 8

1.4 Actors, Use Cases, and Domain Model 9

1.4.1 Actor Definitions . . . 9
1.4.2 Use Case Diagram Overview . . . 9
1.4.3 Domain Class Model Overview . . . 10
1.4.4 Key Use Case Narratives . . . 11
1.5 Functional Requirements . . . 12
1.5.1 Data Acquisition and Ingestion Stage . . . 12
1.5.2 Extraction and Structuring Stage . . . 13
1.5.3 Retrieval and Reasoning Stage . . . 14
1.5.4 User Interaction and Administration Stage . . . 15
1.6 Non-Functional Requirements . . . 15
1.6.1 Performance and Scalability . . . 15
1.6.2 Reliability and Idempotency . . . 15
1.6.3 Security and Data Sovereignty . . . 15
1.6.4 Observability and Auditability . . . 16
1.7 Technology Stack and Selection Rationale . . . 16
1.8 Requirements Validation Criteria . . . 17
1.8.1 Evidence and Metrics . . . 17
1.9 Project Management . . . 18
1.9.1 Validation Methodology . . . 18
1.9.1.1 User Stories . . . 19
1.9.1.2 Kanban Workflow . . . 19
1.9.1.3 Project Management Tools . . . 20
1.9.1.4 Execution Timeline (Gantt Diagram) . . . 21

2 State of the Art **23**
2.1 Conceptual Foundations . . . 23
2.1.1 Scope and Essential Terminology . . . 23
2.2 Grounded Retrieval . . . 24
2.2.1 Large Language Models and Their Limits . . . 24
2.2.2 Retrieval-Augmented Generation . . . 26
2.2.3 Searchable Evidence . . . 27
2.2.3.1 Document Chunking . . . 27
2.2.3.2 Embedding Models and Semantic Similarity . . . 28
2.2.3.3 Vector Databases . . . 30
2.2.4 Hybrid Retrieval and Reranking . . . 31

2.3 Structured Knowledge . . . 32
2.3.1 Information Extraction and Entity Resolution . . . 32
2.3.2 Knowledge Graph Fundamentals . . . 34
2.4 Graph-RAG and Reasoning . . . 35
2.4.1 Classical RAG and Graph-RAG . . . 35
2.4.2 Graph-Based Retrieval Strategies . . . 37
2.4.3 Benefits and Limitations . . . 37
2.4.4 Reasoning and Adaptive Retrieval . . . 37
2.5 Domain Context and Evaluation . . . 39
2.5.1 Political and Temporal NLP . . . 39
2.5.2 Evaluation of Retrieval-Augmented Systems . . . 40
2.6 Existing Solutions and Research Gaps . . . 40
2.6.1 Representative Systems . . . 40
2.6.1.1 Classical RAG . . . 40
2.6.1.2 Microsoft GraphRAG . . . 41
2.6.1.3 Neo4j Graph-Based Retrieval . . . 41
2.6.1.4 Wikidata . . . 41
2.6.1.5 GDELT . . . 41
2.6.2 Comparative Analysis . . . 42
2.6.3 Research Gaps . . . 42

3 System Design and Architecture 45
3.1 Design Principles . . . 45
3.2 Use Case Refinement . . . 50
3.3 Data Flow Architecture . . . 51
3.4 Ingestion Architecture . . . 52
3.5 Knowledge Representation and Ontology . . . 54
3.6 Data Model and Physical Schema . . . 56
3.7 Reasoning Architecture . . . 58
3.8 Deployment Architecture . . . 61

4 Implementation and Engineering 63
4.1 NLP Pipeline and Extraction . . . 64
4.2 Persistence Layer Implementation . . . 65
4.3 Retrieval and Reasoning Implementation . . . 68
4.4 Frontend Implementation . . . 70
4.5 Deployment and Observability . . . 81

5 System Evaluation and Performance Analysis **85**

5.1 Evaluation Methodology 85

5.2 Evaluation Metrics and Results 87

5.3 Optimization and Worked Evaluation Example 91

5.4 Requirements Validation 92

5.5 Interpretation and Limitations 93

Conclusion and Perspectives **95**

Bibliographie **98**

Webographie **100**

List of Figures

1.1 SCIENTIAE, host organization of the project. 4
1.2 Organizational structure of SCIENTIAE. 5
1.3 Overview of system use cases for authenticated users and administrators. 10
1.4 UML domain class model linking documents, extracted knowledge, retrieval evidence, and user interactions. 11
1.5 Representative technologies used by the proposed platform. 16
1.6 Trello Kanban board used to manage the project workflow. 20
1.7 Trello (Kanban board) and Slack (team communication) used to manage internship tasks and coordination. 20
1.8 Gantt diagram of the end-of-study internship timeline aligned with major requirement delivery milestones. 21

2.1 Simplified technical architecture of a Transformer-based large language model, adapted from [25]. 25
2.2 Core stages of a retrieval-augmented generation pipeline: indexing, retrieval, context construction, and grounded generation. 26
2.3 ZVec identity and official benchmark overview for vector retrieval at scale [13, 14]. 31
2.4 General process for constructing a knowledge graph from unstructured documents. 35
2.5 Comparison between classical RAG and Graph-RAG retrieval paths. . . 36

3.1 Logical component architecture and separation of pipeline responsibilities. 46
3.2 Vector-space retrieval from question embedding to ranked evidence chunks. 47
3.3 Graph-space traversal from seed entities to structured relational evidence. 48
3.4 Hybrid evidence fusion across vector, graph, and structured retrieval paths. 49
3.5 System component overview for ingestion, persistence, reasoning, and observability. 50
3.6 Detailed use case model for analytical and administrative workflows. . . 51

3.7 Data flow architecture from acquisition to grounded answer generation. 52
3.8 Technical crawler architecture with routing, extraction fallbacks, quality gates, and retry control. 53
3.9 Document lifecycle and ingestion checkpoints before retrieval. 54
3.10 Domain data model linking evidence, graph topology, structured records, and vector retrieval. 55
3.11 Core geopolitical entities, relation families, and provenance attributes. . 56
3.12 Complementary persistence roles of MongoDB, Neo4j, and ZVec. 57
3.13 Physical persistence schema across MongoDB, Neo4j, and ZVec. 58
3.14 Reasoning execution sequence across retrieval, graph expansion, audit, and synthesis. 59
3.15 Comparison of standard RAG, chain-of-thought, and reflexive reasoning modes. 60
3.16 Technical service structure across API routes, services, connectors, stores, and model clients. 61
3.17 Containerized deployment topology across edge, application, and data services. 62

4.1 Implemented processing path from acquisition to the monitored analytical workspace. 64
4.2 Offline DSPy and MIPROv2 optimization loop for production extraction programs. 65
4.3 Implemented structure-aware chunking, metadata enrichment, and adjacent-context recovery. 67
4.4 Offline embedding and indexing followed by online similarity retrieval in ZVec. 68
4.5 Three reasoning modes in the query engine: one-hop RAG, chain-of-thought multi-hop, and reflexive control. 69
4.6 Hybrid RAG implementation with optional DuckDuckGo web search at query time. 70
4.7 Chat workspace with session history sidebar and reasoning-mode controls. 71
4.8 Cited answer produced in standard RAG mode with linked source evidence. 71
4.9 Expandable reasoning trace for a RAG-mode query (retrieval and synthesis steps). 72
4.10 Document library for inspecting ingested corpus items and metadata. . . 73
4.11 Graph explorer with interactive entity neighborhood layout. 73

4.12 Legislative tracker: bills view with structured metadata from extracted records. . . . 74
4.13 Legislative tracker: votes view linking legislators to roll-call outcomes. . . 74
4.14 Legislative tracker: committees view for institutional roles and memberships. . . . 75
4.15 Political insights: campaign finance dashboard. . . . 76
4.16 Political insights: temporal trend tracking for entities or topics. . . . 77
4.17 Entity comparison workflow (step 1): selection of actors or topics to compare. . . . 77
4.18 Entity comparison workflow (step 2): relation matrix between selected entities. . . . 78
4.19 Entity comparison workflow (step 3): convergence, divergence, and narrative conclusion. . . . 78
4.20 Political insights: conflict and opposing-stance visualization. . . . 79
4.21 Administration dashboard: health metrics, corpus counts, and pipeline overview. . . . 80
4.22 Administration: scraped-data ingestion and vector backfill controls. . . . 80
4.23 Administration: cron scheduler and agentic trend-collection configuration. 81
4.24 Administration: advanced topic search for query-driven source discovery. 81
4.25 Observability stack: metrics, dashboards, and distributed tracing. . . . 82
4.26 Optimization feedback loop from OpenTelemetry signals to measured engineering adjustments. . . . 83
4.27 Deployment topology of containerized services and data stores. . . . 83

5.1 Evaluation pipeline from benchmark questions to aggregated results by mode. . . . 87
5.2 Embedding evaluation dashboard comparing response time and retrieval quality. . . . 89
5.3 Answer-correctness profile by question category and reasoning mode. . . 90

List of Tables

1.1 Registry of high priority functional requirements 13
1.2 Technology stack and rationale for the principal engineering choices. . . 17
1.3 Functional requirements validation criteria by area. 18

2.1 Essential terms used throughout the state-of-the-art chapter. 24
2.2 Comparison of common chunking strategies in RAG systems 28
2.3 Representative embedding-model alternatives and their principal retrieval trade-offs. 29
2.4 Comparison of representative vector-retrieval technologies 30
2.5 Comparison of representative entity and relation extraction approaches. 33
2.6 Comparison of common entity-resolution strategies. 34
2.7 Comparison of representative Graph-RAG retrieval strategies. 37
2.8 Comparison of reasoning and control strategies used in retrieval-augmented systems. 39
2.9 Comparative summary of major approach families across retrieval, reasoning, and evidence-management criteria. 42

4.1 Embedding alternatives and retrieval trade-offs considered during implementation. 66

5.1 Distribution of the 150-question benchmark. 86
5.2 RAGAS scores by reasoning mode. 88
5.3 Comparison of manual and DSPy-optimized extraction. 91
5.4 Worked example of the information required for one RAGAS evaluation sample. 92
5.5 Requirements-validation matrix. 93

List of Acronyms

Acronym	Meaning
ANN	Approximate Nearest Neighbour
AI	Artificial Intelligence
API	Application Programming Interface
BLEU	Bilingual Evaluation Understudy
CoT	Chain of Thought
CSS	Cascading Style Sheets
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DPR	Dense Passage Retrieval
DSPy	Declarative Self-improving Python
ERD	Entity Relationship Diagram
FR	Functional Requirements
F1	Harmonic Mean of Precision and Recall
GDELT	Global Database of Events, Language, and Tone
GKG	Global Knowledge Graph
GPU	Graphics Processing Unit
GQA	Grouped-Query Attention
GraphQL	Graph Query Language
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token

Acronym	Meaning
KG	Knowledge Graph
LLM	Large Language Model
MIPRO	Multi-prompt Instruction Proposal Optimizer
MRR	Mean Reciprocal Rank
MS	Microsoft
NFR	Non-Functional Requirement
NER	Named Entity Recognition
NLP	Natural Language Processing
OCR	Optical Character Recognition
PDF	Portable Document Format
QA	Question Answering
RAG	Retrieval-Augmented Generation
RAGAS	Retrieval-Augmented Generation Assessment
RDF	Resource Description Framework
REST	Representational State Transfer
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
RSS	Really Simple Syndication
SAS	Semantic Answer Similarity
SLI	Service Level Indicator
SOA	Service-Oriented Architecture
SPARQL	SPARQL Protocol and RDF Query Language
SQL	Structured Query Language
SSE	Server-Sent Events
TKG	Temporal Knowledge Graph
UI	User Interface
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
UX	User Experience
XML	eXtensible Markup Language
YAML	YAML Ain't Markup Language

General Introduction

The digital transformation of political communication has produced an unprecedented volume of public information. News agencies, parliamentary bodies, governmental institutions, research organizations, and civil society actors publish large numbers of articles, reports, and official documents every day. These sources contain valuable signals about actors, events, institutions, alliances, conflicts, financial links, and temporal changes, but most of this information remains embedded in unstructured text.

For analysts, journalists, and researchers, the difficulty is no longer only to access information, but to connect dispersed facts and interpret them in a reliable way. Keyword search can retrieve documents, and classical retrieval-augmented generation can summarize passages, yet both approaches remain limited when the question requires entity disambiguation, multi-hop reasoning, temporal filtering, and explicit source attribution.

This difficulty is amplified by the rhythm of political information. Relevant evidence may appear first in an RSS entry, then in a full news article, then in an institutional report or policy PDF. A platform designed for political intelligence must therefore do more than consume a static dataset. It must continuously discover, collect, normalize, and evaluate heterogeneous sources while preserving the provenance of every fact. In this project, that role is fulfilled by an autonomous crawler composed of several complementary levels: curated RSS monitoring, query-driven news discovery, PDF report ingestion, and administrative controls for scheduled collection.

Within its research and development activities, SCIENTIAE is developing a sovereign Graph-RAG framework designed to adapt to different domains rather than remain tied to a single use case. The framework provides reusable components for data acquisition, LLM-based knowledge extraction, graph construction, semantic retrieval, answer generation grounded in evidence, and system evaluation. A new domain can therefore be integrated by adapting its sources, entity and relation models, extraction rules, and evaluation questions while preserving the same core architecture.

Politics and geopolitics were selected as the application and validation domain for this work because they provide a demanding environment for testing the framework. Political intelligence analysis often depends on relational questions: a user may need to understand how two actors are connected, which events created a diplomatic shift, how

an institution evolved over time, or which documents support a specific claim. Such questions cannot be answered reliably by treating documents as isolated text fragments; they require a representation that combines textual evidence with structured relations.

The implemented system constitutes a domain-specific instance of this broader multimodal Graph-RAG framework. It combines dense semantic retrieval, a Neo4j knowledge graph, DSPy-optimized entity and relation extraction during ingestion, and an LLM-based reasoning layer. Its goal is to transform a large corpus of political and geopolitical information into an exploitable knowledge layer that supports grounded, explainable, and temporally aware analysis. The resulting platform serves both as a functional decision-support tool and as an experimental environment for assessing how the framework can later be adapted to other domains.

A second challenge concerns data acquisition itself. Many Graph-RAG systems assume that the corpus already exists and focus only on retrieval and generation. In this project, the corpus is produced by the platform. The crawler must avoid duplicates, preserve source metadata, handle heterogeneous formats, operate politely with external websites, and provide enough operational visibility for an administrator to supervise ingestion. This makes crawling and ingestion a central contribution of the project rather than a peripheral utility.

The final challenge is trust. Political analysis is sensitive because a generated answer may influence the interpretation of institutions, actors, or conflicts. The system must therefore expose its evidence, separate raw documents from extracted relations, preserve timestamps, and make retrieval behavior auditable. This need explains the combination of MongoDB for document storage, Neo4j for explicit relations, ZVec for local semantic search, and observability tools for operational monitoring.

To address these challenges, the project pursues three main objectives. The first is to contribute to a modular Graph-RAG framework whose principal processing stages can be configured for different domains. The second is to instantiate this framework for political and geopolitical analysis by implementing an autonomous ingestion and knowledge-graph construction pipeline together with a reasoning engine that combines vector retrieval, graph traversal, and cited answer synthesis. The third is to evaluate the system in terms of extraction reliability, retrieval quality, answer grounding, reasoning depth, and response time.

These objectives are translated into concrete engineering goals. The platform must collect data autonomously from RSS feeds, Google News queries, direct URLs, and PDF sources. It must normalize the collected material into document records that can be processed by the same extraction pipeline. It must resolve entities deterministically through canonical names and stable identifiers, without depending on a Wikidata backbone as the main source of truth. It must also provide an administration interface

where ingestion status, scheduler state, logs, source filters, and backfill operations can be controlled without modifying code.

From a user perspective, the objective is to provide an integrated analytical workspace. The analyst can ask questions in natural language, inspect sources, explore graph neighborhoods, follow political trends, consult documents, and observe system health. From an engineering perspective, the objective is to build a modular pipeline in which crawling, extraction, graph publication, chunking, embedding, retrieval, and evaluation remain separable and testable.

The expected contribution is therefore both methodological and practical. Methodologically, the project studies how autonomous crawling, knowledge graph construction, and retrieval-augmented generation can be combined in a reusable workflow adaptable to different domains. Practically, it delivers and evaluates a political and geopolitical implementation in which source acquisition, evidence structuring, semantic search, graph exploration, and answer generation are connected through operational backend services and frontend controls.

The remainder of this report is organized into five chapters.

- Chapter 1 introduces the host organization, project context, problem statement, objectives, actors, requirements, technology choices, and project-management approach.
- Chapter 2 presents the state of the art concerning knowledge graphs, retrieval-augmented generation, Graph-RAG, ingestion strategies, embeddings, and large language models.
- Chapter 3 describes the system architecture, data flows, domain model, ingestion pipeline, and reasoning design.
- Chapter 4 details the implementation, persistence layers, interfaces, deployment, observability, and optimization mechanisms.
- Finally, Chapter 5 presents the system evaluation and performance analysis, including RAGAS metrics, reasoning mode comparison, embedding performance, DSPy-based knowledge extraction optimization, requirements validation, and limitations.

Chapter 1

Project Context and Requirements

Introduction

This chapter establishes the context in which the project was carried out and derives the requirements that guide the rest of the report. It first introduces SCIENTIAE and the research need that motivated the internship. It then moves from the problem definition to the proposed solution, the actors and use cases, the functional and non-functional requirements, the selected technology stack, the project management approach, and the validation criteria.

1.1 Host Organization

1.1.1 Company Profile and Activities

The project was carried out at **SCIENTIAE**, a technology company based in Rabat. Its activities cover software engineering, data science, cybersecurity, and the development of decision-support solutions. The company favors open and controlled infrastructure so that sensitive data, processing services, and analytical results can remain under organizational governance.

Figure 1.1: SCIENTIAE, host organization of the project.

As illustrated in Figure 1.2, SCIENTIAE operates under the direction of its General Management and is organized into two principal departments. The **Administrative Department** brings together the Human Resources and Finance functions, which support personnel administration and the company's financial operations. The **Research and Development Department** concentrates the company's technical and innovation activities through three complementary teams: Cybersecurity Engineering, Software Engineering, and Data Science Engineering.

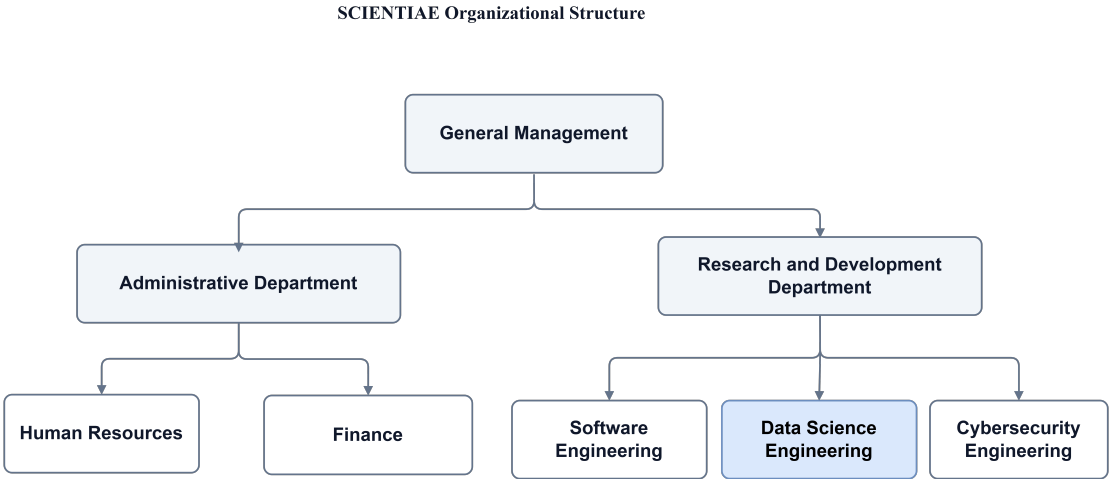


Figure 1.2: Organizational structure of SCIENTIAE.

The project was carried out within the **Data Science Engineering team** of the Research and Development Department, highlighted in the organizational chart. This position placed the work at the intersection of data engineering, natural language processing, knowledge representation, information retrieval, and generative artificial intelligence. The organizational chart clarifies why this team was the appropriate host: it shows that the work belongs to the R&D structure responsible for innovation driven by data rather than to the administrative functions of the company. The team therefore provided the environment required to research and implement a Graph-RAG framework capable of collecting heterogeneous information, extracting entities and relations, constructing knowledge graphs, and producing answers grounded in evidence.

Although the political and geopolitical domain was selected for implementation and validation, the work contributes to SCIENTIAE's broader R&D objective of developing reusable and sovereign decision support solutions based on artificial intelligence. Collaboration with the Software Engineering team supported backend service integration and the analytical interface. The wider R&D structure also helped address system security, controlled infrastructure, maintainability, and deployment throughout the project. This organizational setting explains why the internship was framed not only as an application prototype, but also as a contribution to a reusable research framework.

1.1.2 Research Need

Within its R&D activities, SCIENTIAE is developing a sovereign Graph-RAG framework intended to adapt to multiple domains. The objective is to establish reusable components for acquisition, extraction, graph construction, retrieval, reasoning, and evaluation. Only the elements tied to a particular domain, namely sources, ontology, prompts, and validation questions, should need to change when the framework is applied to a new field.

Politics and geopolitics were selected as the application and validation domain because they combine heterogeneous sources, ambiguous actors, evolving facts, relationships that span several steps, and strict traceability needs. This domain provides a demanding test of whether the framework can transform unstructured information into connected, current, and inspectable knowledge. The following section formalizes this need as an engineering problem.

1.2 Problem Definition

1.2.1 Core Problem Statement

Political intelligence analysis increasingly depends on relational questions. An analyst may need to determine how two actors are connected, which events explain a diplomatic shift, how an institution evolved over time, or which documents support a specific claim. These questions cannot be answered reliably when documents are treated as isolated text fragments. They require a representation that combines textual evidence with structured relations, temporal context, and explicit source attribution.

The modern geopolitical landscape generates a volume of unstructured data that exceeds manual monitoring capacity. Analysts must follow hundreds of heterogeneous sources while preserving traceability from generated statements back to original evidence. Traditional keyword search retrieves documents but does not resolve entity ambiguity or traverse typed relationships. Classical retrieval-augmented generation can summarize passages but remains weak on reasoning across several steps, temporal consistency, and auditable citations.

The platform developed during this end-of-study internship must therefore support four core capabilities: relational traversal across chains of influence with several steps; contextual grounding through granular, citable evidence; synthesis of coherent answers with attribution; and architectural flexibility to adapt the same pipeline to evolving source registries and analytical domains. A representative query illustrates this difficulty: identifying the indirect diplomatic consequences of a legislative vote in one country

for bilateral relations with another requires structured traversal rather than flat chunk retrieval alone.

1.2.2 Scope and Constraints

The project scope covers the complete workflow from the autonomous acquisition of public political documents to grounded question answering over hybrid vector and graph stores. The initial corpus targets **57 distinct and expandable international sources**; ingestion is extended through RSS monitoring, query-driven news discovery, direct URL submission, and PDF report parsing. Proprietary closed datasets unavailable for ingestion, real-time social-media firehose processing at web scale, and fully automated judgments of legal or normative correctness fall outside the core scope.

Engineering constraints include dependence on external LLM APIs for extraction and answer generation, finite GPU and token budgets during development, and the need to operate on sovereign or locally controlled infrastructure for sensitive corpora. The pipeline must remain modular so that crawling, extraction, indexing, retrieval, and evaluation can be executed and inspected independently. These constraints delimit what the solution must provide and what must remain outside the core deliverable.

1.2.3 Target Domain Assumptions

The primary domain assumption is that relevant political evidence appears in public or semi-structured textual sources, including news articles, institutional reports, RSS entries, and policy PDFs, rather than in a pre-built external knowledge base. Entity mentions are noisy, syndicated, and temporally evolving; the same actor may appear under multiple surface forms, and facts may change over time. The system therefore assumes that entity resolution, temporal metadata, and provenance preservation are first-class requirements rather than optional enrichments.

A second assumption is that analysts require both conversational access and direct inspection of underlying artifacts: source documents, graph neighborhoods, ingestion status, and reasoning traces. A third assumption is domain adaptability at the pipeline level: the ingestion layer remains broad while extraction schemas and graph ontologies can be refined without rewriting the frontend or storage interfaces. These assumptions motivate the hybrid retrieval architecture proposed in this project. They also provide the bridge from the problem definition to the solution overview.

1.3 Proposed Solution and Objectives

1.3.1 Solution Overview

The proposed solution is a modular multimodal Graph-RAG framework instantiated for political and geopolitical analysis. It autonomously collects public information, normalizes and stores source documents, extracts entities and relations, constructs a knowledge graph, indexes textual chunks for semantic search, and combines graph and vector evidence to generate cited answers. An analytical interface exposes conversational questioning, source inspection, graph exploration, specialized dashboards, and administrative supervision.

The political implementation is both a usable decision-support platform and a validation environment for the broader framework. Adaptation to another domain is achieved by changing source connectors, extraction schemas, ontology definitions, prompts, and evaluation benchmarks while preserving the principal service boundaries and retrieval workflow. The objectives below translate this solution into concrete project goals.

1.3.2 Project Objectives

- **General objective:** design and validate a sovereign Graph-RAG framework adaptable to domains that require intensive knowledge processing.
- **Domain objective:** instantiate and evaluate the framework using political and geopolitical information.
- **Data objective:** autonomously collect, normalize, deduplicate, and preserve the provenance of heterogeneous sources.
- **Knowledge objective:** improve the reliability of LLM-based extraction during ingestion and publish stable entities, events, temporal facts, and typed relations to an inspectable knowledge graph.
- **Reasoning objective:** combine semantic retrieval and graph traversal for factual, relational, temporal, and multi-step questions.
- **Trust objective:** attach citations, evidence, logs, and reasoning traces to analytical results.
- **Engineering objective:** deliver modular, observable, testable, and deployable services with integrated user and administration interfaces.

These objectives define what the system must accomplish. The next section identifies who interacts with the platform and how those interactions shape the requirements.

1.4 Actors, Use Cases, and Domain Model

1.4.1 Actor Definitions

Two primary actors interact with the platform:

- **Authenticated user:** represents analysts, journalists, and researchers who ask cited questions in natural language, explore graph relationships, inspect documents, follow political trends, compare entities, and consult legislative or financial signals exposed by the application.
- **System administrator:** represents an elevated role responsible for monitoring service health, supervising users, launching and scheduling collection jobs, inspecting ingestion logs, and triggering backfill or maintenance operations on the document and vector indexes.

Both actors depend on traceability: the user must be able to verify evidence behind each answer, while the administrator must be able to verify that the corpus remains fresh, deduplicated, and processable by the extraction pipeline. These roles directly map to the functional requirements for reasoning, administration, and observability developed later in this chapter.

1.4.2 Use Case Diagram Overview

Figure 1.3 provides a consolidated view of the main use cases supported by the implemented frontend and backend modules. The diagram distinguishes analytical workflows (querying, graph exploration, document consultation) from operational workflows (ingestion control, scheduler management, system monitoring).

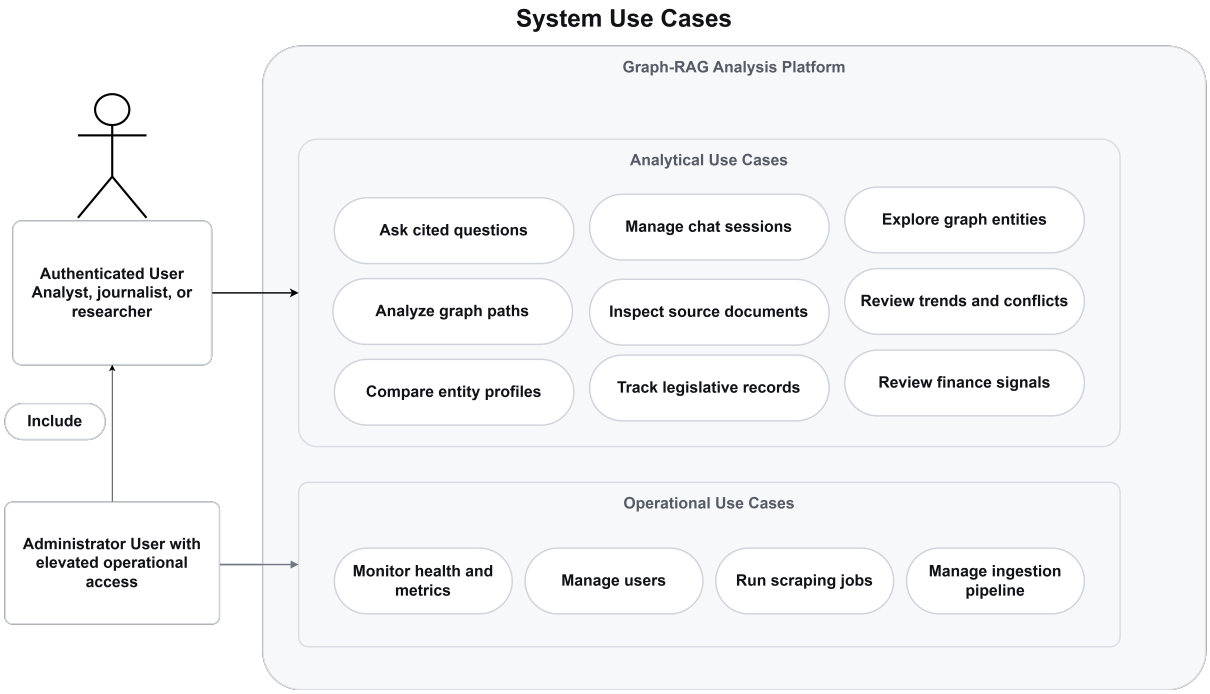


Figure 1.3: Overview of system use cases for authenticated users and administrators.

The diagram shows that query execution is not an isolated feature but part of a broader workspace that includes graph navigation, legislative and financial views, and administrative supervision. This breadth explains why the requirements are grouped not only around retrieval but also around ingestion, structuring, and platform operation.

1.4.3 Domain Class Model Overview

The domain class model complements the behavioral view of the use-case diagram. It identifies the principal information objects manipulated by the framework: source documents, chunks, entities, mentions, assertions, graph relations, users, sessions, and generated answers. The model also makes the provenance chain explicit, since every extracted fact and retrieved chunk remains connected to its original document.

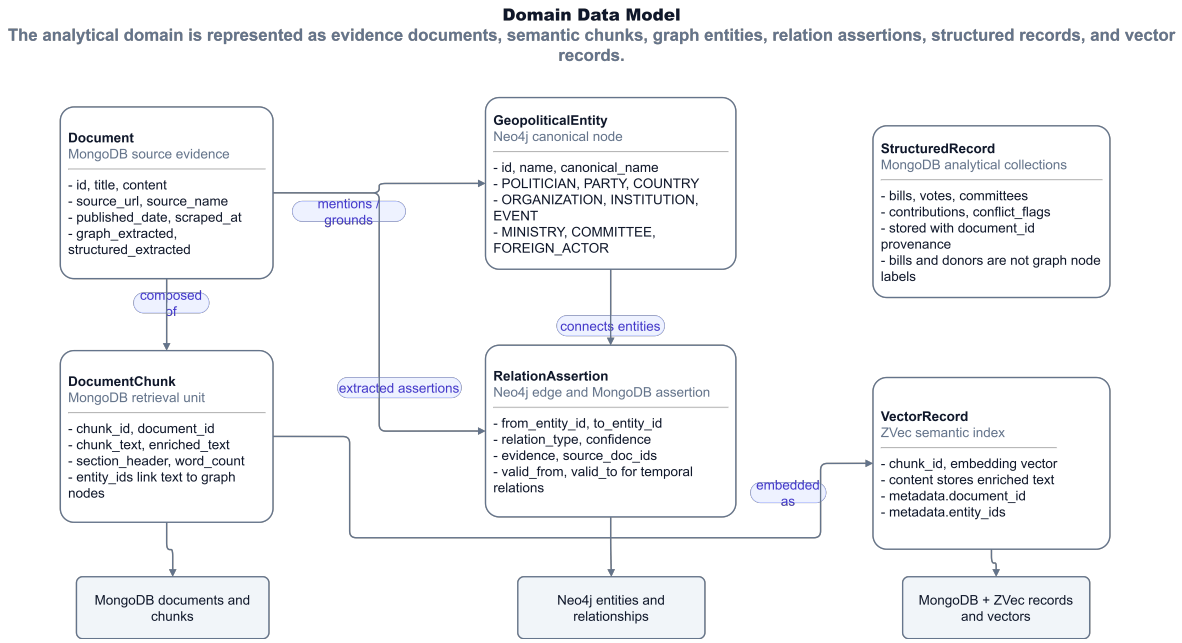


Figure 1.4: UML domain class model linking documents, extracted knowledge, retrieval evidence, and user interactions.

The class model remains conceptually reusable across domains. Domain adaptation mainly changes entity categories, relation types, and specialized structured records; document provenance, chunking, retrieval evidence, users, sessions, and answer objects remain part of the common framework. Together, the use-case diagram and the class model provide the basis for the narrative use cases below.

1.4.4 Key Use Case Narratives

Four main narrative use cases can be defined for this project: cited analytical questioning, graph and document exploration, supervised ingestion, and temporal or comparative analysis. They summarize the behavior expected from the system before the requirements are stated in formal terms.

Cited analytical questioning. An authenticated user submits a multi-hop political question through the chat interface and selects a reasoning depth (standard retrieval, decomposed multi-hop, or reflexive audit). The system retrieves semantic chunks and graph facts, synthesizes a cited answer, and exposes the reasoning trace. Success depends on grounded evidence, source links, and acceptable latency for the selected mode.

Graph and document exploration. The user inspects entity neighborhoods in the graph explorer or opens documents in the library to verify extracted relations and original wording. This use case ensures that conversational synthesis remains connected to inspectable artifacts rather than replacing them.

Supervised ingestion. An administrator triggers topic-based collection, direct URL ingestion, or scheduled scraping, then monitors logs, document counts, and index status from the administration dashboard. Failures must be visible without code changes so that corpus freshness remains under operational control.

Temporal and comparative analysis. The user consults trend, conflict, finance, or comparison views to interpret entity evolution over time. This narrative depends on temporal metadata, stable entity identifiers, and consistent graph publication after extraction.

1.5 Functional Requirements

The use cases above show that the platform must support both analytical work and operational supervision. Functional requirements are split into four stages: Data acquisition and ingestion stage, extraction and structuring stage, retrieval and reasoning stage, and user interaction and administration stage. Table 1.1 summarizes the main 11th high priority functional requirements belonging to the three first stages. There are many other functional requirements that are not presented in the table, which have less priority. In the following, each stage component is developed.

1.5.1 Data Acquisition and Ingestion Stage

There are four components in the data acquisition and ingestion functional requirements:

FR-01 (Autonomous ingestion). The platform shall employ autonomous multi-modal ingestion capable of continuous discovery and normalization of heterogeneous political sources. Crawlers and schedulers must operate without manual reconfiguration for each new document batch, so that the corpus grows under platform control rather than through ad hoc file uploads.

FR-08 (Autonomous discovery). Collection shall extend beyond a fixed registry through LLM-guided topic expansion or administratively triggered discovery jobs. This requirement ensures that emerging sources or thematic priorities can be incorporated when analysts or supervisors identify new monitoring needs.

FR-09 (Domain-agnostic ingestion). The ingestion layer shall remain decoupled from extraction logic specific to a domain. New source types, regions, or formats must be integrable by adjusting acquisition and normalization rules only, without rewriting the reasoning engine or frontend contracts.

FR-10 (Heterogeneous normalization). RSS feeds, HTML pages, direct URLs, and PDF reports shall be transformed into unified document records with consistent metadata

ID	Pipeline stage	Requirement
FR-01	Acquisition & ingestion	Autonomous multi-modal ingestion and continuous corpus growth
FR-08	Acquisition & ingestion	LLM-guided or administratively triggered source discovery
FR-09	Acquisition & ingestion	Ingestion decoupled from domain-specific extraction logic
FR-10	Acquisition & ingestion	Normalization of RSS, HTML, URLs, and PDFs to unified records
FR-02	Extraction & structuring	Joint entity and relation extraction in a single pass
FR-03	Extraction & structuring	Neo4j persistence with deterministic entity resolution
FR-06	Extraction & structuring	Temporal traceability of fact evolution where applicable
FR-04	Retrieval & reasoning	Hybrid ZVec vector retrieval and Neo4j graph traversal
FR-05	Retrieval & reasoning	Selectable RAG, chain-of-thought, and reflexive reasoning modes
FR-07	Retrieval & reasoning	Reasoning traces and precise source citations on every answer
FR-11	Retrieval & reasoning	Offline LLM-as-a-judge validation for extraction and responses

Table 1.1: Registry of high priority functional requirements

(identifiers, timestamps, provenance, and raw text). Normalization is the prerequisite for reliable extraction and for deduplication during graph publication.

Together, FR-01, FR-08, FR-09, and FR-10 ensure that the corpus is produced and supervised by the platform itself rather than assumed as an external input, which was identified earlier in this chapter as a central gap. They directly support the *supervised ingestion* use case for the administrator actor.

1.5.2 Extraction and Structuring Stage

This stage includes three components as follow:

FR-02 (Joint extraction). The system shall perform joint entity and relation extraction in a single pass over each normalized document, producing structured records aligned with the target political schema. A single-pass design reduces inconsistency between separately extracted entities and edges.

FR-03 (Graph storage). Structured outputs shall be persisted in a Neo4j knowledge graph using local deterministic entity resolution and stable canonical identifiers. Disambiguation must be reproducible so that repeated ingestion of syndicated content does not fragment one actor into multiple nodes.

FR-06 (Temporal traceability). Where publication metadata permits, the platform shall record fact evolution over time so that analysts can interpret when a relation became valid or changed. Temporal fields support trend, conflict, and comparative views described in the use-case narratives.

Extraction quality directly conditions graph reliability and therefore constrains every downstream retrieval and reasoning function. FR-02, FR-03, and FR-06 implement the structural layer on which graph exploration and multi-step questioning depend.

1.5.3 Retrieval and Reasoning Stage

In this stage, four main components are of high priority:

FR-04 (Hybrid search). The platform shall combine dense vector retrieval (ZVec) with typed graph traversal (Neo4j) in a single hybrid search layer. Semantic recall over chunks and relational precision over edges must be available within the same query workflow.

FR-05 (Multiple reasoning modes). The reasoning engine shall expose three modes: standard one-hop RAG, chain-of-thought decomposition across several steps, and reflexive agentic control. These modes shall be selectable at query time. Mode selection trades latency against analytical depth for the *cited analytical questioning* use case.

FR-07 (Attribution). Every synthesized answer shall include a reasoning trace and precise source citations linking claims to document spans or graph facts. Attribution is mandatory for analytical trust and for jury-facing demonstration of evidence grounding.

FR-11 (Automated validation). An offline LLM-as-a-judge validation path shall support quality checks on extraction outputs and generated responses during development and evaluation (Chapter 5). This requirement does not replace human review but provides repeatable scoring hooks.

FR-04 through FR-07 and FR-11 define the analytical core of the platform: hybrid evidence retrieval, selectable reasoning depth, auditable synthesis, and measurable quality assessment.

1.5.4 User Interaction and Administration Stage

An additional cross-functional stage is the user interaction and administration stage. The system shall provide interfaces for chat, graph exploration, document browsing, and dashboards, as well as an administration panel for managing ingestion, monitoring system status, and viewing logs. These interaction requirements implement the responsibilities of the authenticated user and system administrator defined earlier. They connect operational supervision to FR-01 and FR-08 and ensure that analytical outputs remain linked to inspectable artifacts (documents, graph neighborhoods, traces). The complete registry below consolidates the functional baseline used in the design and implementation chapters.

1.6 Non-Functional Requirements

1.6.1 Performance and Scalability

Graph traversals up to three hops should complete within approximately **500 ms** under nominal deployment conditions so that interactive analysis remains responsive. The architecture shall support growth in document volume and graph size without proportional loss of retrieval precision. Query latency, token consumption, and throughput shall be monitorable so that optimization targets can be enforced during evaluation (Chapter 5).

1.6.2 Reliability and Idempotency

Ingestion and graph publication shall be idempotent: repeated processing of the same source URL or document identifier must not create uncontrolled duplicate nodes or bias frequency statistics. Pipeline stages shall tolerate partial failures through explicit error states, retry policies, and administrative visibility rather than silent data loss. Worker execution, caching, and proxy layers shall preserve consistent behavior under concurrent load.

1.6.3 Security and Data Sovereignty

Processing shall occur within a **sovereign infrastructure** aligned with project and host-organization constraints: local or contractually governed storage for documents, vectors, and graph data; authenticated access to analytical and administrative functions; and controlled use of external LLM APIs with documented data-handling policies. Sensitive

corpora shall not depend on undisclosed third-party knowledge bases as the primary source of truth.

1.6.4 Observability and Auditability

The platform shall expose metrics, logs, and distributed traces across ingestion, retrieval, and synthesis so that bottlenecks and failures can be diagnosed under conditions close to production. Reasoning traces, citation metadata, and ingestion logs shall remain inspectable by authorized users. Observability satisfies both operational maintenance (administrator actor) and analytical trust (authenticated user actor). These functional and non-functional requirements guide the technology choices presented next.

1.7 Technology Stack and Selection Rationale

The stack was selected according to six criteria derived from the requirements: compatibility with AI and data tooling, support for heterogeneous evidence, graph and vector retrieval capability, sovereignty, deployment simplicity, and maintainability. No single database is forced to represent every information type; each component is assigned the responsibility for which its data model is most appropriate.

Table 1.2 presents the complete technology stack and the rationale behind each engineering choice. Among these components, Figure 1.5 highlights the representative application core. Python provides the common implementation language for ingestion, extraction, retrieval, and AI integration; FastAPI exposes these capabilities through typed asynchronous services; React delivers the interactive workspace for querying, graph exploration, and administration; and DSPy structures and optimizes the LLM-based knowledge extraction performed during ingestion. Together, these technologies connect the data and AI pipeline to a responsive user interface while preserving modularity between backend services, model orchestration, and frontend presentation. The next section explains how the work was organized and validated during the internship.

Figure 1.5: Representative technologies used by the proposed platform.

Layer	Selected technology	Reason for selection	Main alternative considered
Backend	Python and FastAPI	Strong NLP/AI ecosystem, typed validation, asynchronous APIs, and automatic OpenAPI documentation.	Flask offers simplicity but fewer built-in typing and API-contract features.
Frontend	React and Vite	Component reuse, mature visualization ecosystem, fast development server, and support for an interactive analytical workspace.	Server-rendered templates are simpler but less suitable for graph and streaming interactions.
Document store	MongoDB	Flexible schemas for heterogeneous documents, extraction artifacts, processing states, and provenance metadata.	A relational database would require frequent schema migrations for evolving records.
Graph store	Neo4j	Native property-graph model, Cypher traversal, typed relationships, and mature visualization and indexing tools.	Relational joins become cumbersome for variable-depth multi-hop traversal.
Vector store	ZVec	Local persistent semantic retrieval with a lightweight operational footprint aligned with sovereignty requirements.	Managed vector databases provide richer scaling but add services and external dependencies.
LLM orchestration	Mistral and DSPy	Governed model access, structured extraction modules, and systematic optimization of knowledge extraction during ingestion.	Direct prompt calls are faster to prototype but harder to optimize and evaluate systematically.
Cache and streams	Redis and SSE	Low-latency temporary state and simple server-to-client streaming of reasoning progress.	WebSockets provide bidirectional communication that is unnecessary for the main response stream.
Deployment	Docker Compose and nginx	Reproducible local deployment, service isolation, explicit configuration, and controlled networking.	Manual installation is harder to reproduce across development and target environments.
Observability	Prometheus, Grafana, Jaeger	Complementary metrics, dashboards, and distributed traces for an inspectable multi-service pipeline.	Application logs alone do not explain cross-service latency or failures.

Table 1.2: Technology stack and rationale for the principal engineering choices.

1.8 Requirements Validation Criteria

1.8.1 Evidence and Metrics

Operational validation was organized by requirement area. For each area, a criterion specifies the conditions that must be demonstrably satisfied before the area can be considered accepted. The expected evidence must list the artifacts that prove compliance during development and acceptance testing. The following criteria define the minimum observable outcomes expected during implementation:

Ingestion. New sources and formats must be processable without modifying the reasoning engine. Evidence includes ingestion logs, normalized MongoDB document records, and scheduler status visible from the administration dashboard. This criterion verifies modularity between acquisition and downstream analytics.

Extraction. Entities and relations must conform to the target schema and publish cleanly to Neo4j. Evidence includes structured JSON outputs, typed nodes and edges, and rejection or repair logs when parsing fails. Structural correctness is validated before answer-quality metrics are interpreted.

Retrieval. Queries must return both semantic chunks and, when applicable, graph paths supporting the question. Evidence includes ZVec hit lists, Neo4j traversal results, and citation metadata attached to retrieved evidence.

Reasoning. Multi-hop and mode-selected questions must yield grounded answers with inspectable traces. Evidence includes reasoning traces, cited synthesis text, and confidence or audit signals exposed in the chat interface.

Observability. Pipeline behavior must be inspectable throughout the full processing path under load. Evidence includes Prometheus metrics, Grafana dashboards, and Jaeger distributed traces across ingestion, retrieval, and synthesis services.

Table 1.3 summarizes these areas in compact form. Together, the criteria ensure that ingestion remains modular, that structuring and retrieval are verified before answer-quality evaluation, and that analytical trust rests on inspectable evidence rather than opaque generation alone.

on the other hand, the quantitative scores, such as lexical overlap, semantic similarity, and RAGAS style metrics, are reported in Chapter 5 against a curated benchmark.

Requirement area	Validation criterion
Ingestion	New sources and formats processable without modifying the reasoning engine.
Extraction	Entities and relations conform to the target schema and publish to the graph.
Retrieval	Queries return semantic chunks and applicable graph paths for the question.
Reasoning	Multi-hop and mode-selected questions yield grounded, cited answers.
Observability	Ingestion, retrieval, and synthesis behavior inspectable under load.

Table 1.3: Functional requirements validation criteria by area.

1.9 Project Management

1.9.1 Validation Methodology

Requirements were refined iteratively through user-story formulation, supervisor review, and incremental delivery of working modules. Each major function was associated with observable acceptance checks before being considered complete. Two complementary practices structured this process: **user stories**, which express stakeholder intent in testable form, and a **Kanban** workflow, which tracks how work items move from definition to completion. A **Gantt diagram** then situates these deliveries within the effective three-month execution cycle. This management approach connects the requirement baseline to concrete implementation milestones.

1.9.1.1 User Stories

A **user story** is a concise, actor-centered description of a desired system capability. It states *who* needs a function, *what* must be achieved, and *why* that outcome matters for analysis or operation. The standard template is: *As a* [*actor*], *I want* [*goal*], *so that* [*benefit*]. User stories differ from raw feature lists because they preserve the stakeholder perspective established in the actor definitions above and because each story can be paired with explicit acceptance criteria.

During the end-of-study internship, stories were drafted for the main analytical and administrative workflows. Representative examples include: *As an authenticated user, I want to ask a cited question in natural language, so that I can verify claims against source documents*; *As an administrator, I want to schedule and monitor ingestion jobs, so that the corpus remains current without manual intervention*; and *As an authenticated user, I want to explore graph neighborhoods around an entity, so that I can trace multi-hop political relations*. Supervisor review converted each story into one or more engineering tasks with measurable completion checks, linking intent to the validation evidence defined in Table 1.3.

1.9.1.2 Kanban Workflow

Kanban is a visual workflow method in which work items are represented as cards and organized according to their progress. During the end-of-study internship, the project was managed through the Trello board shown in Figure 1.6. The workflow used five states: *Backlog*, *To Do*, *Doing*, *Review*, and *Done*.

Tasks were first recorded and clarified in the backlog, then selected according to project priorities. Active work was moved to *Doing*, validated in *Review*, and transferred to *Done* once its acceptance conditions were satisfied. This organization provided continuous visibility into project progress and supported iterative planning throughout the internship.

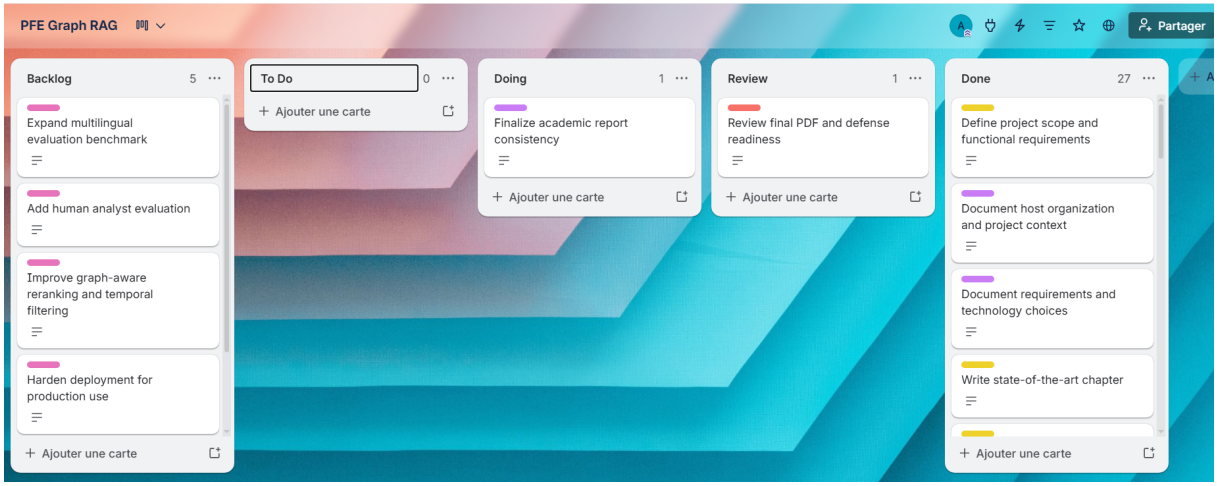


Figure 1.6: Trello Kanban board used to manage the project workflow.

Trello supported the continuous organization, prioritization, monitoring, and validation of project activities. Moving cards across the workflow made the current state of work visible and ensured that validation remained integrated into development rather than being postponed until the end of the project.

1.9.1.3 Project Management Tools

Modern agile teams often implement Kanban and communication practices through dedicated platforms. During the end-of-study internship, **Trello** provided the visual board for story cards, column transitions, due dates, and checklist-based acceptance criteria, while **Slack** supported synchronous coordination, file sharing, and rapid escalation when blockers affected ingestion or model access. Figure 1.7 identifies these tools as representative of contemporary project-management practice; they do not alter the requirement baseline itself but make task ownership, status, and review cycles auditable across the three-month timeline.

Figure 1.7: Trello (Kanban board) and Slack (team communication) used to manage internship tasks and coordination.

Trello translated each user story into a trackable card with assignee, labels (for example ingestion, retrieval, or frontend), and definition-of-done checklists. Slack complemented the board by documenting decisions that were too granular for card updates, such as API key provisioning, corpus expansion priorities, or demo feedback, so that requirement changes remained traceable without polluting the formal requirement tables in this chapter.

1.9.1.4 Execution Timeline (Gantt Diagram)

Figure 1.8 presents a **Gantt diagram** of the project execution timeline aligned with major requirement delivery milestones. Gantt charts express parallel and sequential activities on a shared calendar axis; they complement Kanban boards by showing *when* macro-phases occur, whereas Kanban shows *what* is currently in flight and *who* owns each item.

Figure 1.8: Gantt diagram of the end-of-study internship timeline aligned with major requirement delivery milestones.

The structured Gantt chart covers the effective three-month execution period of the internship, from the initial framing phase to the final evaluation and hardening phase. It is presented because these twelve weeks correspond to the period in which the main engineering deliverables were implemented, integrated, and validated. The task bars show that project framing, corpus acquisition, knowledge graph construction, vector indexing, and the first integrated retrieval pipeline occupied the initial month. Autonomous crawling, framework comparison, and the integration of the three reasoning modes followed during the second month. The final month concentrated on observability, MIPROv2 prompt optimization, concurrency, proxy configuration, and performance hardening. Dependency arrows make the delivery order explicit, while milestone markers identify the pipeline MVP, the integrated analytical platform, and the release ready for production. This sequencing ensured that evaluation and optimization were performed only after the evidence stores and full query path had reached a stable state. The validation criteria below define how this progress was assessed.

Conclusion

This chapter established the institutional, scientific, and technical context of the internship. The project need expressed by SCIENTIAE was translated into a defined problem, a set of objectives, identified actors, functional requirements, non-functional constraints, and validation criteria. This framing shows that the challenge is not limited to generating answers from documents, but also concerns traceability, relational analysis, temporal awareness, multilingual processing, and the ability to justify produced information. Once these requirements are clarified, the central issue becomes determining which existing approaches can support such needs, where their limitations remain, and how they can inform the design of a more suitable system.

Chapter 2

State of the Art

Introduction

This chapter reviews the concepts, research approaches, and existing solutions relevant to Graph-RAG. It first introduces large language models and the mechanisms used to retrieve textual evidence. It then examines information extraction, knowledge graphs, Graph-RAG strategies, reasoning methods, domain-specific constraints, and evaluation principles. The chapter concludes by comparing representative solutions and identifying the research gap addressed by the project.

2.1 Conceptual Foundations

2.1.1 Scope and Essential Terminology

The concepts reviewed in this chapter form one processing chain rather than a collection of independent technologies. Consider the question: *Which institutions connect a political actor to a recent decision, and what sources support that connection?* An LLM can formulate an answer, but it needs external evidence. RAG retrieves relevant passages; chunking defines the searchable units; embeddings represent their meaning; and a vector store finds semantically similar evidence. Information extraction then identifies actors, institutions, events, and relations. Entity resolution merges references to the same real-world object, while a knowledge graph stores the resulting connections. Graph-RAG combines the passages and graph paths, and a reasoning strategy determines whether one retrieval step is sufficient or whether further evidence is required.

Term	Plain definition	Role in Graph-RAG systems
LLM	A model that understands and generates natural-language text.	Interprets questions and synthesizes answers.
RAG	A method that retrieves external evidence before generation.	Grounds answers in collected documents.
Chunk	A small, searchable section of a document.	Provides the basic unit of textual evidence.
Embedding	A numerical representation of textual meaning.	Enables semantic comparison between questions and chunks.
Vector store	A system that indexes embeddings and retrieves nearby vectors.	Returns passages related to the meaning of a query.
Entity	A real-world object such as a person, institution, country, or event.	Becomes a node in the knowledge graph.
Relation	A typed connection between two entities.	Becomes an edge in the knowledge graph.
Knowledge graph	A structured network of entities, relations, and properties.	Supports explicit relational and temporal queries.
Graph-RAG	RAG that also retrieves graph entities and paths.	Combines semantic evidence with relational evidence.
Provenance	Information identifying where a fact originated.	Links answers and graph facts to inspectable sources.

Table 2.1: Essential terms used throughout the state-of-the-art chapter.

These definitions establish the vocabulary used to examine the main research approaches and existing solutions in the remainder of the chapter.

2.2 Grounded Retrieval

2.2.1 Large Language Models and Their Limits

Large Language Models (LLMs) are artificial intelligence systems trained on very large text corpora to understand, process, and generate language that resembles human writing. By learning statistical patterns, contextual dependencies, and semantic relationships from these corpora, they can support tasks such as question answering, translation, summarization, information extraction, and content generation [25]. Most contemporary LLMs are based on the Transformer architecture, whose attention mechanisms efficiently model relationships among tokens in a sequence [8].

General architecture. As illustrated in Figure 2.1, raw text first passes through tokenization, which converts it into discrete units. An embedding layer maps these tokens and their positions to dense numerical representations. Stacked Transformer blocks then alternate self-attention, which models dependencies among tokens, with feed-forward transformations that refine each representation. The output layer converts the final hidden states into task-dependent predictions, while training minimizes a loss function by updating model parameters through optimization [25].

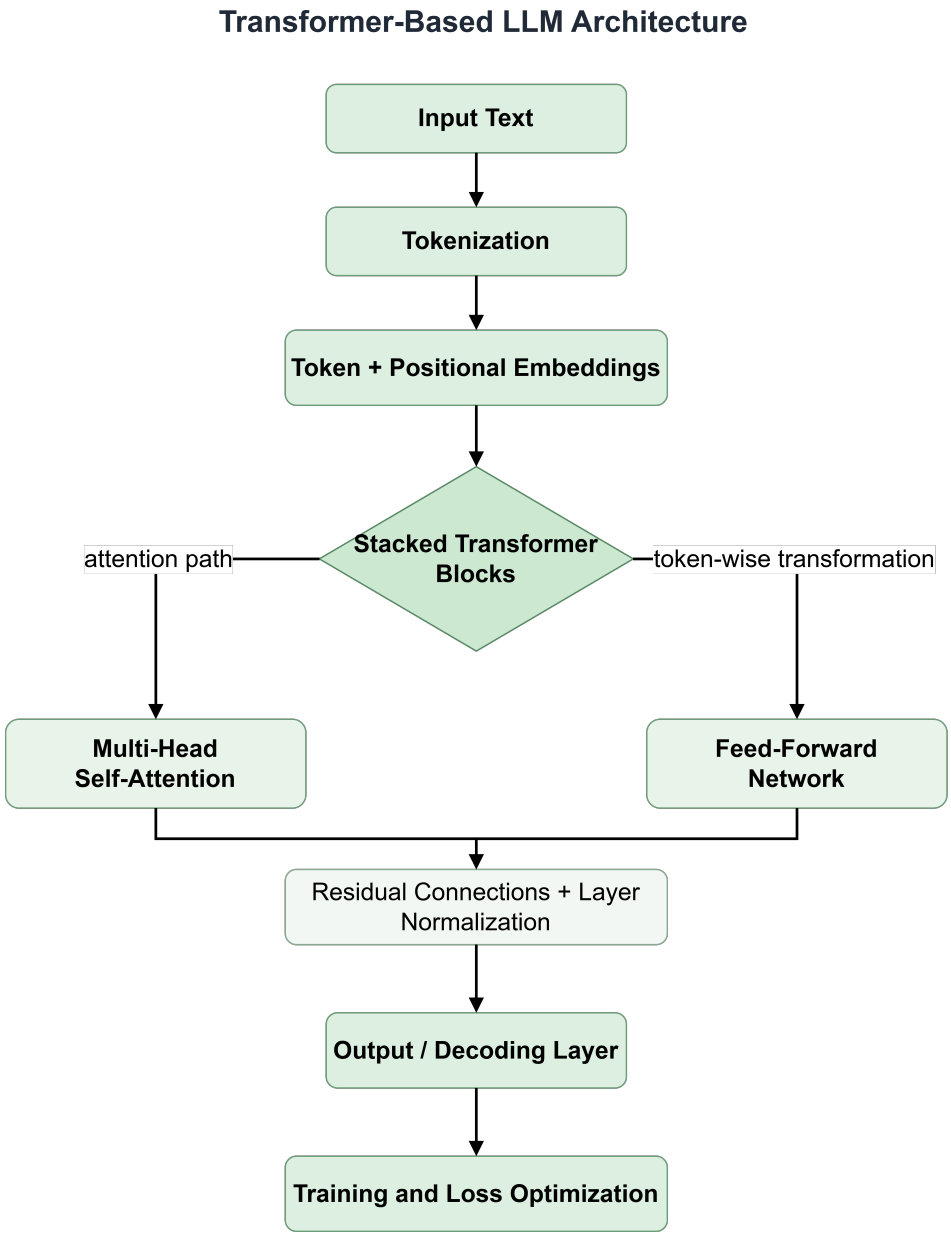


Figure 2.1: Simplified technical architecture of a Transformer-based large language model, adapted from [25].

Proprietary models may offer strong reasoning but raise privacy and sovereignty concerns, while open-weight models reduce vendor dependence at the cost of local

computing requirements. LLMs also require substantial training data and computation, can inherit biases from their source corpora, and may generate unsupported information. These limitations justify grounding generation in retrieved documents, graph facts, and explicit citations [1].

2.2.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) augments LLM generation with dynamically retrieved external knowledge [1]. The classical pipeline indexes documents by chunking and embedding them, retrieves the top-k relevant passages for a query, and conditions answer generation on that evidence. This workflow is efficient for direct factual questions but structurally limited for relational political analysis.

Recent extensions focus on **evidence grounding and citation**: retrieved passages are linked to sources, incomplete evidence triggers additional retrieval, and confidence checks reduce unsupported synthesis. These mechanisms do not replace RAG; they make generation auditable, which is essential when answers may influence interpretation of institutions, actors, or conflicts. Grounding also prepares the transition to graph-backed retrieval, where evidence includes explicit relations rather than isolated text spans.

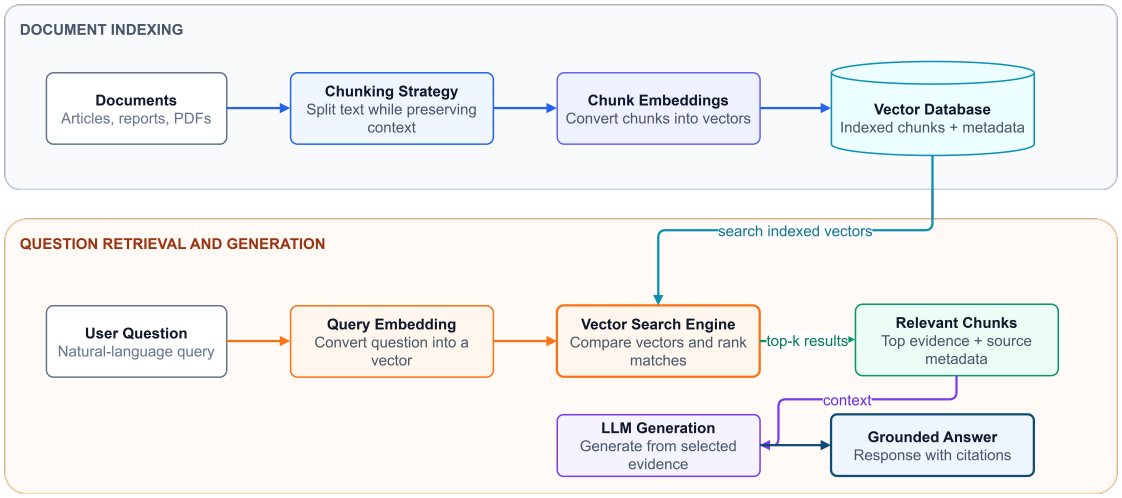


Figure 2.2: Core stages of a retrieval-augmented generation pipeline: indexing, retrieval, context construction, and grounded generation.

Figure 2.2 separates the offline and online responsibilities of RAG. During indexing, source documents are parsed, divided into retrievable units, embedded, and stored with metadata. During querying, the user question is encoded, relevant evidence is selected, and the resulting context is supplied to the language model. The quality of the final answer therefore depends on each upstream stage rather than on generation alone.

2.2.3 Searchable Evidence

2.2.3.1 Document Chunking

Definition and role. Chunking is the process of dividing a source document into smaller textual units before indexing. A RAG system generally does not embed an entire article or report as one vector because a long document may cover several topics and may exceed the context or input limits of the embedding model. Instead, each chunk becomes an independently searchable retrieval unit linked to its source document.

Chunk size directly affects retrieval behavior. Very small chunks can match a precise query but may omit the surrounding explanation needed to interpret the evidence. Very large chunks preserve context but can mix unrelated ideas, weaken similarity scores, consume more model context, and reduce the number of distinct sources that fit into one prompt. The objective is therefore not to maximize or minimize chunk length, but to preserve coherent evidence while keeping retrieval sufficiently precise.

Main strategies. Fixed-size chunking splits text after a predefined number of characters, words, or tokens. Overlapping chunking repeats part of the previous segment to reduce information loss at boundaries. Recursive chunking attempts progressively smaller separators, such as sections, paragraphs, sentences, and words. Structure-aware chunking follows document elements such as headings, paragraphs, lists, or pages. Semantic chunking groups neighboring sentences according to meaning rather than position alone. Table 2.2 summarizes these alternatives.

Strategy	Principle	Main strengths	Main limitations
Fixed-size	Split after a fixed number of characters, words, or tokens.	Simple, fast, predictable, and easy to reproduce.	Can cut sentences, arguments, dates, or relations at arbitrary boundaries.
Overlapping	Repeat a configurable portion of neighboring chunks.	Reduces boundary loss and preserves short-range continuity.	Creates duplicate content and increases index size and retrieval redundancy.
Recursive	Split by sections or paragraphs first, then use smaller separators when necessary.	Preserves natural text structure while respecting size limits.	Behavior depends on document formatting and separator quality.
Structure-aware	Use headings, paragraphs, lists, pages, or document markup as boundaries.	Produces interpretable chunks aligned with source organization.	Requires reliable parsing and varies across HTML, PDF, and other formats.
Semantic	Group sentences according to semantic similarity or topic shifts.	Better preserves coherent ideas and discourse units.	More computationally expensive and sensitive to the segmentation model.

Table 2.2: Comparison of common chunking strategies in RAG systems [2, 23, 24].

Context expansion and project relevance. Metadata should retain the source identifier, chunk position, section title, and links to adjacent chunks. A retrieval system can then search over compact units and selectively recover neighboring segments when a hit lacks sufficient context. This balances precision during initial retrieval with contextual completeness during answer synthesis. Chapter 4 describes the project-specific chunking configuration and adjacent-chunk expansion; the present discussion establishes the general design alternatives.

2.2.3.2 Embedding Models and Semantic Similarity

What is an embedding? An embedding is a numerical vector that represents the semantic content of a text. An embedding model transforms both document chunks and user queries into vectors located in the same multidimensional space. Texts with similar meanings should be positioned closer together even when they use different vocabulary. A vector database can therefore retrieve a passage about diplomatic sanctions for a query that uses related wording without requiring an exact keyword match.

Similarity search. Retrieval commonly compares a query vector $\mathbf{q}$ with a document vector $\mathbf{d}$ using cosine similarity:

$$\text{cosine}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \, \|\mathbf{d}\|}.$$

A higher score indicates a smaller angular distance and usually a stronger semantic match. The embedding model is therefore part of both indexing and querying: documents and questions must be encoded compatibly for their vectors to be meaningfully compared.

Selection criteria. Embedding models differ in vector dimension, language coverage, training data, inference cost, maximum input length, and domain specialization. Compact models reduce indexing time, storage, and query latency. Larger models may capture richer semantic distinctions but require more memory and computation. Multilingual models are important when the corpus contains English, French, Arabic, or transliterated names. Domain-specific models can improve specialized terminology, although they may generalize less effectively outside their training domain.

Model	Dim.	Language scope	Main strengths	Main limitations
`all-MiniLM-L6-v2`	384	Mainly English	Lightweight, fast, and economical for local indexing and baseline experiments.	Less suitable for multilingual corpora and subtle domain-specific distinctions.
`all-mpnet-base-v2`	768	Mainly English	Richer semantic representation for general English retrieval.	Higher inference, memory, and vector-storage cost.
`paraphrase-multilingual-MiniLM-L12-v2`	384	Multilingual	Compact cross-language retrieval option covering English, French, and Arabic.	Multilingual breadth may reduce specialization for a particular domain.
Production-compatible embedding service	Not disclosed	Multilingual	Integration with sovereign infrastructure and the deployed language-model ecosystem.	Exact model characteristics must be validated before scientific comparison.

Table 2.3: Representative embedding-model alternatives and their principal retrieval trade-offs.

Interpretation and project relevance. Vector dimension alone does not determine retrieval quality. Chunk coherence, corpus language, metadata, query formulation, reranking, and the match between the model's training distribution and the target documents can outweigh a nominal increase in dimensionality [2]. Chapter 4 presents the embedding models considered during implementation and explains the transition

from a compact baseline to a production-compatible multilingual service.

2.2.3.3 Vector Databases

Vector stores index dense representations and retrieve passages whose embeddings are close to the query even when their surface vocabulary differs. Their engineering differences concern more than raw search speed: persistence, metadata filtering, deployment complexity, distributed scaling, and integration with existing infrastructure all affect suitability for a RAG system. Table 2.4 compares representative options considered against the project's local-sovereignty and hybrid-retrieval requirements.

Tool	Type	Main strengths	Principal trade-offs
FAISS	Similarity-search library	Mature CPU/GPU indexes for efficient dense-vector search and experimentation.	Persistence, metadata filtering, and service APIs must be engineered separately.
Qdrant	Vector database	Payload filtering, persistent collections, and REST/gRPC interfaces suited to production services.	Introduces an additional managed service and its operational footprint.
Weaviate	Vector database	Schema support, hybrid retrieval, filtering, and a broad semantic-search ecosystem.	A comparatively broad platform when only a compact local retrieval layer is required.
Milvus	Distributed vector database	Multiple ANN indexes, hardware-aware acceleration, and horizontal scaling for very large collections.	Distributed deployment and operations may be excessive for a focused end-of-study internship deployment.
Chroma	Embedded / client-server vector database	Accessible developer workflow for local prototypes and small RAG applications.	Fewer large-scale operational controls than distributed production-oriented systems.
Elasticsearch / OpenSearch	Search platform with vector support	Combines lexical retrieval, filters, aggregations, and vector search in one search cluster.	Cluster administration and resource requirements are heavier than a dedicated local vector component.
ZVec	Local vector database	In-process persistence, metadata-linked chunk retrieval, and alignment with sovereignty constraints.	A smaller ecosystem and less operational maturity than long-established search platforms.

Table 2.4: Comparison of representative vector-retrieval technologies [16, 17, 18, 19, 20, 21, 22, 13].

The comparison explains why no vector technology is universally preferable. FAISS is a strong algorithmic library but not a complete persistence service; Qdrant, Weaviate, Milvus, Elasticsearch, and OpenSearch provide richer service-level capabilities at the cost of additional deployment complexity; and Chroma emphasizes ease of prototyping. ZVec is relevant to the proposed architecture because semantic retrieval can remain local

and lightweight while document provenance and explicit graph relations remain the responsibility of specialized stores.

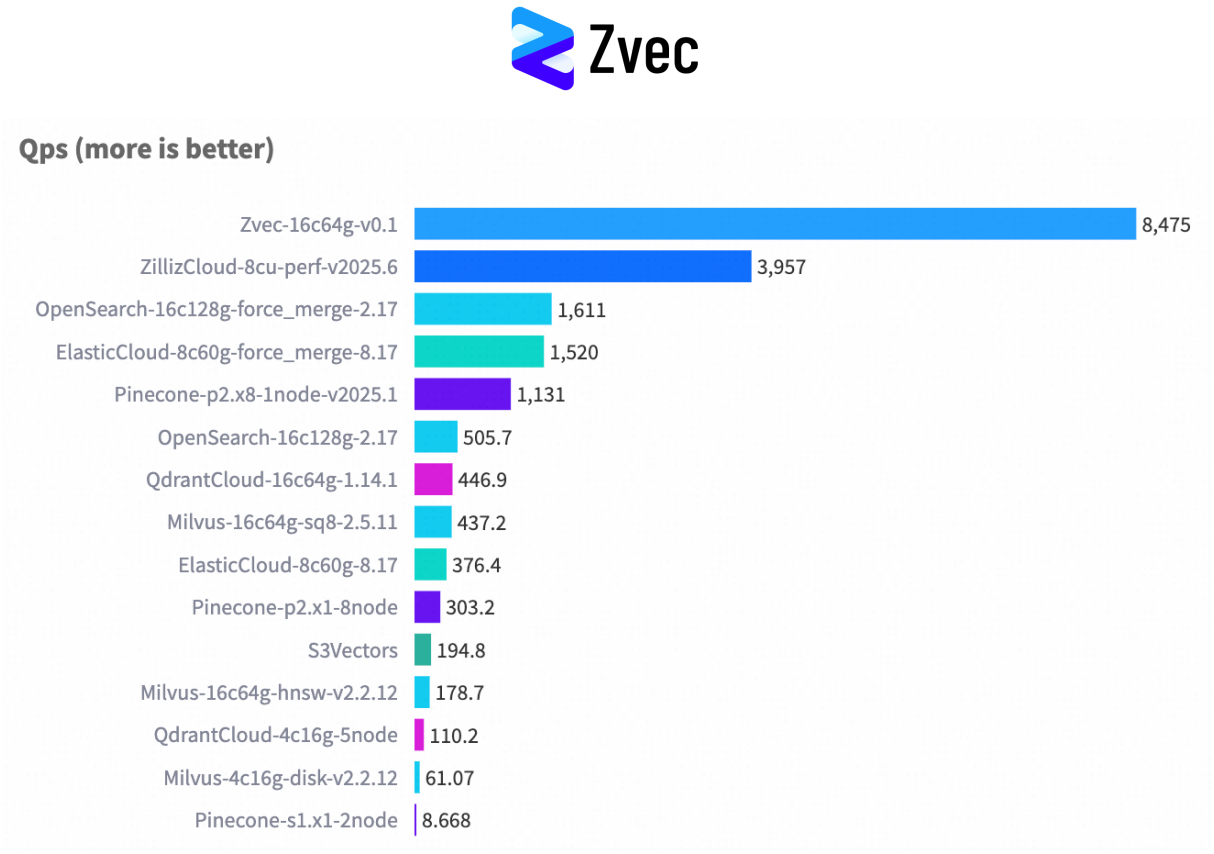


Figure 2.3: ZVec identity and official benchmark overview for vector retrieval at scale [13, 14].

Figure 2.3 complements the qualitative comparison with the benchmark overview published for ZVec. It indicates the performance characteristics that make an embedded vector engine relevant when low-latency retrieval and local persistence are required. These published results provide contextual evidence rather than measurements of the proposed platform; the concrete integration, indexing flow, and project-specific evaluation are therefore presented separately in Chapters 4 and 5.

2.2.4 Hybrid Retrieval and Reranking

Definition and role. Hybrid retrieval combines several evidence-selection mechanisms instead of relying on one similarity score. Dense retrieval identifies semantically related passages, lexical retrieval favors exact words and rare names, metadata filtering enforces constraints such as source, language, or date, and graph traversal recovers explicit relations between entities. Their candidate sets are fused before answer generation.

Reranking. The first retrieval stage is usually optimized for recall and returns a broader candidate set. A reranker then reorders or filters these candidates using stronger

relevance signals. These may include semantic similarity, lexical overlap, graph distance, temporal validity, source quality, deduplication, or a cross-encoder that jointly reads the query and passage. Reranking improves precision but adds latency and requires calibration because scores from vector, lexical, and graph retrieval are not directly comparable.

Project relevance. Political questions frequently combine exact entities and dates with broader semantic intent. Hybrid retrieval is therefore better suited than a single search mechanism: lexical signals preserve precise names, vectors recover paraphrases, and graph paths expose relations that span several steps. RAG surveys treat query rewriting, filtering, reranking, and context assembly as central retrieval stages rather than optional post-processing [2]. The concrete fusion rules used by the platform are specified in Chapters 3 and 4.

2.3 Structured Knowledge

2.3.1 Information Extraction and Entity Resolution

Entity and relation extraction. Knowledge construction begins by identifying entities, mentions, attributes, events, and relations in unstructured text. Rule-based approaches offer predictable behavior for stable patterns but require substantial maintenance when vocabulary or domains change. Supervised sequence-labeling and relation-classification models generalize better when annotated data are available, although separate extraction stages can propagate errors from entity detection to relation prediction. Transformer and LLM-based methods support joint or schema-guided extraction with less task-specific training data, but their outputs require structural validation because fluent generation does not guarantee factual or ontological correctness.

Approach	Principle	Strengths	Limitations
Rule-based extraction	Handcrafted patterns, dictionaries, and grammatical rules identify entities and relations.	Predictable, interpretable, and effective for stable formats.	High maintenance cost and weak adaptation to new wording or domains.
Supervised pipelines	Trained models perform named-entity recognition followed by relation classification.	Measurable task performance and efficient inference after training.	Requires annotated data and can propagate errors between stages.
Joint transformer models	A shared encoder predicts entities and relations in one model.	Reduces pipeline fragmentation and captures contextual dependencies.	Still depends on domain-labeled data and predefined relation schemas.
LLM-based extraction	Prompted or fine-tuned LLMs generate schema-constrained structured records.	Flexible schemas, few-shot adaptation, and broad language understanding.	Higher cost, output variability, and risk of unsupported relations.

Table 2.5: Comparison of representative entity and relation extraction approaches.

Entity resolution. Extracted mentions must be linked to canonical entities so that spelling variants, aliases, titles, and multilingual forms do not create fragmented graph nodes. Resolution may combine normalized names, contextual similarity, external identifiers, organizational roles, and temporal evidence. Deterministic rules improve reproducibility, while probabilistic or embedding-based methods improve recall on ambiguous mentions. In political corpora, the same surface name can refer to different people or institutions, and roles can change over time; resolution must therefore retain confidence, provenance, and the possibility of later correction.

Method	Evidence used	Strengths	Limitations
Exact normalization	Case folding, punctuation removal, aliases, and normalized strings.	Fast, deterministic, and easy to audit.	Misses ambiguous, translated, or context-dependent mentions.
Knowledge-base linking	Candidate identifiers and descriptions from an external knowledge base.	Provides canonical identifiers and reusable background knowledge.	Coverage, update frequency, and external dependence may be limiting.
Contextual similarity	Mention context is compared with candidate descriptions or previous mentions.	Handles aliases and ambiguous surface forms more effectively.	Requires thresholds and may produce opaque similarity errors.
Hybrid resolution	Combines deterministic rules, context, metadata, and optional human review.	Balances reproducibility, recall, and correction mechanisms.	More complex to calibrate and maintain across domains.

Table 2.6: Comparison of common entity-resolution strategies.

Graph quality and provenance. Extraction confidence alone is insufficient for a trustworthy knowledge graph. Pipelines must detect duplicates, reject malformed relations, preserve source spans, and represent contradictory or temporally superseded claims without silently overwriting them. Provenance connects every node or relation to the document and passage from which it was derived. This evidence chain supports graph repair, analyst verification, and cited generation, while temporal metadata distinguishes a current fact from a historically valid one.

2.3.2 Knowledge Graph Fundamentals

Knowledge graph fundamentals. A knowledge graph represents information as entities connected by typed relations. Entities become nodes, relations become edges, and both may carry properties such as names, dates, confidence values, source identifiers, or validity intervals. Unlike a flat document index, this representation makes relationships explicit and supports traversals such as identifying organizations connected to an actor through several intermediate events.

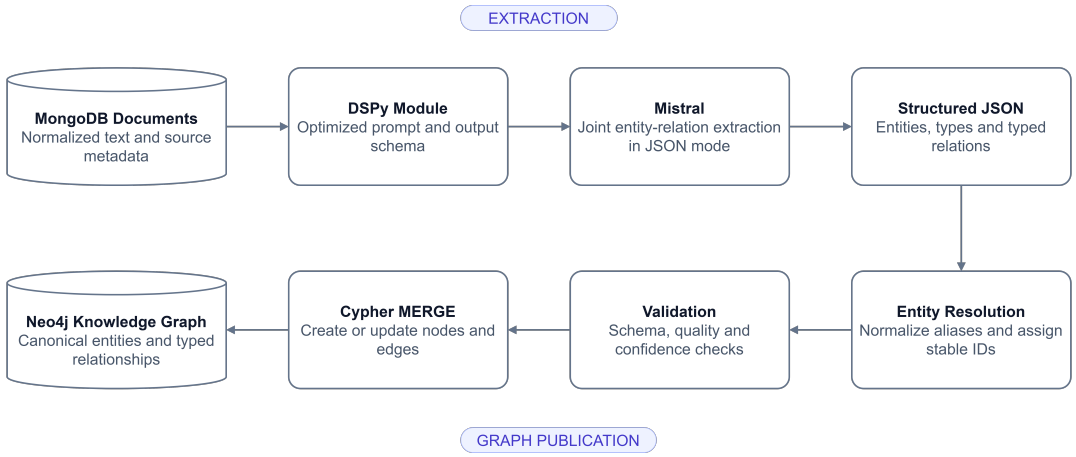


Figure 2.4: General process for constructing a knowledge graph from unstructured documents.

As shown in Figure 2.4, graph construction transforms documents into mentions, canonical entities, and validated relations before publication. An ontology or schema constrains the permitted entity and relation types, while provenance links the resulting facts back to their evidence. Graph queries can then filter by type, follow multi-hop paths, aggregate neighborhoods, and reason over temporal or organizational structures.

2.4 Graph-RAG and Reasoning

2.4.1 Classical RAG and Graph-RAG

Classical RAG retrieves passages through semantic similarity. Graph-RAG constructs or uses graph-structured knowledge and retrieves through entities, relationships, communities, and source passages [3, 11]. Figure 2.5 contrasts the two retrieval paths and shows why geopolitical questions often require relational traversal.

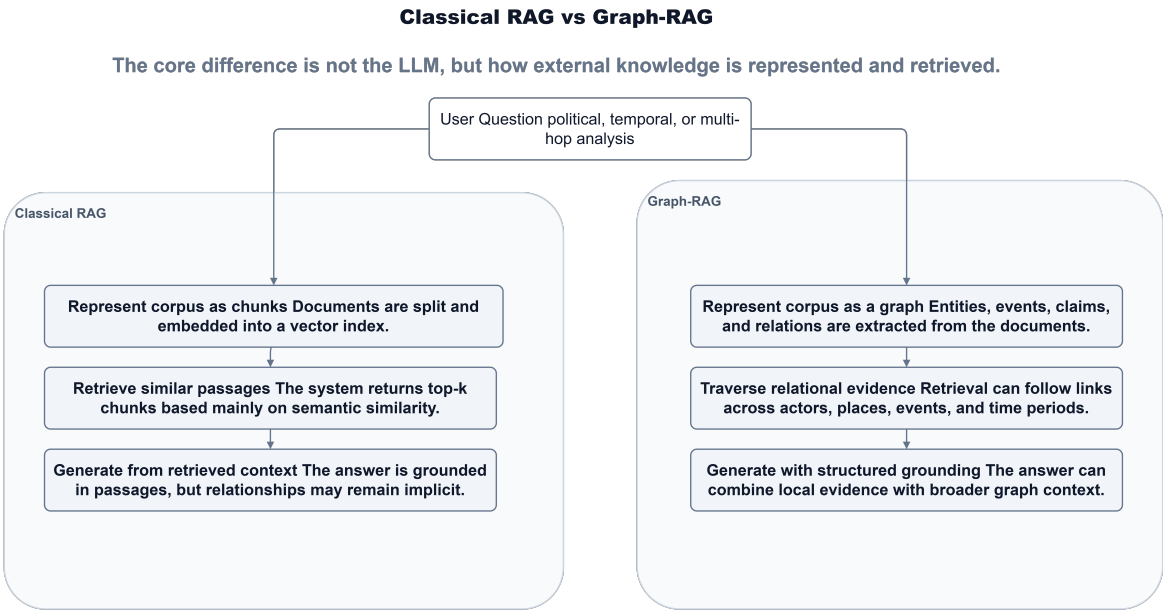


Figure 2.5: Comparison between classical RAG and Graph-RAG retrieval paths.

Graph-RAG retrieval and reasoning. Once published, the graph enables traversal across several relations, community summarization, and fusion with textual chunks retrieved by dense search [3, 11]. This combination is particularly relevant when an answer depends on a chain of actors and events rather than on a single paragraph. The graph supplies the explicit path, while the associated passages supply the wording, context, and provenance needed to verify it.

2.4.2 Graph-Based Retrieval Strategies

Strategy	Retrieval focus	Strengths	Limitations
Local Graph-RAG	Entity neighborhoods and short paths around query-linked nodes.	Precise evidence for entity-centric and multi-hop questions.	Can miss corpus-wide themes or disconnected relevant evidence.
Global and community Graph-RAG	Community summaries and hierarchical graph themes.	Supports broad questions requiring corpus-level synthesis [3].	Summary construction is costly and may hide minority evidence.
Hybrid Graph-RAG	Dense chunks are fused with graph nodes, paths, or communities.	Combines semantic recall with relational precision and provenance.	Requires score calibration, deduplication, and more complex orchestration.
Agentic Graph-RAG	A controller iteratively selects graph, vector, or external tools.	Adapts retrieval depth and tool choice to question complexity.	Higher latency, cost, and risk of unstable tool-selection loops.

Table 2.7: Comparison of representative Graph-RAG retrieval strategies.

2.4.3 Benefits and Limitations

Graph-RAG transfers part of the retrieval problem into graph construction and maintenance. Extraction errors can create incorrect paths, entity-resolution errors can merge unrelated actors or fragment one actor across nodes, and community summaries can hide minority or contradictory evidence. Graph construction also adds computational cost, storage, update logic, and ontology-management effort. As the source corpus evolves, stale relations and summaries must be refreshed while provenance and temporal validity remain intact. Graph-RAG is therefore most useful when relational retrieval justifies this additional complexity and when graph evidence remains connected to inspectable source passages.

2.4.4 Reasoning and Adaptive Retrieval

Beyond retrieval format, recent work studies how a language model *uses* evidence during generation. Classical RAG follows a mostly linear pipeline: retrieve once, then generate. This design is efficient for localized factual questions, but it is structurally weak when a query requires several retrieval steps, intermediate planning, or verification of partial evidence [1].

Chain-of-Thought and query decomposition. Chain-of-Thought (CoT) prompting asks the model to produce intermediate reasoning steps before the final answer [9]. In retrieval settings, decomposition splits a complex question into sub-questions and guides evidence collection step by step. The goal is not longer answers for their own sake, but better coverage when the first retrieved batch is incomplete.

Interleaved retrieval and reasoning across several steps. IRCoT interleaves retrieval with ongoing chain-of-thought generation: each reasoning step can trigger a new retrieval call guided by what was learned in the previous step [14]. This pattern is especially relevant for questions in which bridge entities only become visible after an initial pass. When knowledge is graph-structured, traversal across several steps plays a similar role by following explicit paths between actors and events rather than relying on lexical similarity alone.

ReAct and tool-interleaved control. ReAct combines reasoning traces with actions such as search, graph lookup, or tool invocation [13]. Instead of a fixed retrieve-then-generate sequence, the model alternates between thinking steps and operations on external memory. This architecture is a precursor to modern agentic pipelines in which a controller decides when to retrieve again, when to stop, and when to synthesize.

Reflection and Self-RAG. Reflection-based methods add a critique step that evaluates whether the current evidence supports the draft answer. Self-RAG trains the model to decide when retrieval is needed, when generation is sufficient, and when the output should be revised [15]. These approaches reduce unsupported synthesis at the cost of additional model calls and higher latency.

Agentic RAG. Agentic RAG generalizes the above patterns into loops that may include planning, routing, iterative retrieval, critique, and final synthesis. Surveys of the field describe a recurring trade-off: iterative and reflective pipelines improve faithfulness on relational or multi-step tasks, but they increase token cost and response time compared with naive RAG. For political intelligence, this literature motivates architectures in which reasoning depth can be adapted to question complexity rather than forcing every query through the heaviest pipeline.

Method	Core mechanism	Main benefit	Main trade-off
Chain-of-Thought	Generates intermediate reasoning steps before answering [9].	Improves decomposition of complex questions.	Reasoning can remain disconnected from new evidence retrieval.
IRCoT	Alternates reasoning steps with additional retrieval [14].	Recovers bridge evidence for multi-hop questions.	Requires several retrieval and generation cycles.
ReAct	Interleaves reasoning with explicit tool actions [13].	Supports search, graph lookup, and observable actions.	Tool-selection errors can propagate through the trajectory.
Self-RAG	Reflects on retrieval need, evidence quality, and generated output [15].	Improves evidence sufficiency and self-correction.	Adds model calls, training complexity, and latency.
Agentic RAG	Plans, routes, retrieves, critiques, and stops dynamically.	Adapts reasoning depth to the task and available tools.	Highest orchestration complexity and operational cost.

Table 2.8: Comparison of reasoning and control strategies used in retrieval-augmented systems.

Chapter 4 presents the reasoning modes implemented in the platform and describes how these ideas are operationalized in production.

2.5 Domain Context and Evaluation

2.5.1 Political and Temporal NLP

Political NLP extends language understanding to actors, institutions, stances, conflicts, and temporal relations. Joint entity-and-relation extraction reduces error propagation, stance analysis identifies how actors relate to events or entities [11], and temporal knowledge graphs attach validity intervals to evolving facts [12]. These capabilities explain why political intelligence requires extraction, temporal modeling, and retrieval rather than a single summarization model.

Political text also presents ambiguity specific to the domain. Statements may report allegations rather than established facts, sources may express conflicting positions, and institutional roles may change while names remain constant. Systems must distinguish

the author of a claim from the actor described by it, preserve publication dates separately from event dates, and avoid treating repeated syndicated reports as independent confirmation. Multilingual coverage further complicates entity aliases, transliteration, and stance interpretation across English, French, and Arabic sources.

2.5.2 Evaluation of Retrieval-Augmented Systems

RAG evaluation must distinguish retrieval failure from generation failure. Retrieval-oriented measures examine whether relevant evidence was returned, commonly through precision, recall, or ranked-relevance analysis. Generation-oriented measures assess whether the answer addresses the question, remains supported by the retrieved context, and correctly reflects a reference answer. Citation evaluation adds another layer by checking whether cited passages actually support the claims attributed to them.

RAGAS formalizes this multidimensional view through metrics such as faithfulness, answer relevancy, context precision, context recall, and answer correctness [4, 12]. These measures are useful because a fluent answer can still be unsupported, while a weak final answer may originate from insufficient retrieval rather than from the generator itself. Automatic LLM-based judges make repeated evaluation practical but introduce model bias and should be complemented by transparent benchmark construction, error analysis, and human review for high-stakes domains.

For Graph-RAG, evaluation should additionally test coverage across several reasoning steps, entity resolution, path validity, provenance completeness, temporal consistency, and the trade-off between cost and latency in iterative reasoning. A system may improve relational answer quality while increasing response time or model calls; reporting only final answer accuracy would hide this operational trade-off.

2.6 Existing Solutions and Research Gaps

Existing solutions illustrate how the concepts reviewed above are combined in research and operational systems.

2.6.1 Representative Systems

2.6.1.1 Classical RAG

The original RAG architecture combines dense passage retrieval with sequence generation for tasks that require external knowledge [1]. Its principal contribution is to separate parametric knowledge, stored in model weights, from non-parametric knowledge, retrieved from an external corpus. This approach supports evidence updates

without retraining the generator, but relationships distributed across several documents remain difficult to recover through passage similarity alone.

2.6.1.2 Microsoft GraphRAG

Microsoft GraphRAG extracts entities and relations, constructs a graph, detects communities, and generates hierarchical summaries for local and global questions [3, 11]. Local search focuses on entities and their neighborhoods, whereas global search uses community summaries to answer corpus-level questions. The approach improves broad relational synthesis but introduces substantial indexing, summarization, and graph-maintenance costs.

2.6.1.3 Neo4j Graph-Based Retrieval

Neo4j provides a property-graph database and the Cypher query language for representing and traversing typed relationships [5]. In Graph-RAG architectures, it can store canonical entities, provenance, and explicit paths while a separate vector index retrieves unstructured passages. Its flexibility supports domain-specific schemas and precise traversal, but extraction, entity resolution, retrieval orchestration, and answer generation must be designed around the database; Neo4j alone is not a complete Graph-RAG solution.

2.6.1.4 Wikidata

Wikidata is a collaboratively maintained knowledge graph that exposes structured facts through SPARQL [6]. It demonstrates the value of canonical identifiers, typed properties, and structured queries at large scale. However, its curated and general-purpose model differs from a continuously ingested, organization-specific document corpus, and conversational answer generation requires an additional application layer.

2.6.1.5 GDELT

GDELT continuously monitors global news and publishes structured event and knowledge data, including the Global Knowledge Graph [1, 2]. It is particularly relevant to political analysis because it captures actors, events, locations, themes, and media signals over time. Its broad coverage enables large-scale trend analysis, although its predefined coding and global scope do not replace domain-specific extraction, source validation, or tailored reasoning.

These solutions address complementary parts of the problem. Their differences become clearer when they are evaluated using common retrieval, reasoning, evidence, temporal, and operational criteria.

2.6.2 Comparative Analysis

Table 2.9 compares the corresponding approach families along criteria relevant to political intelligence. The comparison does not claim that one row is universally superior; it reveals their structural trade-offs.

Criterion	Classical RAG	MS GraphRAG	Knowledge Graph	Agentic RAG
Multi-hop reasoning	Limited	Yes	Yes	Yes
Entity disambiguation	Limited	Yes	Yes	Depends on tools
Temporal reasoning	No	Partial	Yes	Partial
Source citation	Yes	Yes	Partial	Yes
Structured queries	No	Limited	Yes	Depends on tools
Explicit provenance	Passage level	Passage and graph	Fact level	Tool dependent
Multilingual support	Model dependent	Model dependent	Schema dependent	Model dependent
Adaptive reasoning	No	Limited	No	Yes
Autonomous ingestion	External pipeline	External pipeline	External pipeline	Possible

Table 2.9: Comparative summary of major approach families across retrieval, reasoning, and evidence-management criteria.

Classical RAG supports passage-level citation but lacks explicit multi-hop and temporal modeling. Microsoft GraphRAG improves structured retrieval and graph-backed summarization but assumes that ingestion and graph maintenance are handled by surrounding processes. Pure knowledge graphs excel at structured, provenance-aware, and temporal queries but do not, by themselves, provide conversational synthesis. Agentic RAG adapts retrieval actions to the question but depends strongly on the quality of its tools and can increase latency and cost. No approach family satisfies every criterion by default; the remaining challenge is to integrate ingestion, hybrid retrieval, graph reasoning, provenance, and grounded generation in one auditable workflow.

2.6.3 Research Gaps

The central research gap is the inability of classical retrieval systems to represent political information as evolving relational structures. Semantic search can surface topically similar documents, but it does not explicitly model who is connected to whom, through which event, under which institutional role, and during which temporal interval.

The literature indicates a need for an integrated approach in which semantic retrieval over unstructured documents is combined with explicit relational retrieval, provenance, and adaptable reasoning depth. Compared with Table 2.9, the project does not claim

universal superiority; it investigates a combination that remains insufficiently covered by the compared approach families:

- **Hybrid evidence:** joint use of vector chunks, graph relations, and document provenance in the same query workflow.
- **Supervised ingestion:** autonomous collection with operational control, rather than a static benchmark corpus only.
- **Adaptable reasoning depth:** retrieval effort can be adjusted to the complexity of the question rather than applying the same process to every query.
- **Domain-aware analysis:** multilingual, temporal, and potentially contradictory political information is treated as a central requirement.

From this perspective, the project contributes along three axes:

- **Methodological contribution:** studying how graph-structured evidence improves faithfulness, explainability, and temporal consistency of LLM-based political analysis.
- **Technical contribution:** combining ingestion, hybrid retrieval, graph publication, and reasoning modes in one platform grounded in provenance.
- **Operational contribution:** delivering a complete workflow in which analysts query evidence, inspect sources, explore graph neighborhoods, and supervise ingestion without modifying backend code.

This positioning establishes the scientific motivation for the proposed work. The architecture and technology choices used to implement it are presented separately in Chapters 3 and 4.

Conclusion

This chapter reviewed the conceptual and technical foundations required to understand retrieval-augmented and graph-based generation. It began with the limits of large language models, then examined grounded retrieval, searchable evidence construction, information extraction, knowledge graphs, Graph-RAG strategies, reasoning methods, domain-specific constraints, and evaluation principles. The analysis of representative systems such as classical RAG, Microsoft GraphRAG, Neo4j-based retrieval, Wikidata, and GDELT shows that existing solutions address important parts of the problem, but do not jointly provide continuous ingestion, explicit provenance, multilingual and temporal analysis, adaptive reasoning, and auditable synthesis. These observations define the

research gap: how can these complementary capabilities be combined into a coherent architecture that remains operational, explainable, and evaluable?

Chapter 3

System Design and Architecture

Introduction

This chapter translates the requirements defined in Chapter 1 into a concrete system design for the political Graph-RAG platform. The objective is to specify how evidence is acquired, structured, stored, retrieved, and synthesized without yet entering implementation-level code detail (Chapter 4) or empirical evaluation (Chapter 5). The presentation proceeds from design principles and refined use cases to data flows, ingestion architecture, knowledge representation, physical schemas, and the reasoning loop that connects hybrid retrieval to grounded answers.

3.1 Design Principles

Four principles guided architectural decisions throughout the end-of-study internship: separation of concerns, hybrid evidence representation, traceability by design, and evolvability with modularity. Together they ensure that each pipeline stage can be inspected, replaced, or extended independently while preserving analytical trust.

Separation of concerns. Acquisition, normalization, extraction, graph publication, vector indexing, retrieval, and synthesis are isolated into distinct services and storage roles. The frontend exposes analytical and administrative routes separately; the backend groups API endpoints by responsibility (ingestion, query, administration, health). Figure 3.1 summarizes the major component boundaries and their interactions across the offline and online paths. Ingestion workers, extraction services, persistence connectors, and the reasoning engine communicate through well-defined interfaces rather than

shared mutable state. This separation supports the modularity required by FR-09 and allows partial pipeline reruns when only one derived artifact changes.

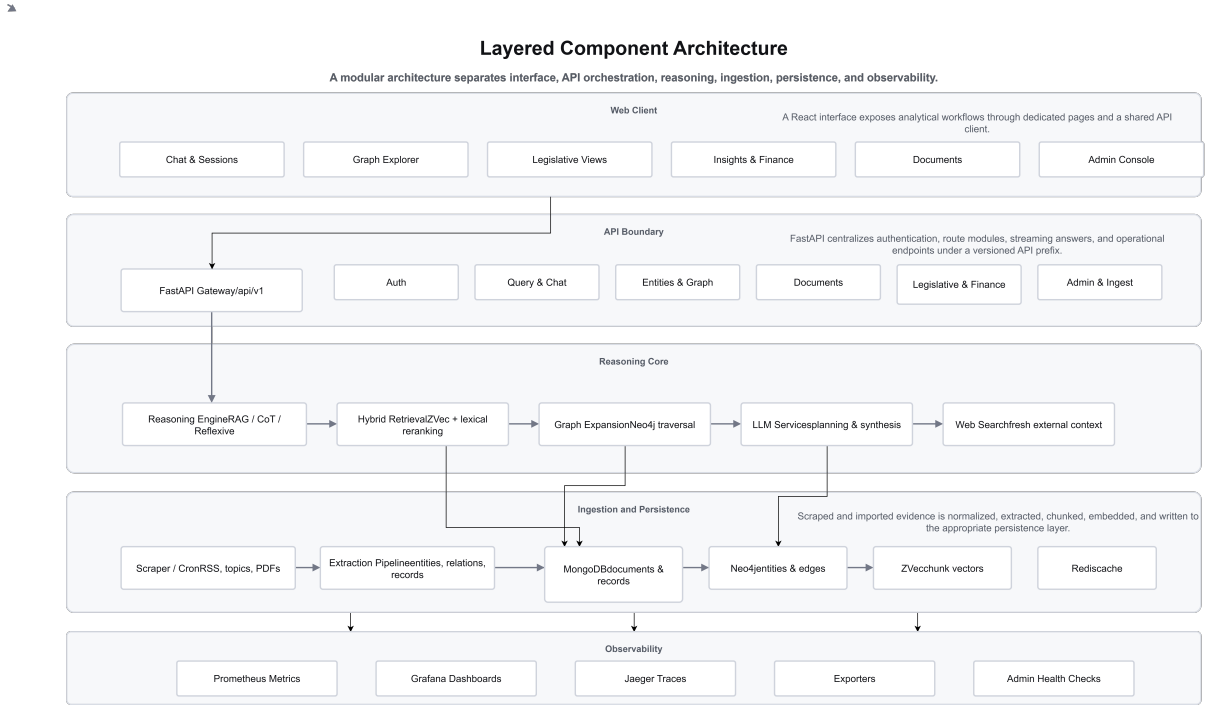


Figure 3.1: Logical component architecture and separation of pipeline responsibilities.

With the main service boundaries established, the next design decision concerns the form in which evidence is represented and retrieved. The platform cannot rely on a single retrieval path because political questions often combine broad topical relevance with explicit relationships between actors, institutions, and events.

Hybrid evidence representation. Political analysis requires both unstructured textual recall and structured relational precision. The design therefore maintains parallel representations: document-centric records and chunks for semantic search, canonical entities and typed edges for traversal, and structured analytical records (legislative, financial, conflict-oriented) where schema-specific views add value. Figures 3.2 and 3.3 present these two paths in a compact form before their outputs are combined.

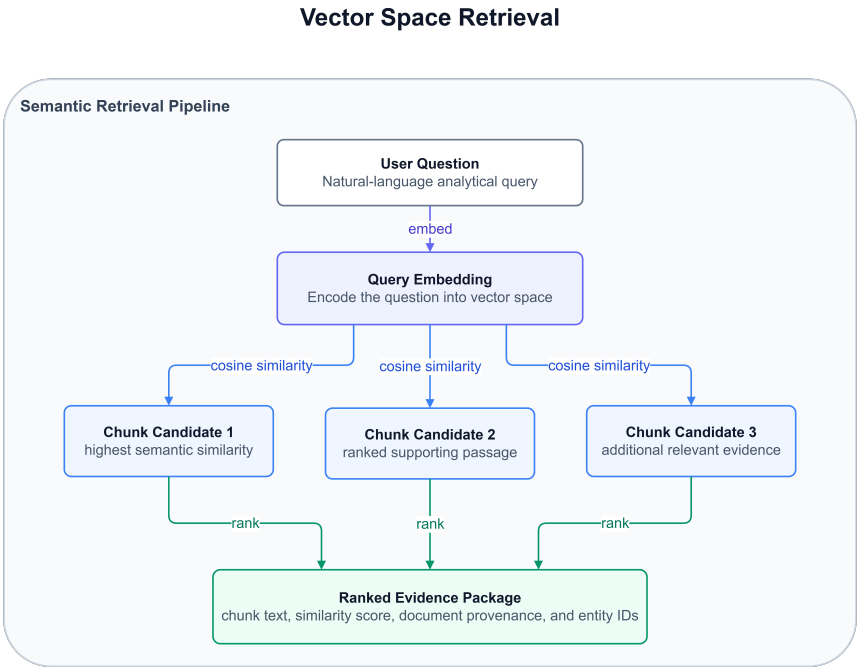


Figure 3.2: Vector-space retrieval from question embedding to ranked evidence chunks.

Vector-space retrieval. As illustrated in Figure 3.2, the user question is encoded into the same dense vector space as the indexed document chunks. ZVec compares the query embedding with stored vectors, ranks candidates by semantic similarity, and returns the strongest passages together with their scores, document provenance, and associated entity identifiers. This path provides efficient topical recall even when the query and source passages use different wording. Its output remains passage oriented: it identifies relevant textual evidence but does not, by itself, reconstruct explicit relations that span several entities or documents. For this reason, semantic retrieval is complemented by graph traversal.

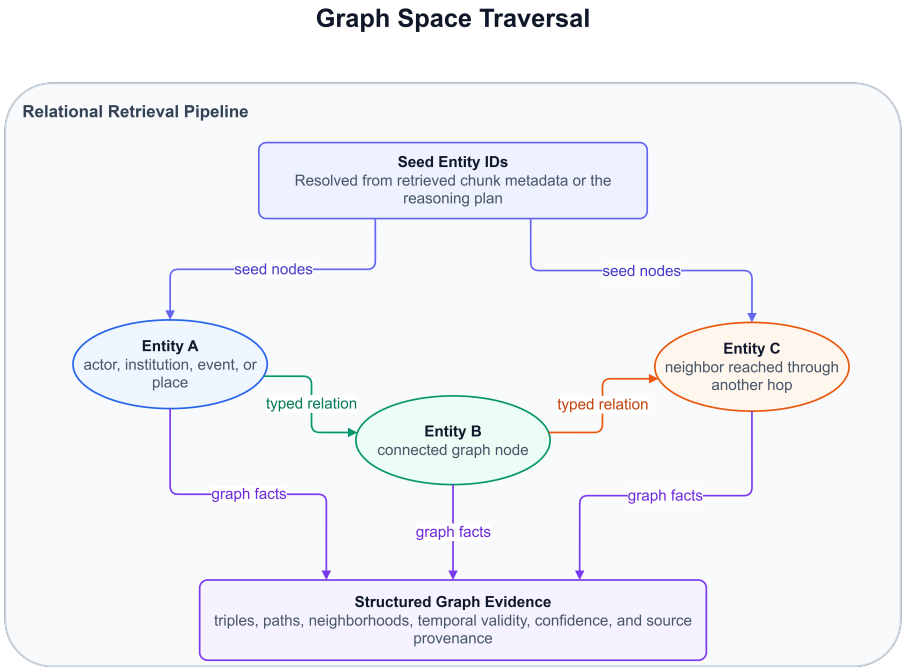


Figure 3.3: Graph-space traversal from seed entities to structured relational evidence.

Graph-space traversal. Figure 3.3 shows how entity identifiers obtained from chunk metadata or the reasoning plan become traversal seeds in Neo4j. The retrieval engine follows typed edges to neighboring actors, institutions, events, and places while preserving temporal validity, confidence, and source provenance. The resulting triples, paths, and neighborhoods make relationships explicit and support questions that require entity disambiguation, chronology, or several reasoning hops. Since each path captures a different form of evidence, the architecture requires a fusion stage before answer generation.

Evidence fusion. The two paths are complementary rather than competing. Vector retrieval contributes topical similarity and passage-level grounding, whereas graph traversal contributes relational precision and structural context. The reasoning layer combines ranked chunks with graph facts before synthesis so that generated answers remain both semantically relevant and structurally supported. Neither store alone satisfies FR-04; their fusion implements this hybrid retrieval requirement.

Figure 3.4 makes this combination explicit. Independently retrieved passages, graph paths, and structured records are normalized into an evidence package, deduplicated, temporally filtered, and reranked before synthesis. This intermediate fusion stage prevents the language model from receiving disconnected retrieval outputs and preserves provenance for every accepted claim.

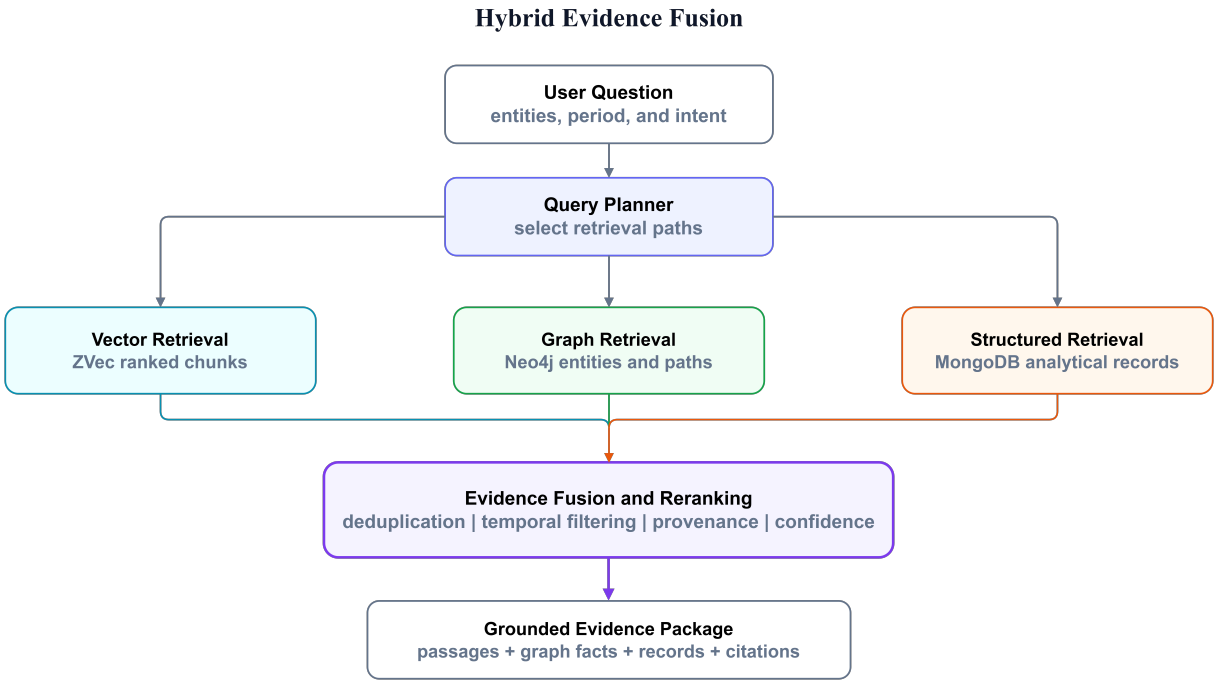


Figure 3.4: Hybrid evidence fusion across vector, graph, and structured retrieval paths.

Traceability by design. Every analytical output must remain linked to inspectable artifacts: source documents, chunk identifiers, graph nodes and edges used in traversal, and reasoning-step metadata exposed in the interface. Traceability is enforced at the data-model level (provenance fields on documents and assertions) and at the reasoning level (cited spans, graph-fact references, and optional audit loops in reflexive mode). This principle implements the validation criteria for reasoning and observability defined in Chapter 1.

Evolvability and modularity. Domain ontologies, extraction prompts, source connectors, and embedding settings may evolve during the internship and in later deployments. The architecture therefore treats MongoDB documents as the system of record for raw evidence and processing checkpoints, while Neo4j and ZVec hold derived views that can be rebuilt from documents without re-scraping sources. Containerized services and horizontal scaling of stateless workers (ingestion, API, reasoning) support growth in corpus size without redesigning the logical topology illustrated in Figure 3.5. Observability (metrics, logs, traces) is treated as a cross-cutting concern rather than an afterthought, consistent with the non-functional requirements on auditability and operational diagnosis.

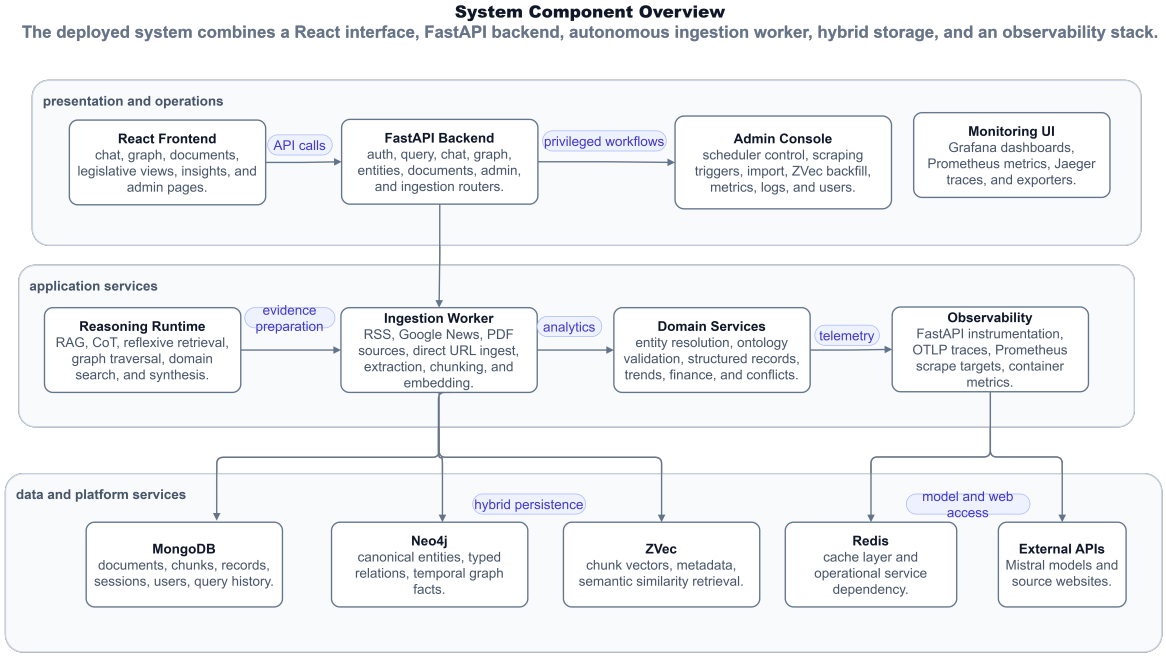


Figure 3.5: System component overview for ingestion, persistence, reasoning, and observability.

3.2 Use Case Refinement

Chapter 1 introduced actors and primary use cases at requirement level. The detailed model refined here guided API and frontend boundaries. Authenticated users interact through cited question answering (with selectable reasoning depth), chat history, graph exploration, document library consultation, legislative and financial dashboards, trend and conflict views, and entity comparison. Each analytical workflow consumes prepared evidence from MongoDB, Neo4j, and ZVec rather than triggering full ingestion at query time; live web supplementation applies only when configured for recent events not yet indexed locally.

The system administrator supervises scheduler configuration, scraping and backfill jobs, ingestion log inspection, user management, health metrics, and selective reindexing of derived artifacts. Operational routes are isolated from standard analytical navigation through role-based access control in the frontend and privileged backend endpoints, so infrastructure actions are not conflated with everyday analytical sessions. Access control is modeled as a strict boundary between *user* and *administrator* roles: analytical capabilities require authentication, while administrative capabilities require an elevated role verified on both client and server.

Figure 3.6 formalizes the use cases attached to each side of this boundary and aligns them with the implemented route structure. The model clarifies that graph exploration and document inspection are peer workflows to chat-based questioning, not secondary

features. Administrative use cases (scheduler control, scraping supervision, reindexing) operate on the same evidence pipeline but must not bypass quality gates or idempotency rules defined for ingestion.

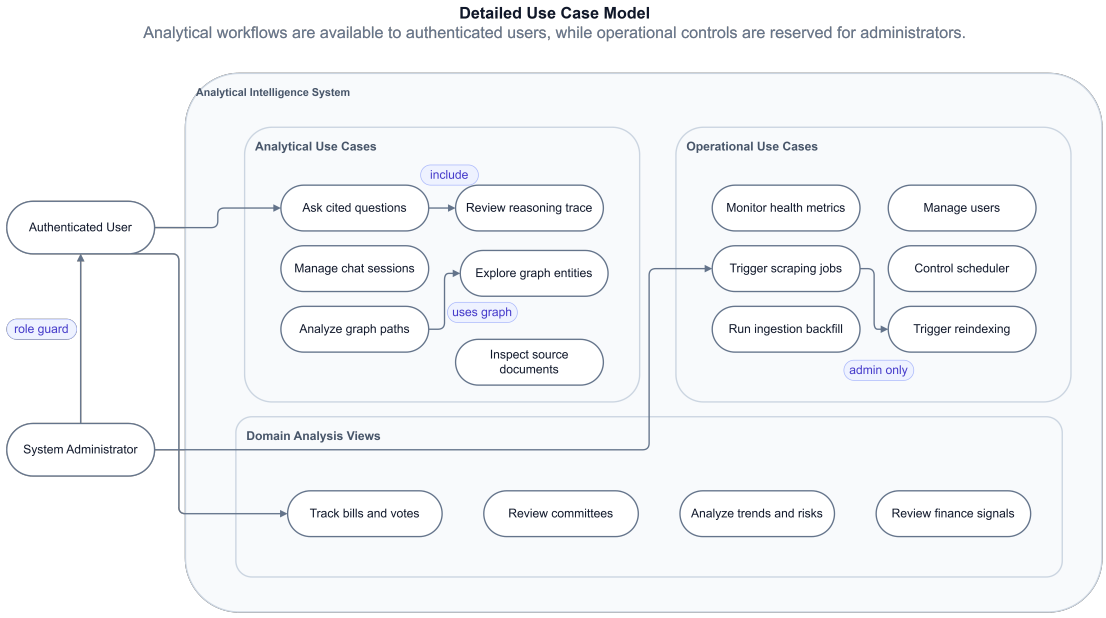


Figure 3.6: Detailed use case model for analytical and administrative workflows.

3.3 Data Flow Architecture

The platform distinguishes **offline** evidence construction from **online** query-time reasoning. Offline flows prepare documents, structured records, graph topology, and vectors; online flows consume those artifacts under latency constraints suitable for interactive analysis.

External sources (RSS feeds, web articles, Google News topics, direct URLs, PDF reports) enter through the acquisition layer and are normalized into MongoDB documents with source metadata and processing flags. Quality gating rejects unsuitable content before extraction. Accepted documents pass through graph extraction (entities and relations to Neo4j; mentions and assertions in MongoDB), structured extraction (domain records such as bills, votes, committees, contributions, conflict flags), and chunking with embedding into ZVec. Checkpoint flags on each document make the offline sequence resumable and idempotent.

At query time, a request arrives at the FastAPI endpoint with parameters for reasoning mode, hop limit, and optional web search. The reasoning engine retrieves ZVec chunks and Neo4j paths (and structured records when the planner requests them), fuses evidence, and invokes synthesis with citation metadata. The response returns answer text, sources, graph facts, traces, and confidence signals for the frontend. Online flow

does not mutate the corpus except through explicit administrative ingestion triggers.

Figure 3.7 consolidates both paths from external sources to cited answers. The diagram emphasizes responsibility splits: MongoDB holds documents and derived textual artifacts; Neo4j holds canonical relational topology; ZVec holds embedded chunks linked to document and entity identifiers. Query execution reads from all three stores but writes only to session and history collections unless an administrator launches ingestion.

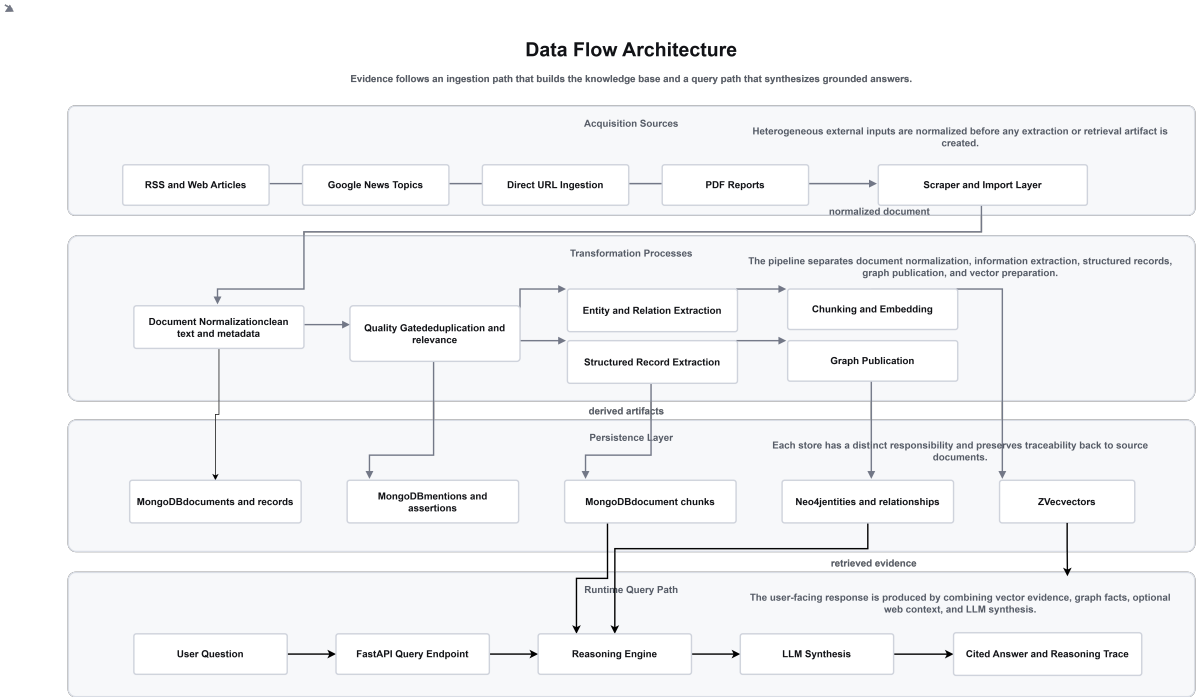


Figure 3.7: Data flow architecture from acquisition to grounded answer generation.

3.4 Ingestion Architecture

Ingestion implements FR-01, FR-08, FR-09, and FR-10 through a multi-level acquisition strategy, explicit normalization and quality gates, and a document-level state machine.

The ingestion engine combines three complementary levels rather than relying on a single crawler template. **Level 1** monitors a curated registry of **57 high-precision outlets** according to predefined schedules (RSS feeds, institutional portals, and established political sources). **Level 2** extends coverage through query-driven discovery of thematic gaps, including search, link traversal, dynamic rendering, and pagination. **Level 3** processes unstructured documents, such as PDF reports, using high-fidelity parsers that preserve layouts and tables. Figure 3.8 details the crawler control flow. A source registry and scheduler populate a prioritized URL frontier; acquisition policies enforce robots directives, rate limits, and retry rules; and a content router selects lightweight extraction

or browser-assisted acquisition according to the response. Both paths converge on a unified document representation, quality gates, MongoDB persistence, and downstream knowledge processing. Failed acquisitions enter a bounded retry and manual-review process rather than silently contaminating the corpus.

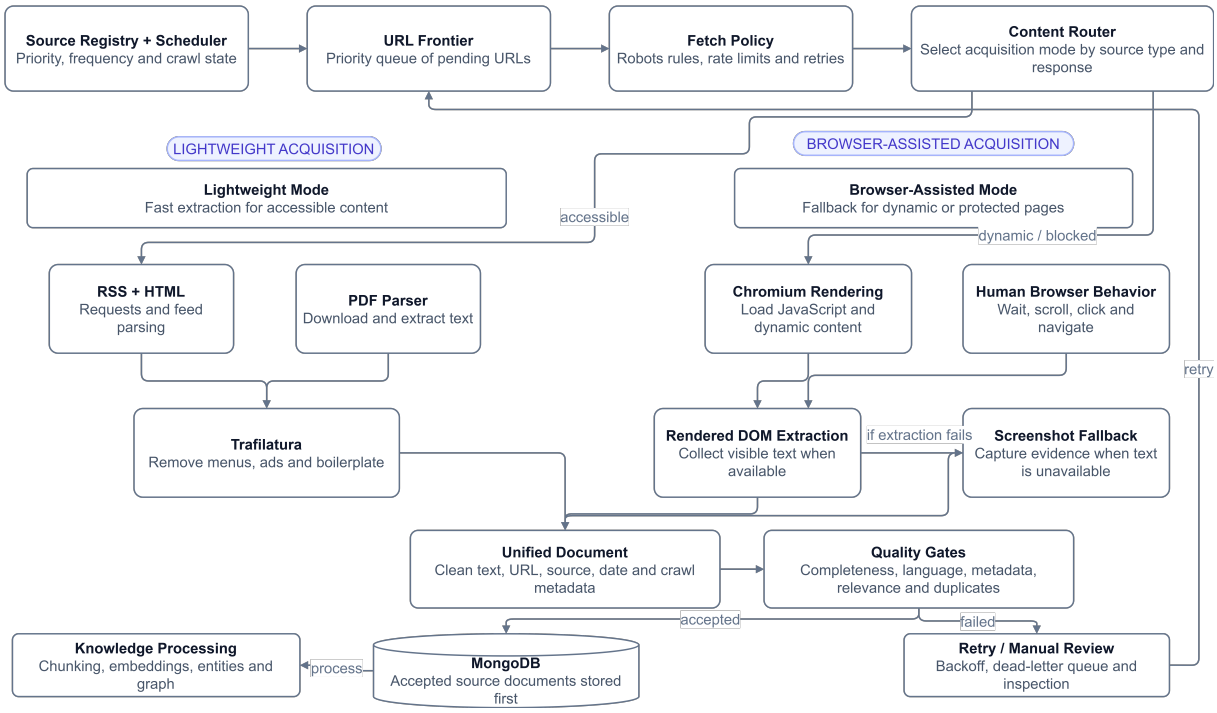


Figure 3.8: Technical crawler architecture with routing, extraction fallbacks, quality gates, and retry control.

All acquired content converges on a unified document schema: identifiers, timestamps, provenance, language hints, and normalized body text. Quality gates filter documents whose length, metadata, or topical signal fall below thresholds for reliable extraction; rejected items remain logged for administrator review without polluting graph or vector indexes. Normalization is the prerequisite for deduplication and for consistent joint extraction (FR-02).

The document lifecycle is modeled as an explicit state machine with checkpoints rather than a monolithic ingest operation. Flags such as `graph_extracted`, `structured_extracted`, and `chunks_created` record completion of each derived stage so that partial reprocessing is possible when prompts, schemas, or embeddings change. Figure 3.9 shows transitions from raw acquisition through quality gating to graph publication, structured records, and vector indexing. Explicit states improve operational control: the pipeline can resume after interruption, skip completed work, or rebuild only derived layers without re-scraping originals.

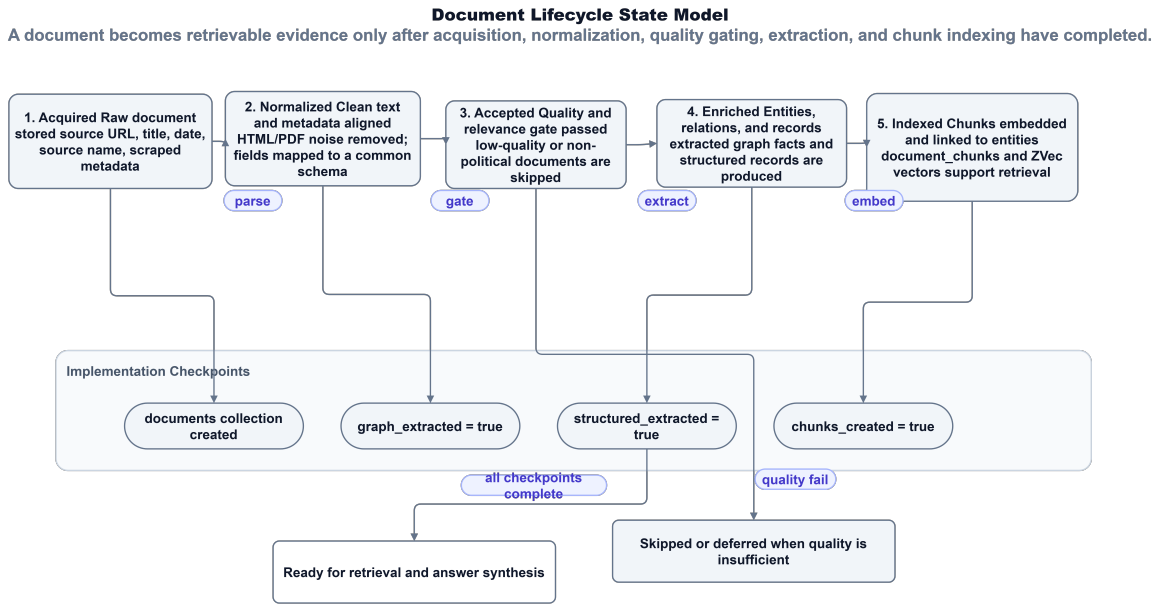


Figure 3.9: Document lifecycle and ingestion checkpoints before retrieval.

3.5 Knowledge Representation and Ontology

The knowledge layer centers on a geopolitical property graph supplemented by document-grounded structured records and vectorized chunks.

The Neo4j ontology defines canonical node types including `POLITICIAN`, `PARTY`, `ORGANIZATION`, `INSTITUTION`, `EVENT`, and related geopolitical categories. Approximately **32 typed relation families** cover actor ties, geopolitical links, events, financial or influence relations, and assertions supported by evidence. Entity resolution maps surface mentions to stable canonical identifiers so syndicated or variant spellings do not fragment the graph. Figure 3.10 links graph topology to MongoDB evidence objects and ZVec chunks: bills, votes, contributions, and conflict flags remain structured data grounded in documents in MongoDB, while actors, institutions, places, and events form traversable graph structure in Neo4j.

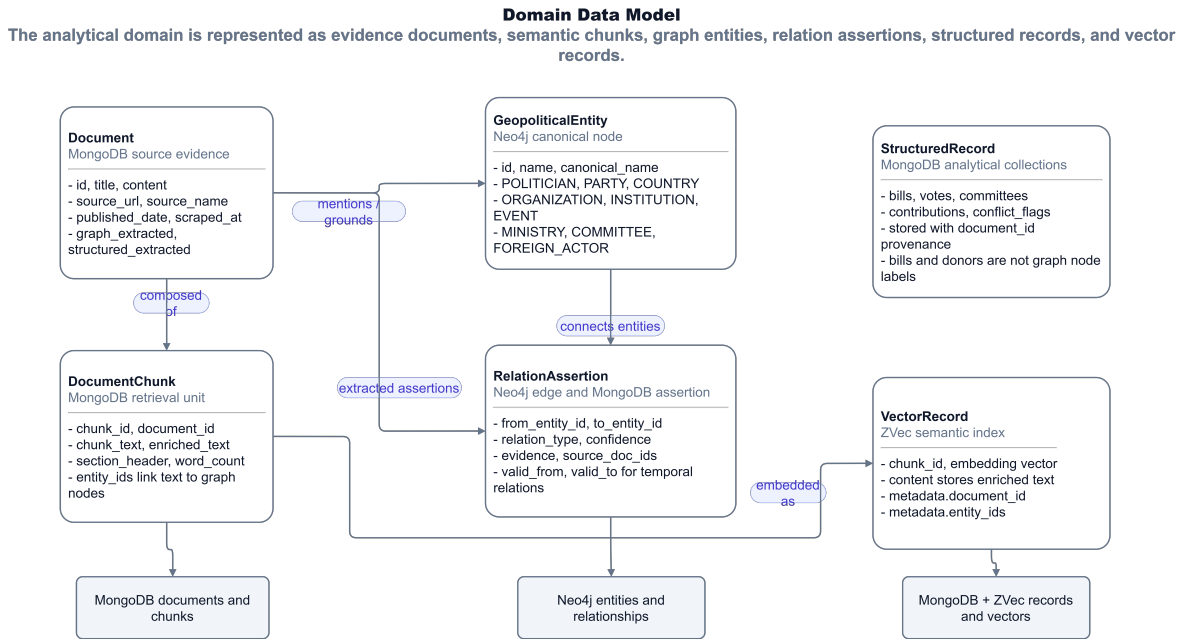


Figure 3.10: Domain data model linking evidence, graph topology, structured records, and vector retrieval.

The storage-oriented domain model is complemented by the geopolitical ontology in Figure 3.11. The ontology identifies the principal analytical concepts and relation families used during extraction and traversal. It separates relatively stable actors and institutions from time-dependent events, mandates, alliances, conflicts, and financial links, while requiring each extracted relation to retain temporal and documentary evidence.

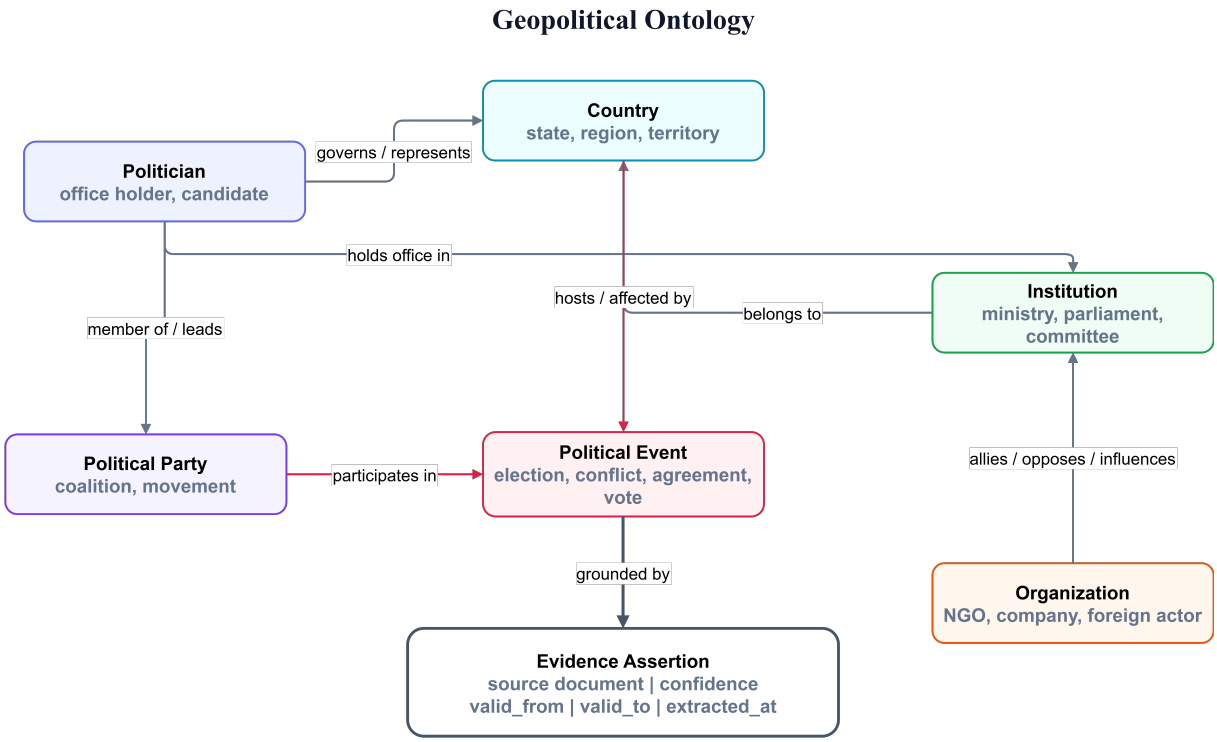


Figure 3.11: Core geopolitical entities, relation families, and provenance attributes.

Political relations are time-bounded: office holding, alliances, and opposition change with elections, conflicts, and institutional reform. Where chronology is available, edges carry `valid_from` and `valid_to` attributes so queries can reconstruct states valid at a requested period rather than mixing obsolete and current facts. Temporal attributes support trend, conflict, and comparative use cases from Chapter 1 and condition multi-hop answers on FR-06.

Provenance is anchored on the `documents` collection: each chunk, mention, assertion, and structured record references its source document and extraction pass. Graph edges in Neo4j remain traceable to assertions stored in MongoDB before publication. This dual linkage satisfies traceability by design and enables the citation layer in the reasoning architecture.

3.6 Data Model and Physical Schema

Persistence is deliberately polyglot: a document store for evidence and operational metadata, a graph store for relational traversal, and a vector store for semantic recall.

MongoDB is the system of record for raw and derived textual evidence. Core collections include `documents` (provenance and processing state), `document_chunks`, entity `mentions`, relation `assertions`, and domain-specific structured collections (legislative, financial, conflict). Application collections (`users`, chat `sessions`, `messages`, query

`history`) are separated from evidence collections so that user activity does not complicate corpus rebuilds. This document-centric role preserves full source text, flexible extraction artifacts, processing checkpoints, and provenance for heterogeneous political sources.

Neo4j complements rather than replaces MongoDB. It holds canonical entities and typed relationships produced after resolution and validation, enabling direct traversal of connected political facts without reconstructing each reasoning path from document records. The schema is optimized for multi-hop queries up to the performance target defined in Chapter 1 (approximately three hops under nominal load). Indexes and relationship types align with the ontology categories above; temporal attributes are applied on edges when evidence supports interval bounds.

ZVec stores dense embeddings of `document_chunks` with metadata linking chunk identifiers, document identifiers, and associated entity identifiers where available. Vector records enable semantic recall in hybrid search (FR-04) and feed the first hop of standard RAG mode. Re-embedding can be executed independently of graph republication when only the embedding model changes.

Figure 3.12 summarizes these complementary responsibilities. MongoDB preserves authoritative evidence and processing state, Neo4j exposes its relational topology, and ZVec provides semantic access to textual passages. The reasoning engine reads from all three representations and retains links to the same source documents.

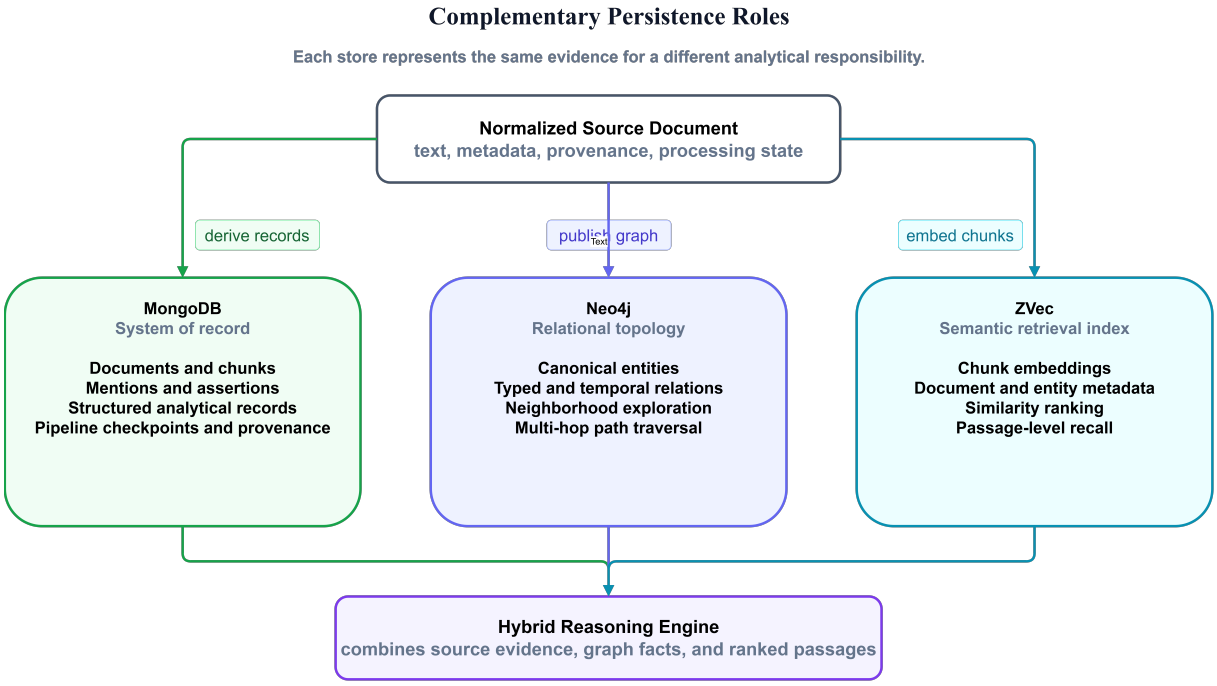


Figure 3.12: Complementary persistence roles of MongoDB, Neo4j, and ZVec.

Figure 3.13 presents the integrated physical view across MongoDB, Neo4j, and

ZVec, including runtime collections for authentication and chat. The ERD clarifies that `documents` sit at the center of provenance links to chunks, structured records, and extraction artifacts, while Neo4j and ZVec remain rebuildable derived stores.

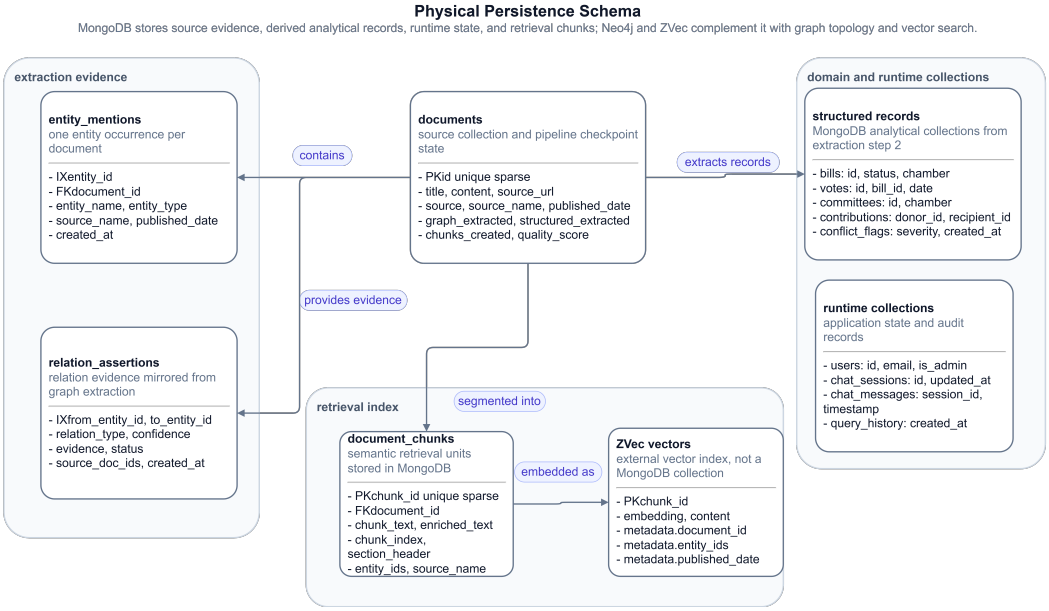


Figure 3.13: Physical persistence schema across MongoDB, Neo4j, and ZVec.

3.7 Reasoning Architecture

The reasoning layer implements FR-04, FR-05, and FR-07 through a plan, retrieve, and synthesize loop with selectable modes and explicit evidence fusion.

At design level, each query iteration performs three stages. **Planning** interprets the question and, in modes involving several reasoning steps, decomposes sub-questions or selects seed entities. **Retrieval** fetches ZVec chunks, Neo4j paths, and optional structured records. **Synthesis** generates a cited answer from fused context. Reflexive mode adds an audit step that tests evidence sufficiency and may trigger additional hops up to **MAX_HOPS = 3**. Optional live web search supplements the local corpus when configured for breaking events not yet ingested. Figure 3.14 shows interactions among the API, reasoning engine, persistence stores, and language model; the loop terminates when acceptance criteria for the selected mode are met or the hop limit is reached.

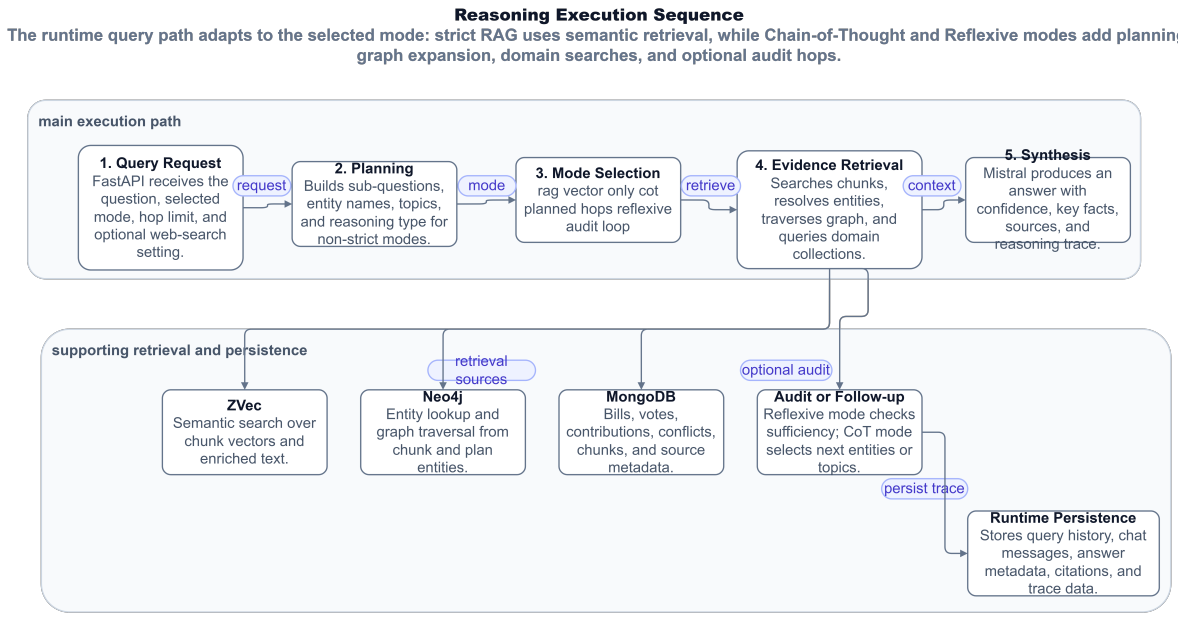


Figure 3.14: Reasoning execution sequence across retrieval, graph expansion, audit, and synthesis.

Three modes are exposed at query time, trading latency against analytical depth. **Standard RAG** (`rag`) performs one-hop semantic retrieval from ZVec using the user question, then direct synthesis; it suits factual lookups with minimal latency. **Chain-of-thought** (`cot`) decomposes complex questions into sub-queries and may follow entities or topics across hops before synthesis. **Reflexive** (`reflexive`) evaluates completeness and consistency after retrieval; insufficient evidence triggers refined retrieval rather than immediate answer generation. Mode selection implements FR-05 at architecture level; Chapter 4 details the `ReasoningEngine` and DSPy orchestration. Interface-level traces expose steps to satisfy FR-07 regardless of mode.

Figure 3.15 compares the control flow of the three modes. Standard RAG follows the shortest retrieval path, chain-of-thought introduces planned sub-questions and graph expansion, and reflexive reasoning adds a decision on evidence sufficiency with a bounded feedback loop.

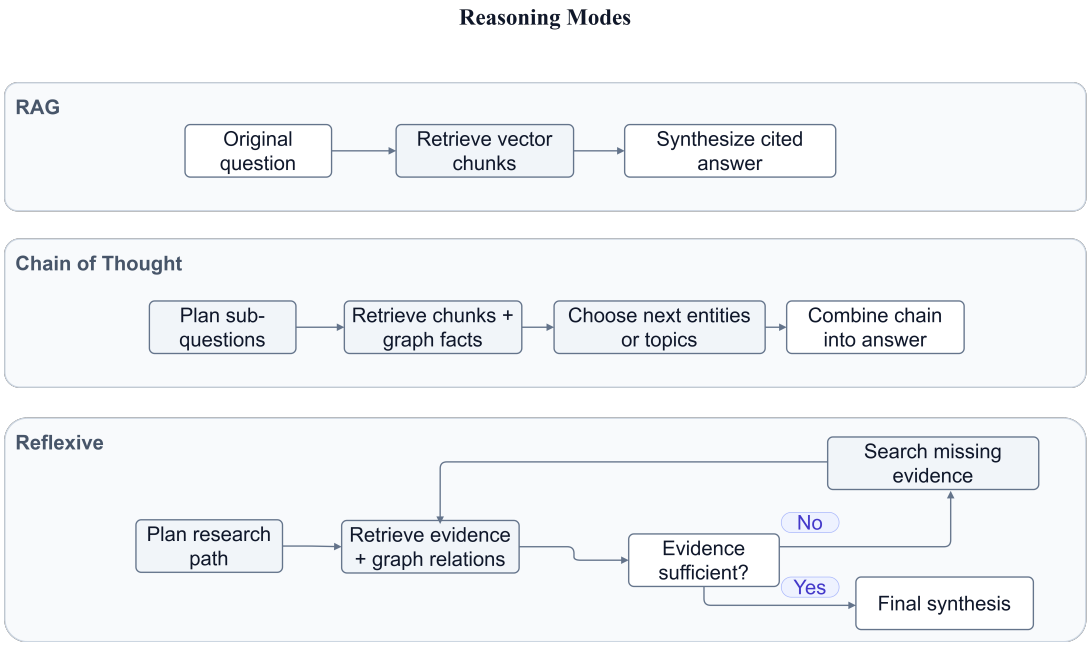


Figure 3.15: Comparison of standard RAG, chain-of-thought, and reflexive reasoning modes.

Evidence fusion merges ranked chunks, graph facts (nodes, edges, paths), and relevant structured records into a single context package for the synthesizer. Ranking considers semantic score, graph distance to seed entities, temporal validity, and deduplication of syndicated passages. Conflicting assertions are passed with provenance so synthesis can qualify claims rather than collapse contradictions silently. Fusion rules are mode-dependent: standard RAG prioritizes chunk relevance; multi-hop modes weight graph expansion; reflexive mode may narrow retrieval targets after audit feedback.

Figure 3.16 summarizes the logical service layering (API routes, domain services, store connectors, model clients) that implements the reasoning and ingestion designs above. Persistence access is isolated from orchestration logic so stores can be mocked or replaced during testing and evolution.

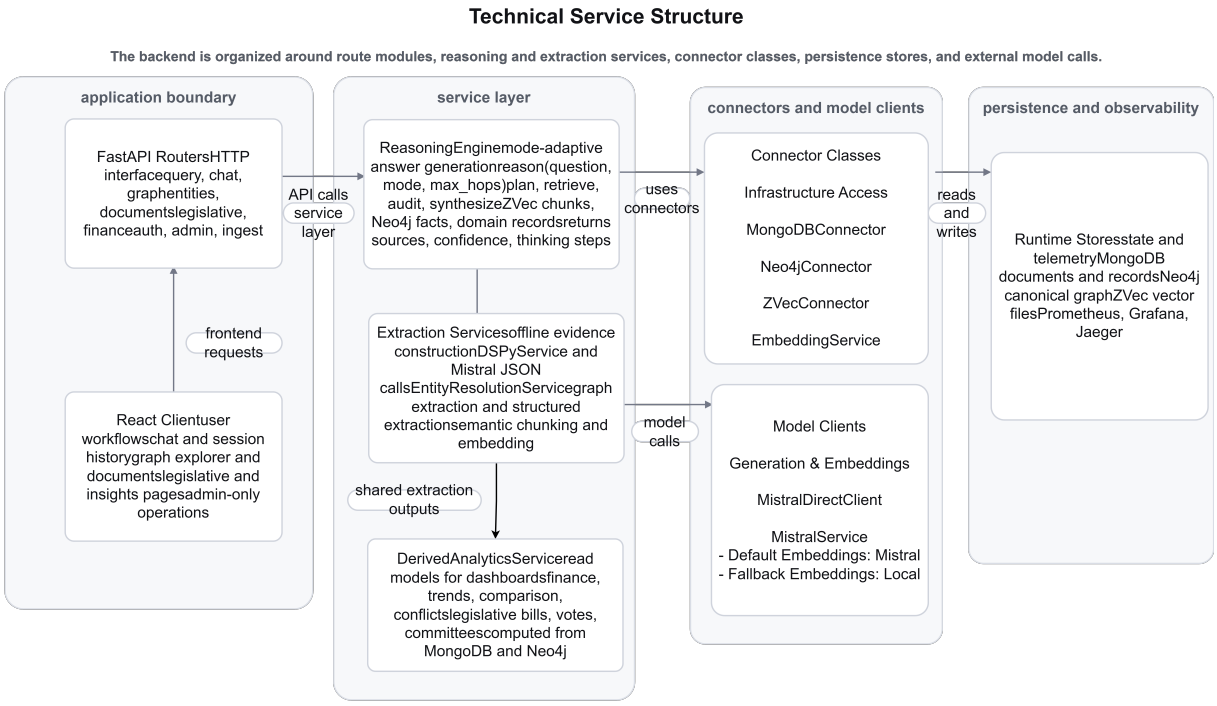


Figure 3.16: Technical service structure across API routes, services, connectors, stores, and model clients.

3.8 Deployment Architecture

The logical design is deployed as containerized services connected through a shared application network. Nginx terminates incoming connections and serves the React frontend, while FastAPI exposes authenticated analytical and administrative routes. Ingestion workers and the reasoning engine remain independently deployable application services, allowing background acquisition workloads to scale without competing directly with interactive queries.

MongoDB, Neo4j, ZVec, and Redis retain distinct persistence responsibilities and isolated data volumes. This arrangement preserves the polyglot model established earlier while keeping service dependencies explicit. Figure 3.17 presents the principal runtime topology; detailed container configuration and operational tooling are discussed in Chapter 4.

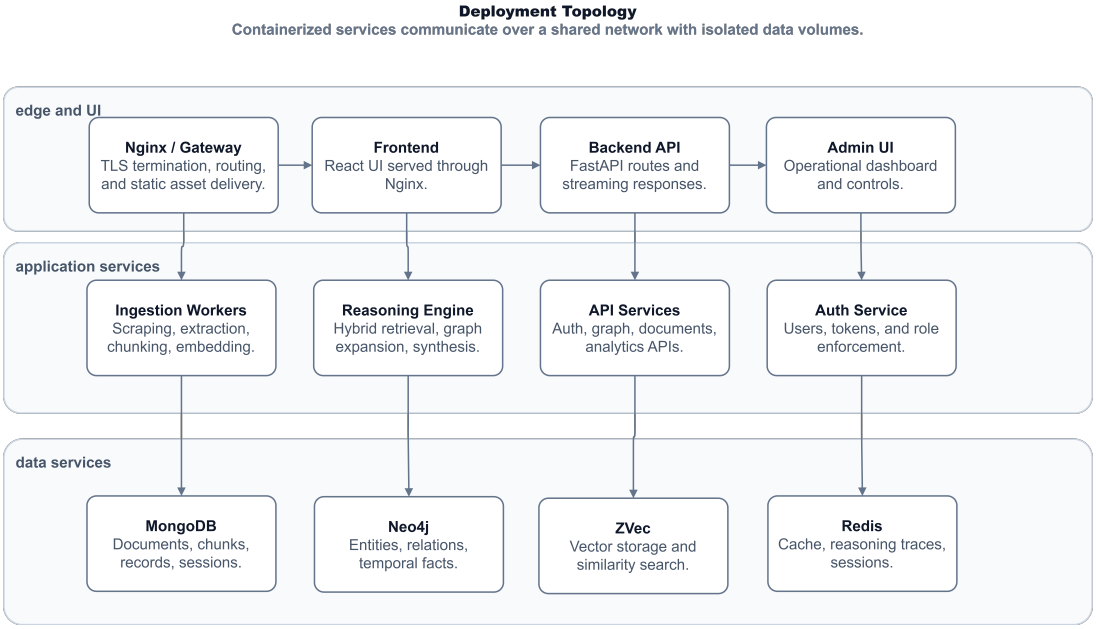


Figure 3.17: Containerized deployment topology across edge, application, and data services.

Conclusion

This chapter translated the previously identified requirements and research gap into a system architecture. The proposed design combines modular ingestion, quality control, hybrid knowledge representation, polyglot persistence, provenance management, and three reasoning modes adapted to different analytical needs. These choices establish a coherent framework for transforming heterogeneous documents into structured evidence and grounded responses. However, an architecture remains only a design hypothesis until it is implemented as reliable, maintainable, and observable software components. The next step is therefore to examine how these architectural decisions can be concretely realized within an operational platform.

Chapter 4

Implementation and Engineering

Introduction

This chapter describes how the design of Chapter 3 was engineered in code, containers, and interfaces intended for users. The presentation covers the NLP and extraction pipeline, polyglot persistence, hybrid retrieval and reasoning services, the React frontend documented through operational screenshots, and the deployment plus observability stack that supports operation under conditions close to production. Implementation choices emphasize explicit service boundaries, deterministic identifiers where possible, and traceability from interface actions back to backend evidence. Evaluation against curated benchmarks is deferred to Chapter 5.

Figure 4.1 provides an implementation-level orientation. Acquired evidence is normalized and checkpointed in MongoDB before DSPy-assisted extraction produces graph and structured artifacts. Chunk embeddings are published to ZVec, canonical relations are published to Neo4j, and FastAPI coordinates these stores for the React analytical workspace. Instrumentation surrounds both background and interactive paths.

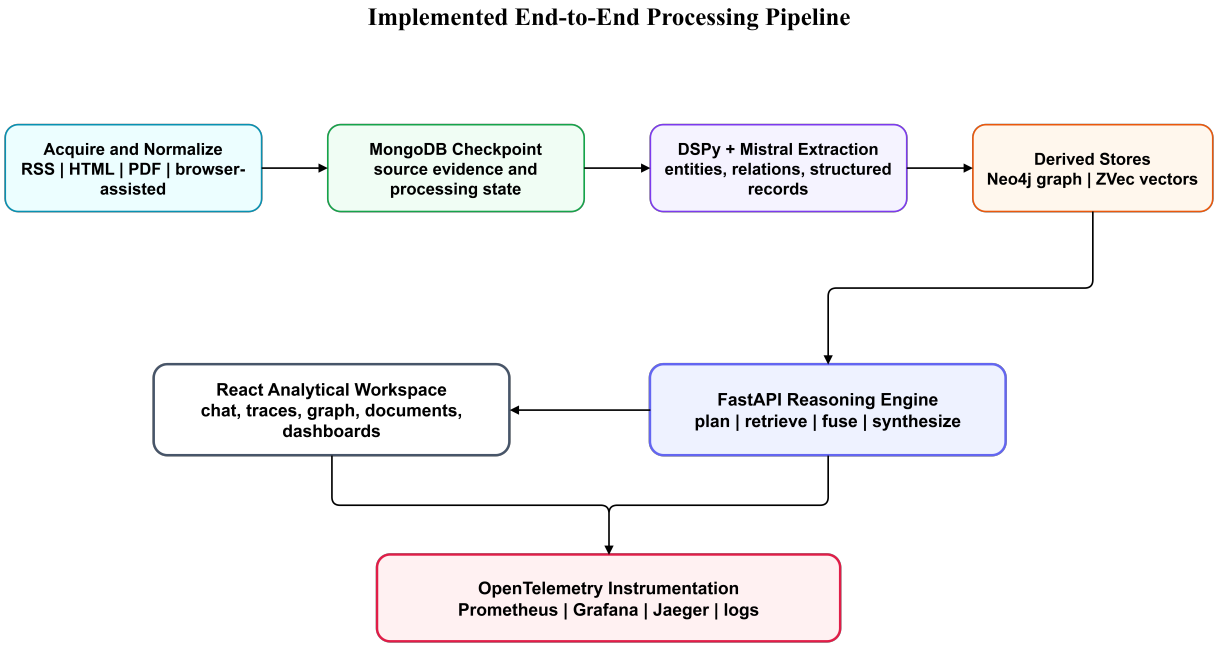


Figure 4.1: Implemented processing path from acquisition to the monitored analytical workspace.

4.1 NLP Pipeline and Extraction

Joint entity and relation extraction is orchestrated through DSPy modules calling Mistral in JSON mode. Prompt templates are treated as optimizable modules rather than static strings: the MIPROv2 Bayesian optimizer explores alternative instructions and few-shot examples on controlled batches, then retains the configuration that maximizes a composite extraction score. Reported gains on the project benchmark include approximately **+2.9%** on the composite score and roughly **1.18 s** lower average inference time per document compared with the initial manual prompt. Offline optimization with DSPy is therefore separated from online serving with the production Mistral client, so prompt iteration does not destabilize the live API.

Figure 4.2 details this separation. Curated examples and expected outputs define the optimization set; MIPROv2 proposes instructions and demonstrations; Mistral executes each candidate under the same JSON contract; and a composite evaluator scores schema validity, extraction quality, and latency. Only the strongest versioned program is promoted to production extraction.

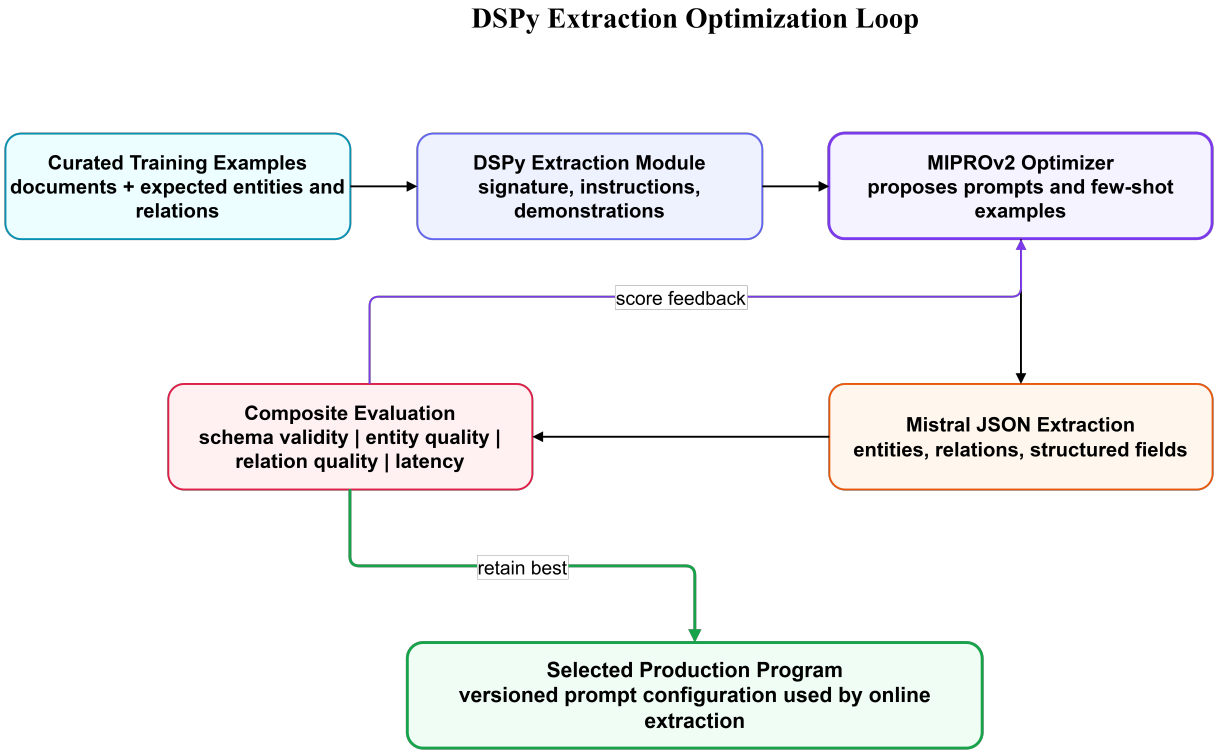


Figure 4.2: Offline DSPy and MIPROv2 optimization loop for production extraction programs.

The extraction contract requires a single pass over each normalized MongoDB document, emitting entities, typed relations, and structured fields aligned with the geopolitical ontology (Chapter 3). Deterministic identifier generation is enforced in prompt logic so that repeated runs map the same canonical actor to the same graph key, preserving topological consistency between MongoDB assertions and Neo4j nodes. Validation rejects malformed JSON, empty extractions, and role titles mistaken for personal names before graph publication. Failed documents remain in MongoDB with explicit error states for administrator inspection rather than silently polluting Neo4j or ZVec indexes.

4.2 Persistence Layer Implementation

MongoDB. Fourteen indexed collections hold documents, chunks, mentions, assertions, structured legislative and financial records, conflict flags, users, chat sessions, messages, and query history. The `documents` collection carries processing checkpoints (`graph_extracted`, `structured_extracted`, `chunks_created`) that drive idempotent pipeline scripts. Dashboard views and the document library read directly from these collections while keeping application state separate from evidence records.

Neo4j. The property graph enforces ten canonical entity types and thirty-two relationship types. The `EntityResolutionService` normalizes surface forms, filters verb-like

or generic role phrases, maps aliases to canonical types, and assigns stable identifiers before `MERGE` operations. Traversal queries use parameterized Cypher with depth limits aligned to the design target (up to three hops under nominal load). Graph writes occur only after MongoDB assertions pass schema and quality checks.

ZVec. Embeddings of `document_chunks` are stored locally with chunk, document, and entity metadata for hybrid ranking. At query time the backend embeds the user question, retrieves candidates by cosine similarity, and applies lexical reranking before passing chunks to synthesis. Local persistence supports data-sovereignty constraints and removes dependency on an external vector SaaS. The comparative rationale and published ZVec benchmark are discussed in Chapter 2; this chapter focuses on its concrete integration into the platform.

Embedding-model selection. The first retrieval baseline used `all-MiniLM-L6-v2`, whose 384-dimensional vectors offered low storage cost and fast local experimentation while the complete indexing and query path was still being validated. The model choice was treated as an engineering trade-off rather than a permanent architectural constraint. Table 4.1 summarizes representative alternatives considered for stronger semantic capacity or multilingual coverage.

Model	Dim.	Language scope	Engineering interest
`all-MiniLM-L6-v2`	384	Mainly English	Lightweight baseline with fast local indexing and limited storage cost.
`all-mpnet-base-v2`	768	Mainly English	Higher semantic capacity with greater compute and storage requirements.
`paraphrase-multilingual-MiniLM-L12-v2`	384	Multilingual	Compact option for English, French, and Arabic retrieval experiments.
Production-compatible embedding service	Not disclosed	Multilingual	Alignment with SCIENTIAE's deployed, sovereign AI infrastructure.

Table 4.1: Embedding alternatives and retrieval trade-offs considered during implementation.

Vector dimension alone does not determine retrieval quality. Chunk coherence, overlap, metadata, query formulation, reranking, and corpus language can outweigh a nominal increase in dimensionality. As the ingestion pipeline expanded toward multilingual sources, the production path moved toward the embedding service already available in SCIENTIAE's Mistral-compatible infrastructure. This reduced integration overhead and kept retrieval within the organization's sovereign deployment boundary without claiming an undocumented model name or vector dimension.

Chunking and context expansion. Documents are indexed as paragraph aware chunks rather than as complete files. Neighboring chunks retain document identifiers, sequence positions, section headers, source metadata, and entity links. This structure limits the risk that a fixed boundary separates an actor from an action or a claim from its date. When a highly ranked chunk lacks sufficient local context, the retrieval layer can recover adjacent segments from the same document before synthesis. The first search therefore remains precise over compact vectors, while selective expansion improves recall around promising evidence instead of filling the model context with an entire source document. Recursive splitting patterns documented by LangChain and parsing concepts based on nodes documented by LlamaIndex provide practical reference points for this implementation strategy [23, 24].

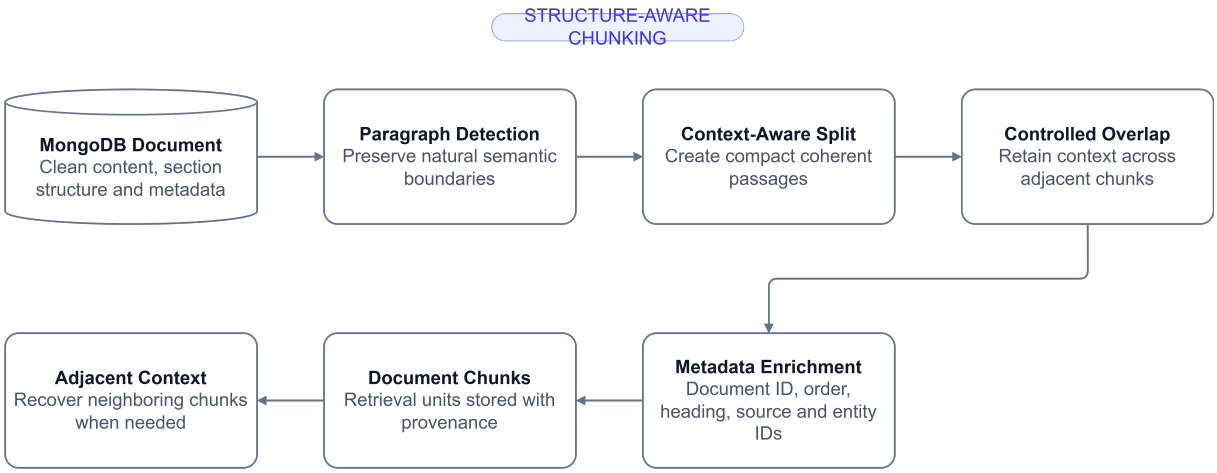


Figure 4.3: Implemented structure-aware chunking, metadata enrichment, and adjacent-context recovery.

Approximate vector retrieval. Interactive RAG cannot compare every query embedding exhaustively with every stored chunk as the corpus grows. ZVec provides indexed similarity search so that the first retrieval hop returns a small candidate set for lexical reranking, metadata filtering, adjacent-chunk expansion, and graph enrichment. This follows the same engineering motivation as large-scale approximate nearest-neighbour systems such as Proxima: practical vector search must balance recall, latency, index-construction cost, update behavior, and metadata constraints rather than optimize only exact distance computation [15]. In the hybrid architecture, ZVec supplies semantic recall, Neo4j supplies relational precision, MongoDB preserves provenance, and the reasoning engine fuses these signals into the evidence package presented to Mistral.

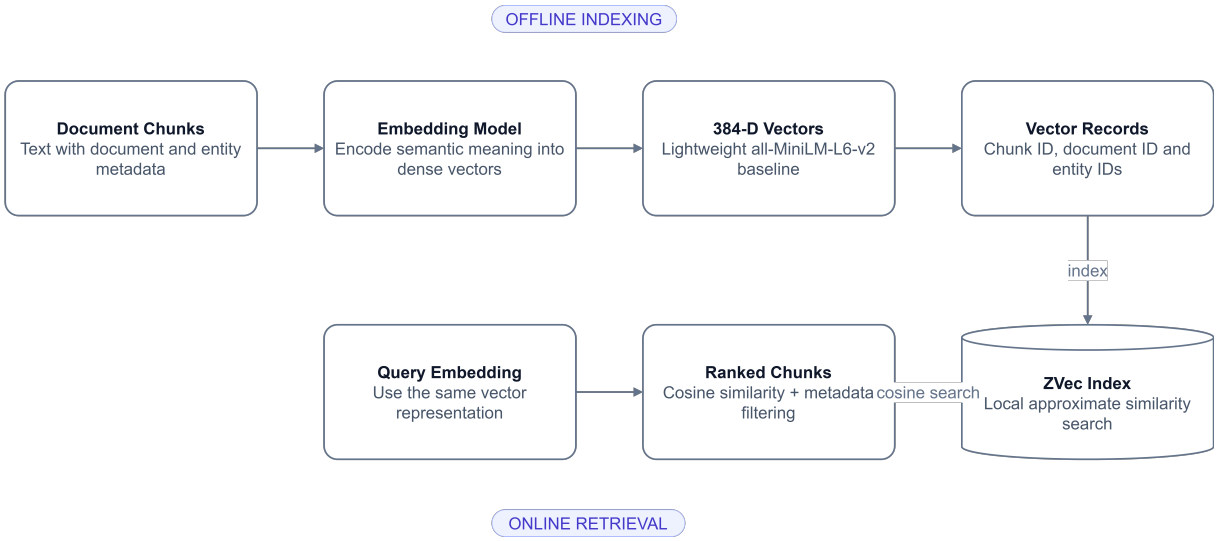


Figure 4.4: Offline embedding and indexing followed by online similarity retrieval in ZVec.

Redis. A cache layer stores reasoning traces, session context, and frequently accessed metadata so repeated analytical sessions remain responsive. Cache entries are keyed by session and query identifiers; invalidation follows session lifecycle events rather than corpus-wide flushes, so ingestion backfills do not unnecessarily evict active conversations.

4.3 Retrieval and Reasoning Implementation

The `ReasoningEngine` service implements the plan, retrieve, and synthesize loop described in Chapter 3. Regardless of mode, the skeleton comprises planning (when enabled), ZVec chunk retrieval, optional Neo4j entity lookup and traversal (depth 2 by default, up to `MAX_HOPS = 3` iterations in multi-hop modes), synthesis through Mistral with structured citation fields, and serialization of a reasoning trace for the frontend. Optional web search augments the local corpus when the query signals recency or when the user explicitly enables it.

Figure 4.5 maps the three production modes to retrieval depth. **RAG mode** skips decomposition and graph expansion: the original question queries ZVec once, chunks are deduplicated, and synthesis proceeds immediately, which minimizes latency for factual lookups. **Chain-of-thought mode** runs the planner, executes sub-questions sequentially, accumulates chunks and graph facts across hops, then synthesizes. **Reflexive mode** adds an audit step after each hop; insufficient evidence triggers refined retrieval before synthesis. Redis caches traces so the UI can replay steps without recomputation.

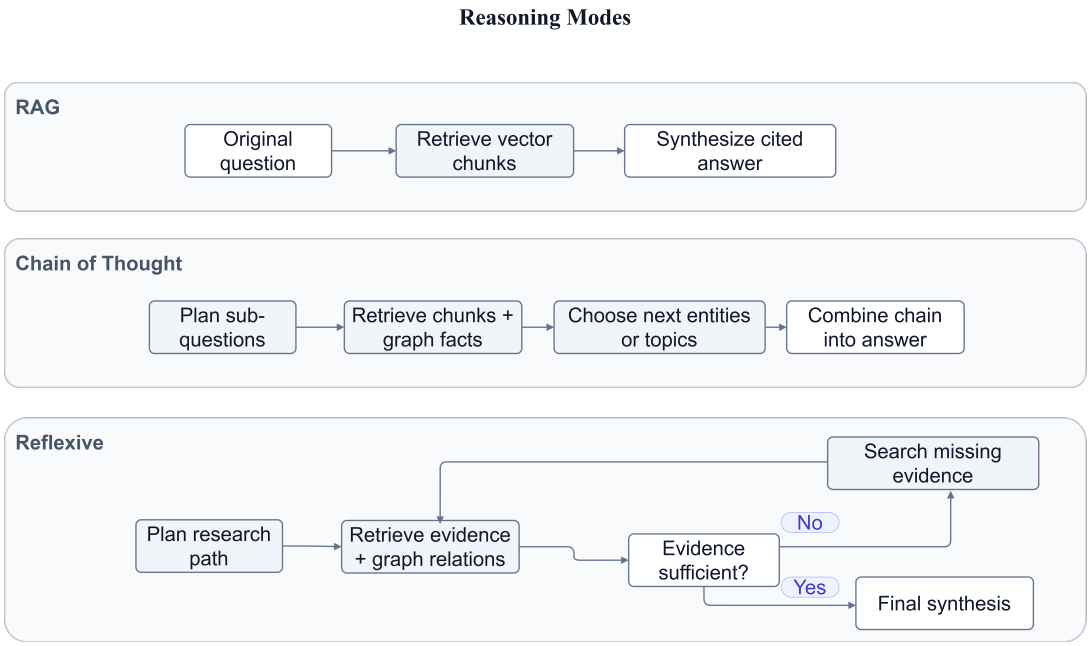


Figure 4.5: Three reasoning modes in the query engine: one-hop RAG, chain-of-thought multi-hop, and reflexive control.

Hybrid ranking combines semantic scores from ZVec, graph distance to seed entities, temporal validity on edges, and deduplication of syndicated passages. Synthesis prompts require explicit source references: chunk identifiers, document titles, and graph fact summaries are passed as structured context. The FastAPI `/query` route streams thinking steps to the client using server-sent events so the chat interface can render planning, retrieval, audit, and synthesis phases in real time. Chapter 5 evaluates answer quality and grounding across modes on the same deployed endpoint.

Optional live web search (DuckDuckGo). All three reasoning modes can augment the local corpus with external web evidence through `DuckDuckGoSearchRun` (`services/websearch.py`). **DuckDuckGo** is a privacy-oriented search engine: it returns ranked results without user profiling and without a commercial search API key, which aligns with sovereignty constraints and with patterns used by many modern AI agents that combine indexed knowledge with fresh public web pages. The service posts queries to DuckDuckGo's HTML results endpoint (Instant Answer API as fallback), normalizes hits as `type: web_search` objects, and passes them to synthesis separately from ZVec chunks so provenance remains explicit. The chat **Web On / Off** toggle sets `enable_web_search`; recency keywords can auto-enable search; administration exposes *AI Search* (*DuckDuckGo*) for topic-driven ingestion. Figure 4.6 summarizes the implementation flow.

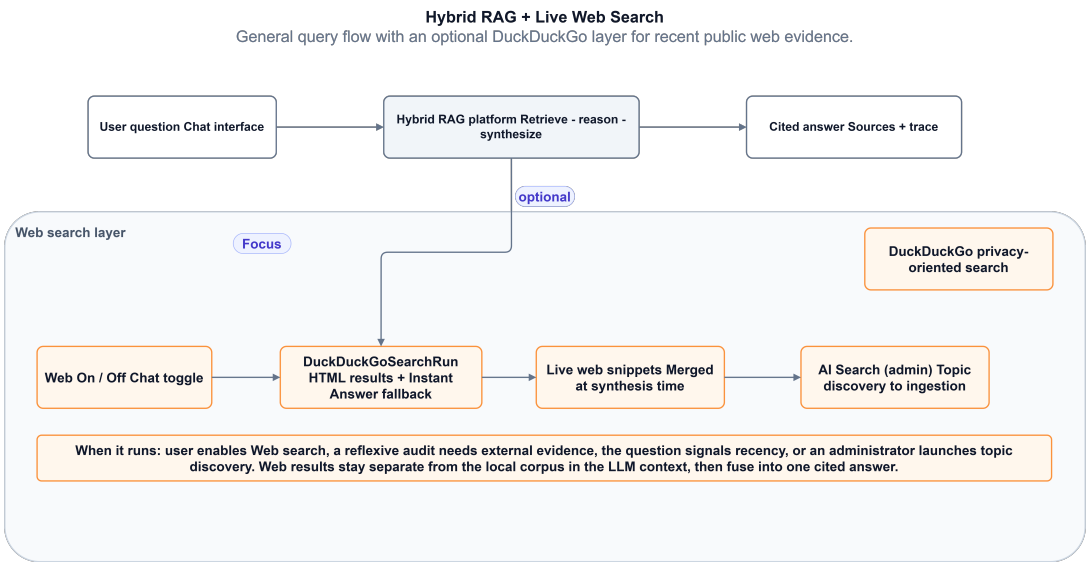


Figure 4.6: Hybrid RAG implementation with optional DuckDuckGo web search at query time.

The diagram shows that internal retrieval (ZVec, Neo4j, MongoDB) remains the primary path, while DuckDuckGo supplies supplementary snippets when the user, the reflexive auditor, or an admin discovery job requires information not yet indexed locally. Web evidence does not replace the corpus; it bridges the gap until scheduled ingestion ingests the same sources permanently.

4.4 Frontend Implementation

The client is a React and Vite single-page application with React Router, TanStack Query for server state, and a resizable layout pairing a persistent chat-history sidebar with the active workspace. Routes include the home chat (`/`), session deep links (`/chat/:sessionId`), authentication (`/auth`), administration (`/admin`), graph explorer (`/graph`), legislative tracker (`/legislative/...`), political insights (`/insights/...`), and document library (`/documents`). A centralized API client wraps FastAPI endpoints for queries (including SSE streams), graph search, documents, auth, and admin operations, preserving a strict boundary between presentation and backend reasoning services.

Analytical workspace: chat and reasoning trace. The chat interface is the primary entry point for cited questioning. Figure 4.7 shows the layout: session sidebar, message thread, reasoning-mode selector (`rag`, `cot`, `reflexive`), and a **Web On / Web Off** control linked to DuckDuckGo retrieval in the reasoning implementation above. The mode and web flags are forwarded to the backend as `reasoning_mode` and `enable_web_search`.

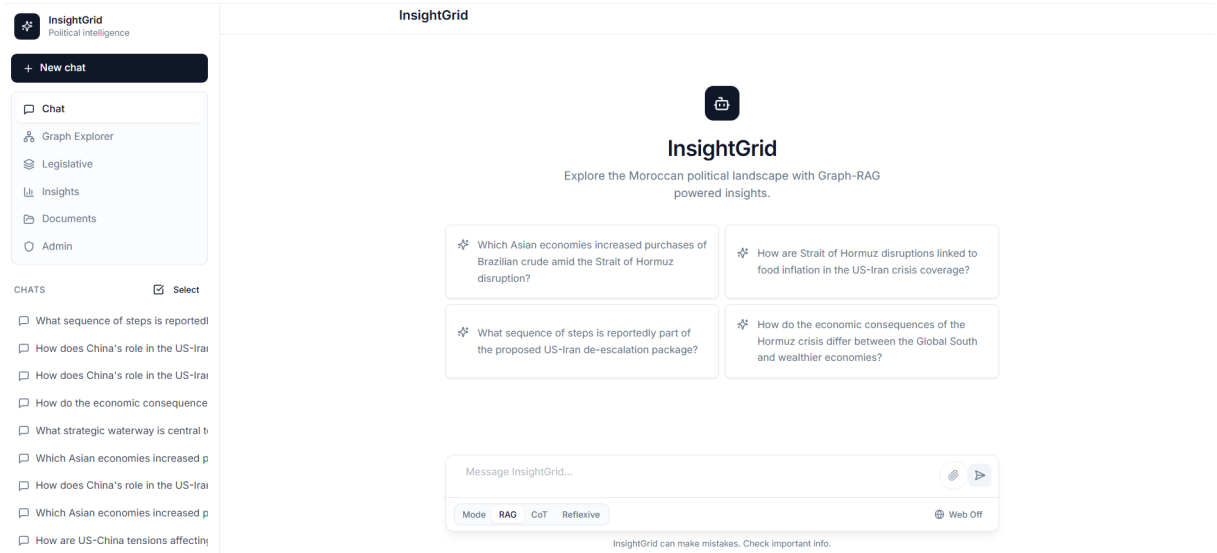


Figure 4.7: Chat workspace with session history sidebar and reasoning-mode controls.

Figure 4.8 illustrates a completed answer in RAG mode: the assistant message includes inline citations, source cards, and confidence cues tied to retrieved chunks. Figure 4.9 expands the reasoning trace panel, exposing sequential steps (ZVec retrieval, synthesis) so the user can verify which evidence supported each claim, implementing FR-07 at interface level.

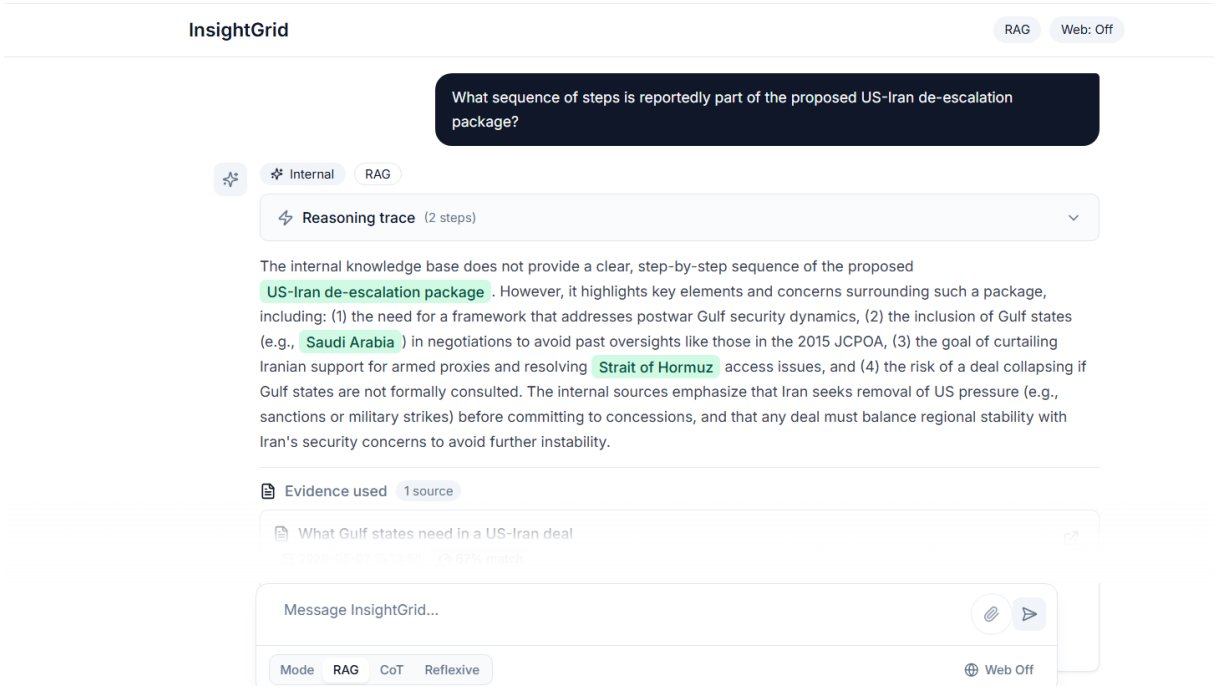


Figure 4.8: Cited answer produced in standard RAG mode with linked source evidence.

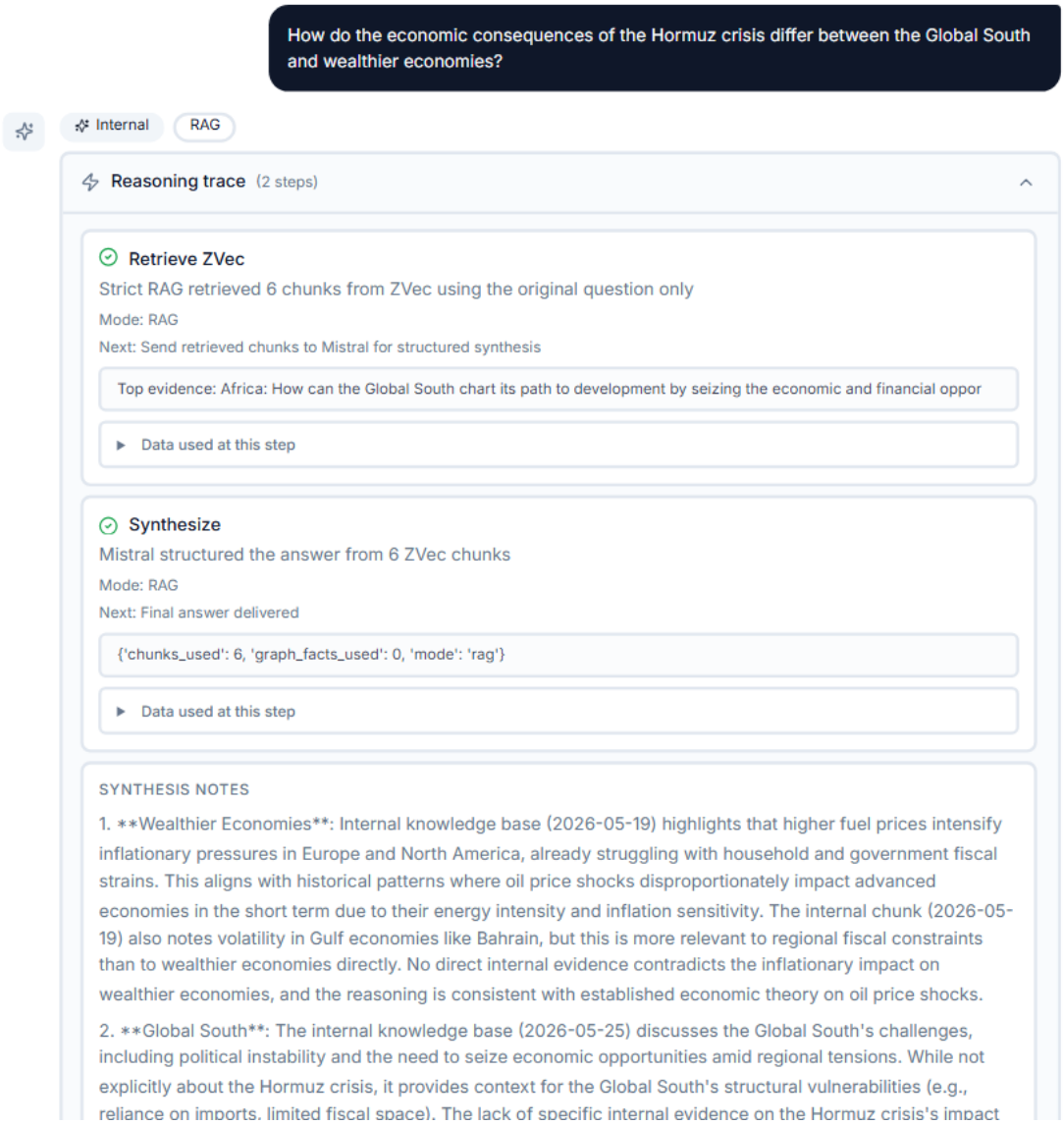


Figure 4.9: Expandable reasoning trace for a RAG-mode query (retrieval and synthesis steps).

Document library and graph explorer. The document library (Figure 4.10) lists ingested sources with filters on metadata, supporting the use case in which analysts verify wording and provenance before trusting synthesized summaries. Selecting a document opens normalized text and extraction status without leaving the analytical shell.

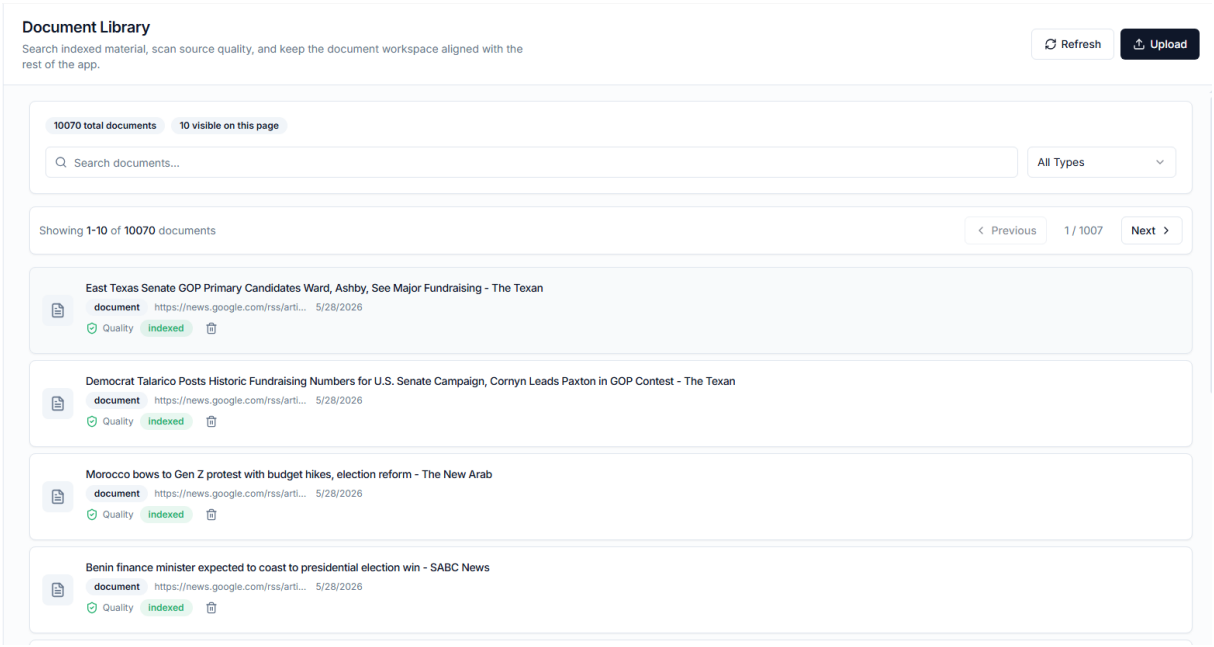


Figure 4.10: Document library for inspecting ingested corpus items and metadata.

The graph explorer (Figure 4.11) renders force-directed neighborhoods with `react-force-graph-2d` and D3.js interactions: pan, zoom, node selection, and typed edge labels. Users pivot from a chat answer to structural exploration by opening the same entities in the graph view, closing the loop between conversational synthesis and relational inspection.

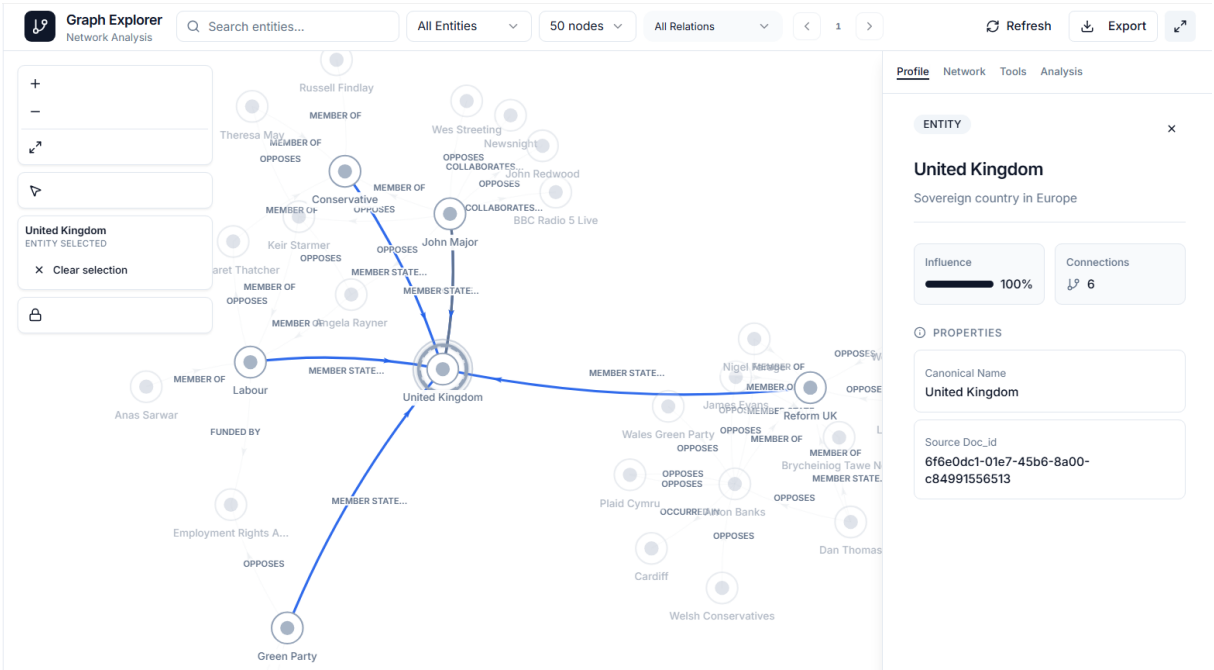


Figure 4.11: Graph explorer with interactive entity neighborhood layout.

Legislative and structured political views. The legislative tracker exposes MongoDB

structured records extracted from policy corpora. Figure 4.12 presents bill listings with status and sponsorship metadata; Figure 4.13 focuses on roll-call votes; Figure 4.14 shows committee assignments and related actors. These views complement graph traversal by offering tabular, schema-stable summaries for institutional monitoring.

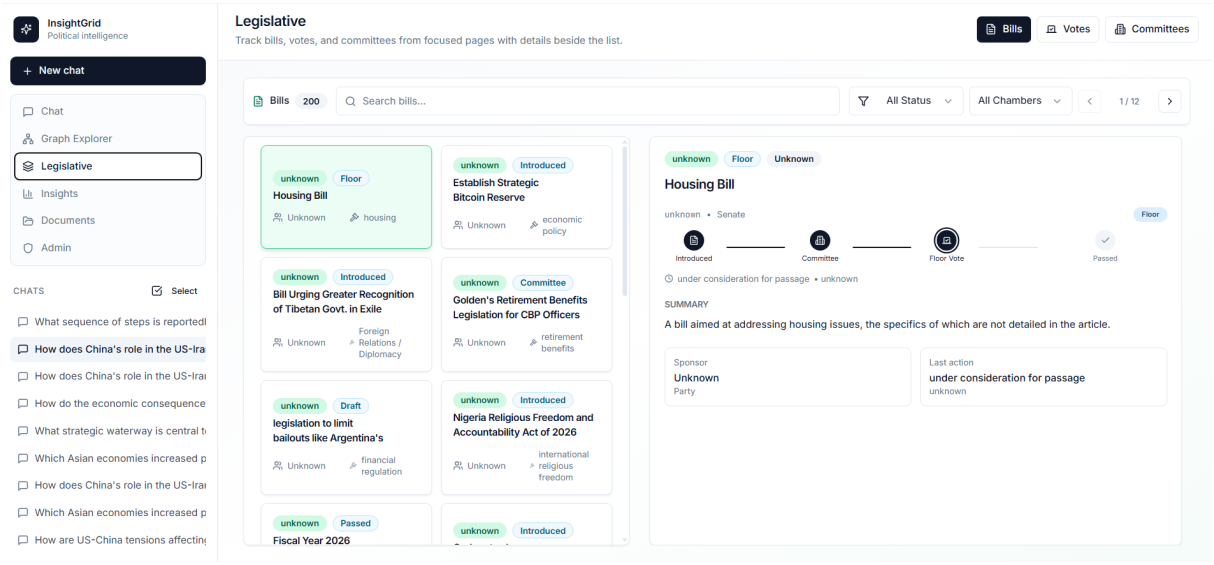


Figure 4.12: Legislative tracker: bills view with structured metadata from extracted records.

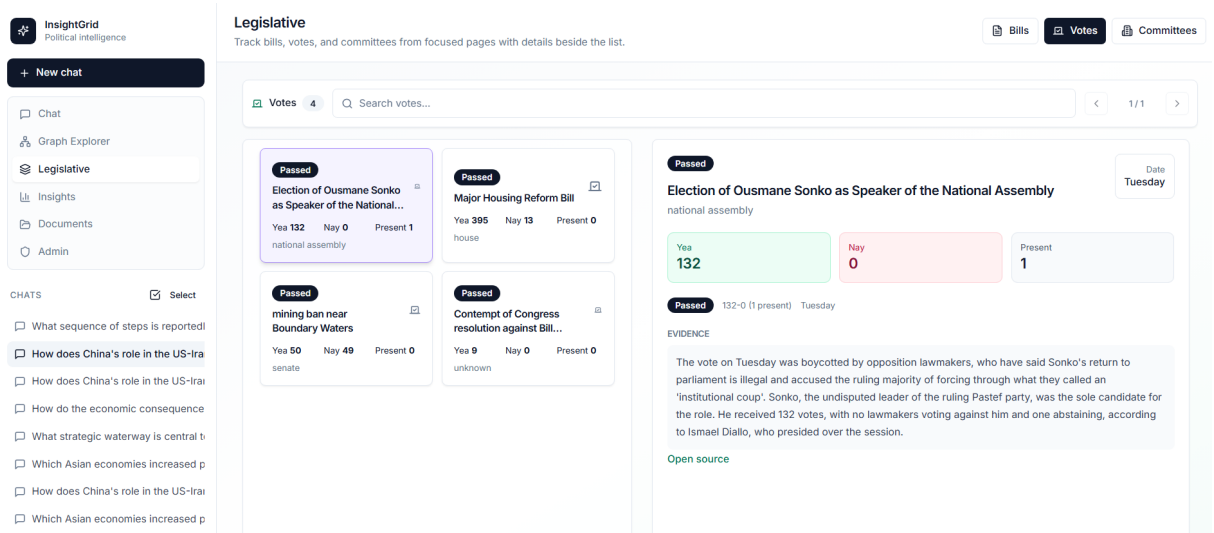


Figure 4.13: Legislative tracker: votes view linking legislators to roll-call outcomes.

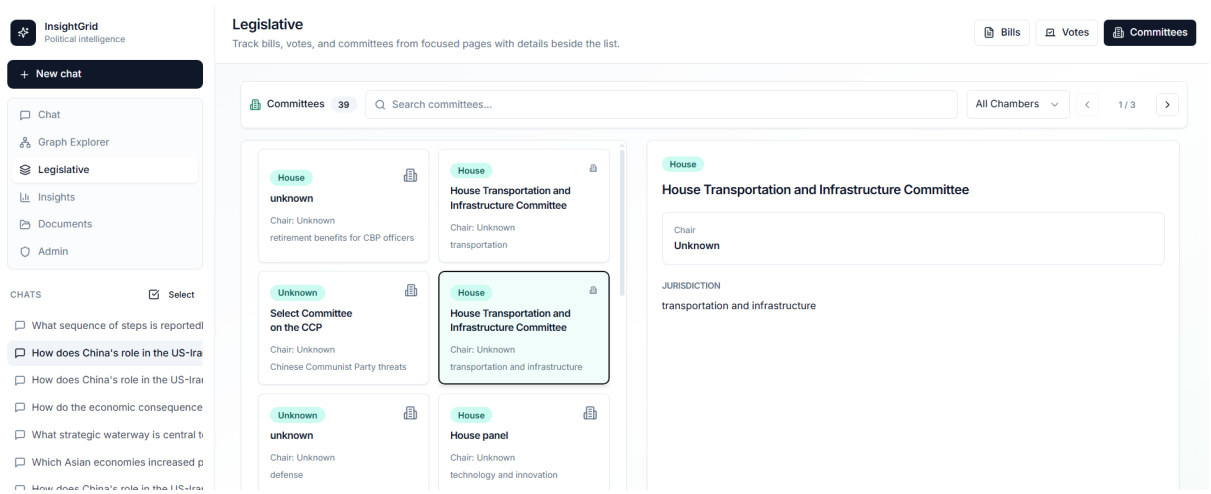


Figure 4.14: Legislative tracker: committees view for institutional roles and memberships.

Political insights dashboards. The `/insights/...` module groups four analytical tabs on top of MongoDB structured records, Neo4j neighborhoods, and derived analytics services.

The **finance** tab (Figure 4.15) queries campaign-contribution and donor records filtered by electoral cycle, country, entity type, and free-text search. It surfaces top donors, aggregated amounts, and finance-related conflict flags when the same actor appears on both sides of a funding relationship, supporting scrutiny of economic influence without leaving the platform.

The **trends** tab (Figure 4.16) plots mention frequency and sentiment volatility for entities or topics over selectable windows (e.g. seven days) and optional regional filters. Recharts time-series views help analysts distinguish sustained narrative shifts from short-lived spikes in media coverage.

The **conflicts** tab (Figure 4.20) lists `conflict_flags` extracted from documents: opposing stances, shared donors with competing beneficiaries, committee overlaps, and similar patterns where two actors occupy incompatible roles in the evidence. Each row links back to document provenance for manual verification.

The **compare** tab implements pairwise entity intelligence across eight analytical aspects (or all aspects at once). The user selects two canonical entities via autocomplete (`/graph/search` suggestions), then chooses an *aspect scope*:

- **Political stance:** party alignment, ideology, support/opposition, coalitions, elections;
- **Economic policy:** budgets, taxation, trade, inflation, subsidies, public spending;
- **Influence and network:** allies, donors, committees, appointments, lobbying ties;
- **Governance style:** reform, transparency, accountability, institutional oversight;

- **Legislative record:** bills, amendments, roll-call votes, committee hearings;
- **Foreign policy:** diplomacy, alliances, sanctions, security treaties;
- **Public sentiment:** polls, approval, protests, media backlash, popularity;
- **Risk and conflict exposure:** scandals, investigations, lawsuits, controversies;
- **All aspects:** runs the full matrix below in one report.

Figure 4.17 shows the selection panel (two entities + aspect dropdown). The backend (`POST /political/compare`) resolves both subjects in Neo4j, fetches ZVec/MongoDB evidence filtered by aspect keywords, computes shortest-path graph distance, and builds an **aspect matrix** (Figure 4.18): for each row, evidence scores for entity A, entity B, term overlap, shared neighbors, and graph proximity. Cells summarize how strongly each actor is documented on that dimension.

Figure 4.19 presents the narrative layer: *convergences* (shared terms, common neighbors, short graph paths), *divergences* (unique evidence terms and asymmetric neighborhoods), a conclusion generated by the LLM, and optional *graph path* visualization between the two nodes. The report can be exported as JSON for offline review. This three stage flow (configure, matrix, interpret) mirrors analyst practice: quantify overlap on defined dimensions before drawing qualitative judgments.

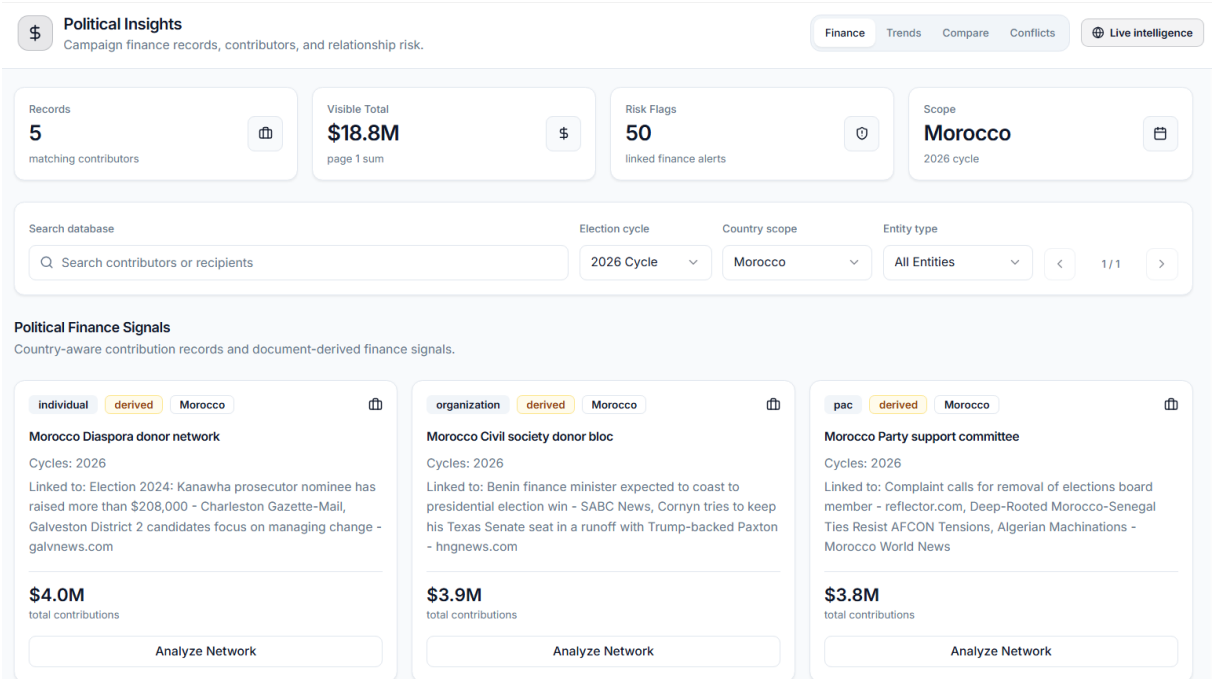


Figure 4.15: Political insights: campaign finance dashboard.

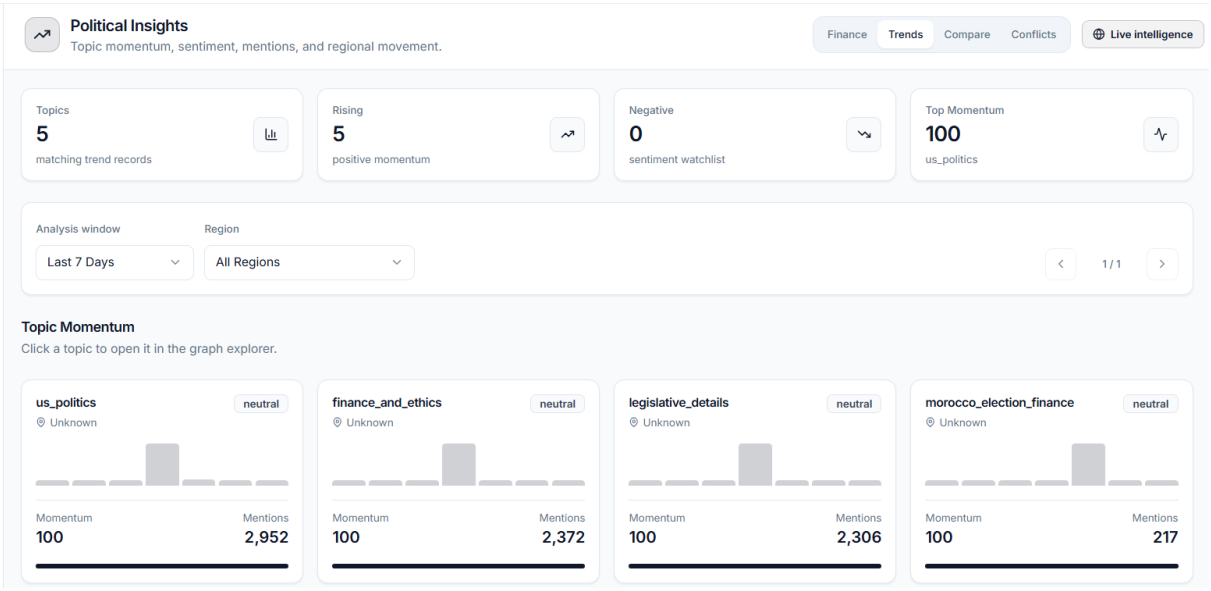


Figure 4.16: Political insights: temporal trend tracking for entities or topics.

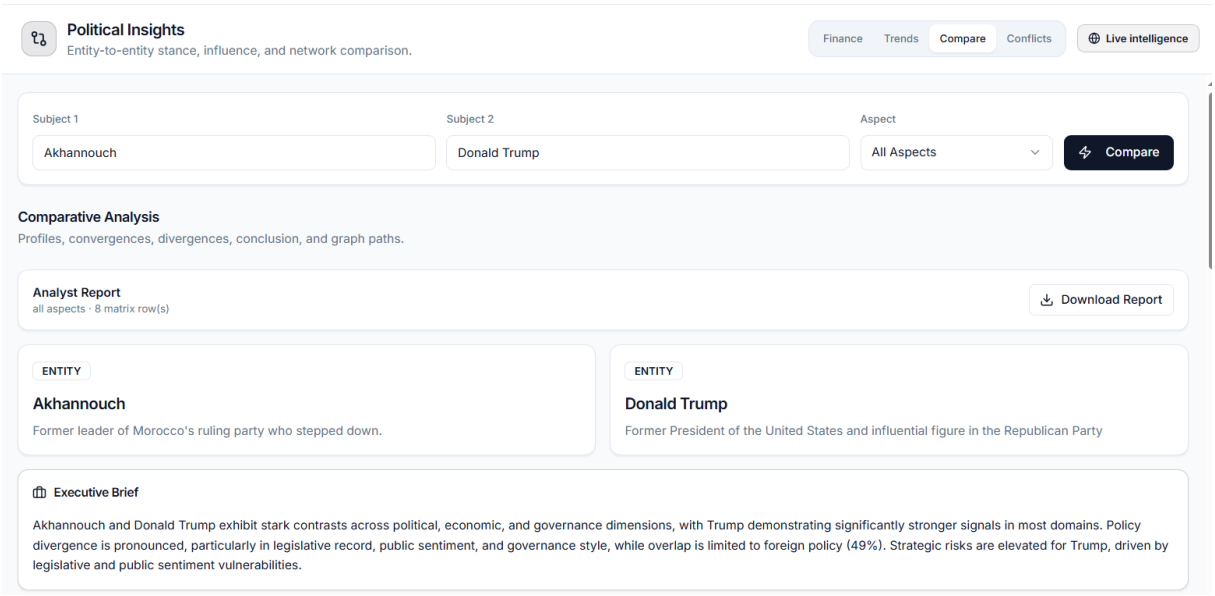


Figure 4.17: Entity comparison workflow (step 1): selection of actors or topics to compare.

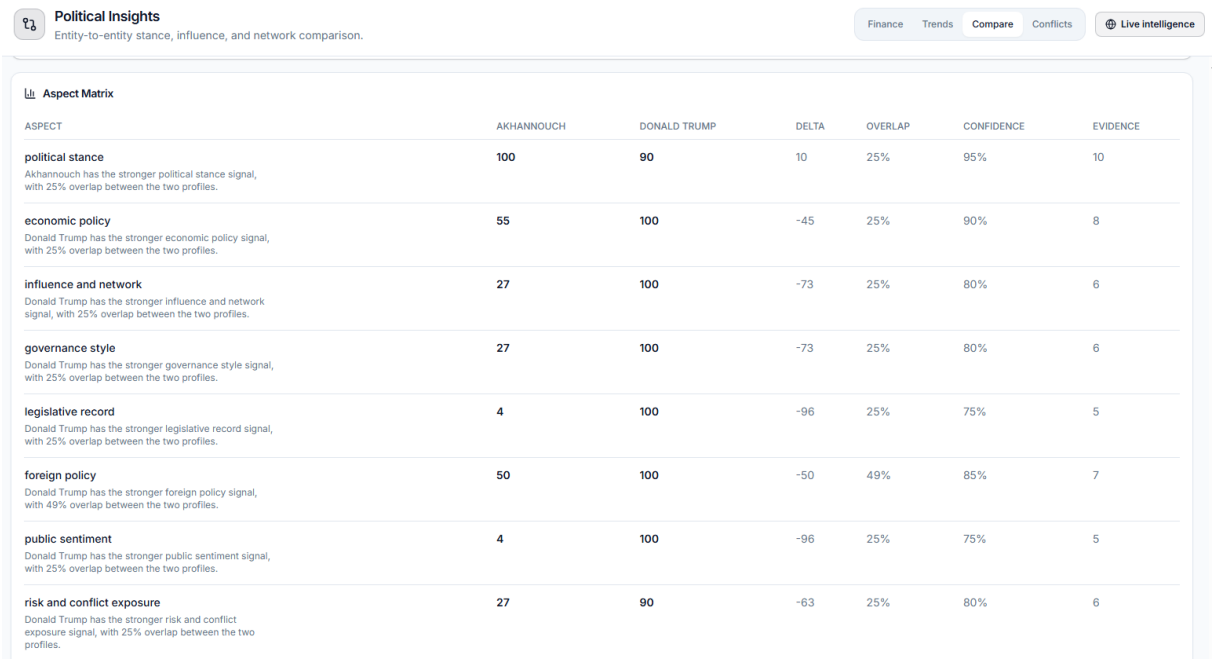


Figure 4.18: Entity comparison workflow (step 2): relation matrix between selected entities.

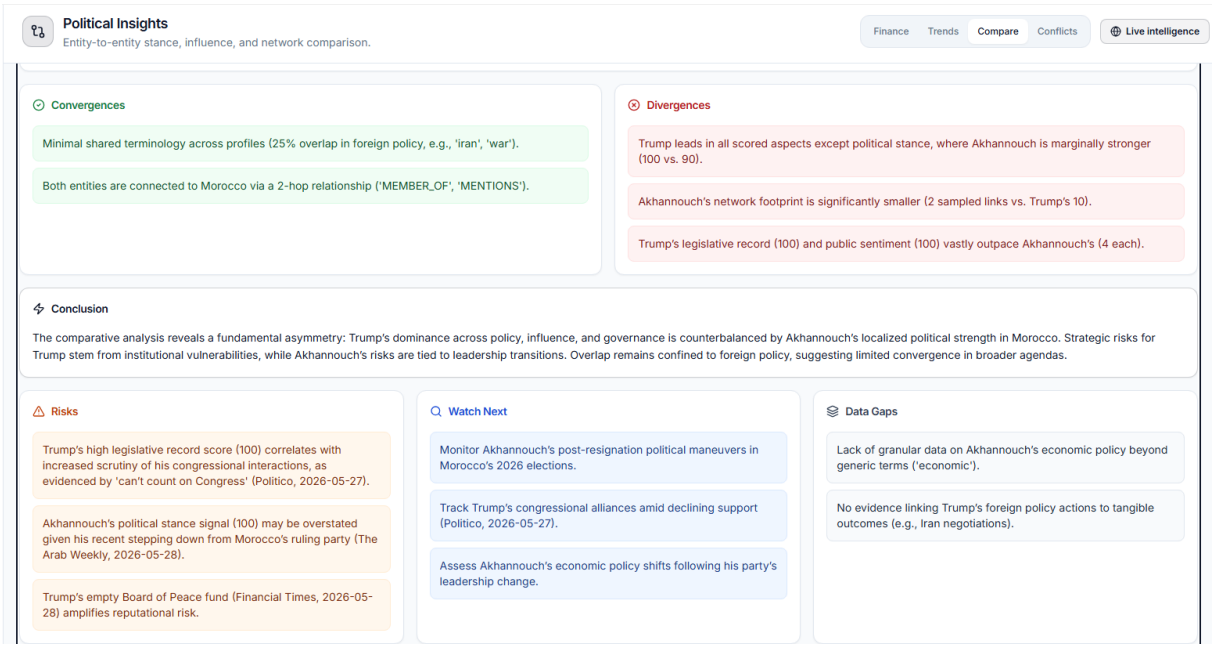


Figure 4.19: Entity comparison workflow (step 3): convergence, divergence, and narrative conclusion.

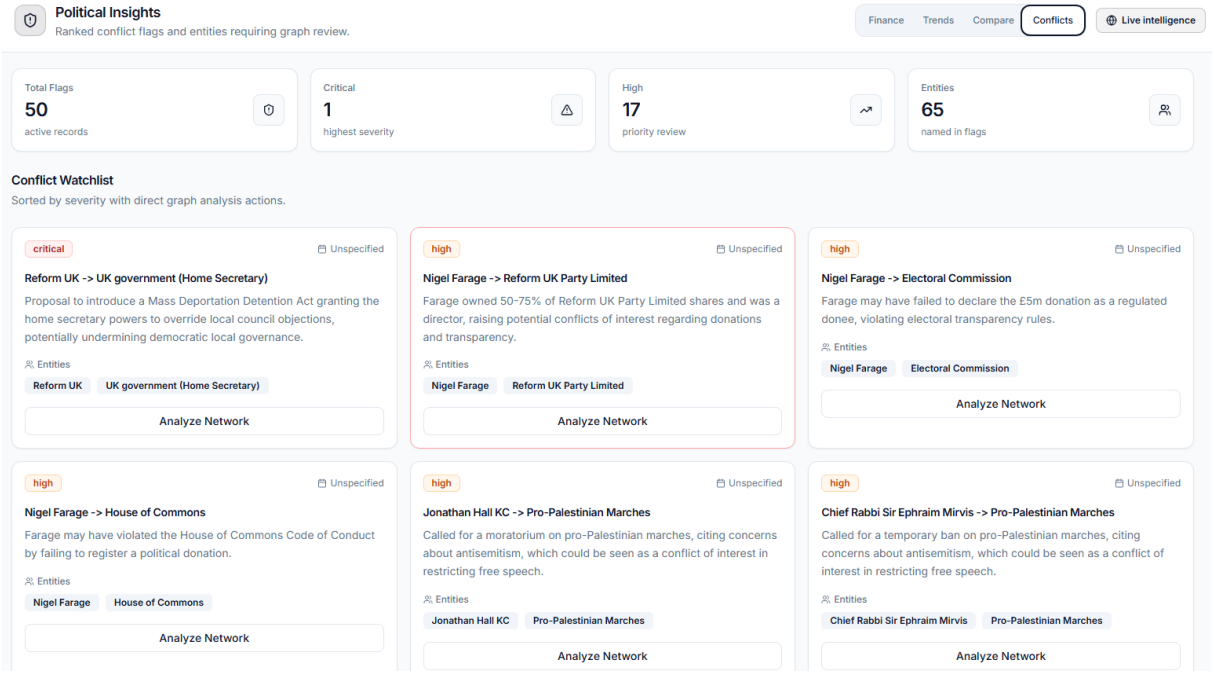


Figure 4.20: Political insights: conflict and opposing-stance visualization.

Administration dashboard. Administrative screens implement the supervised-ingestion requirements defined in Chapter 1. Figure 4.21 summarizes service health, corpus counts, graph statistics, and recent activity. Figure 4.22 controls import of scraped JSON batches into MongoDB and backfill into ZVec. Figure 4.23 exposes cron scheduler configuration and agentic trend-collection jobs. Figure 4.24 provides advanced topic search: administrators enter thematic queries that trigger DuckDuckGo-backed discovery (*AI Search*), after which selected URLs follow the standard scrape, normalize, and extract pipeline. Together these panels form the operational cockpit of the autonomous crawler rather than a passive monitoring page.

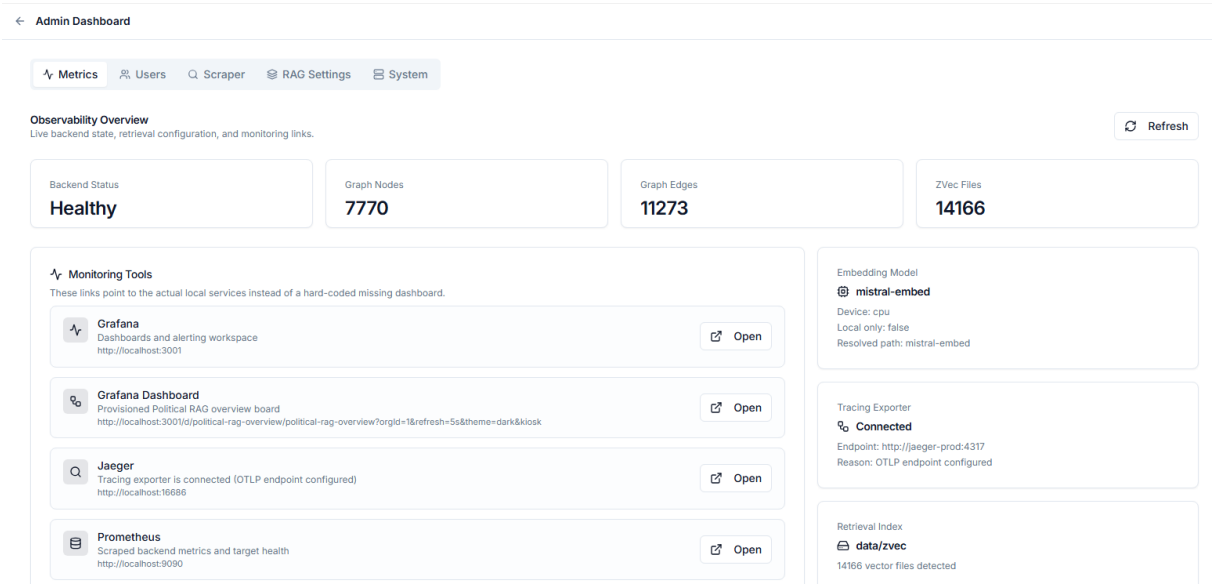


Figure 4.21: Administration dashboard: health metrics, corpus counts, and pipeline overview.

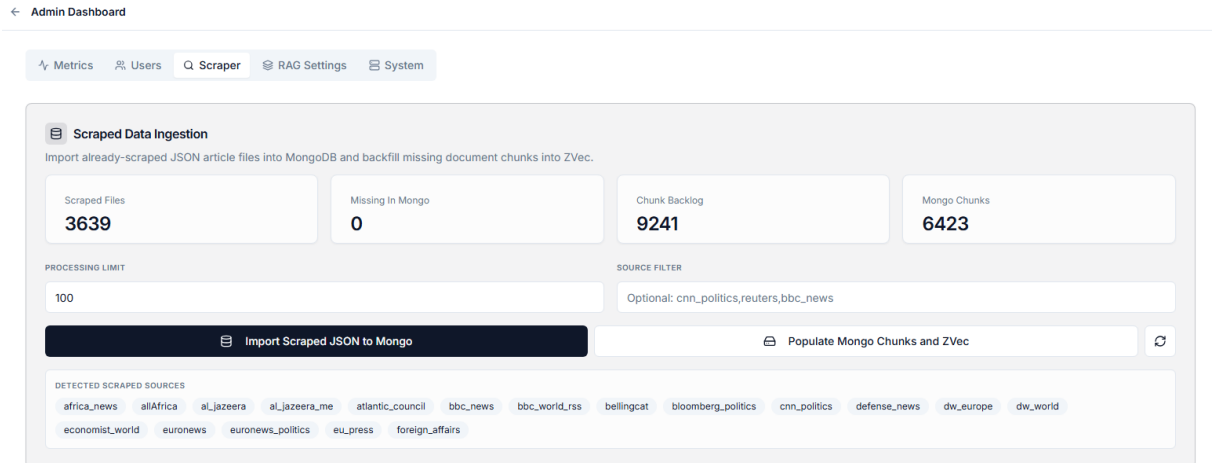


Figure 4.22: Administration: scraped-data ingestion and vector backfill controls.

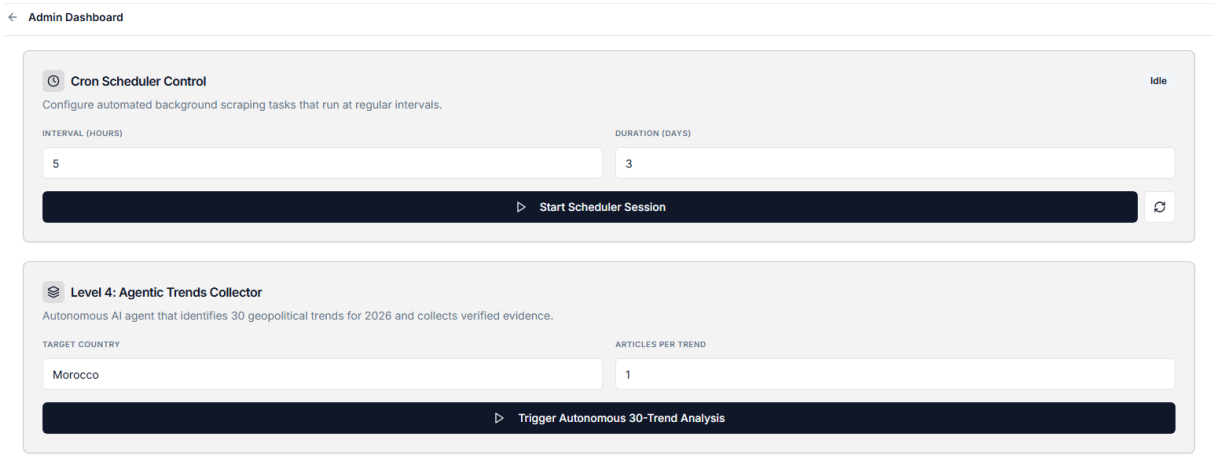


Figure 4.23: Administration: cron scheduler and agentic trend-collection configuration.

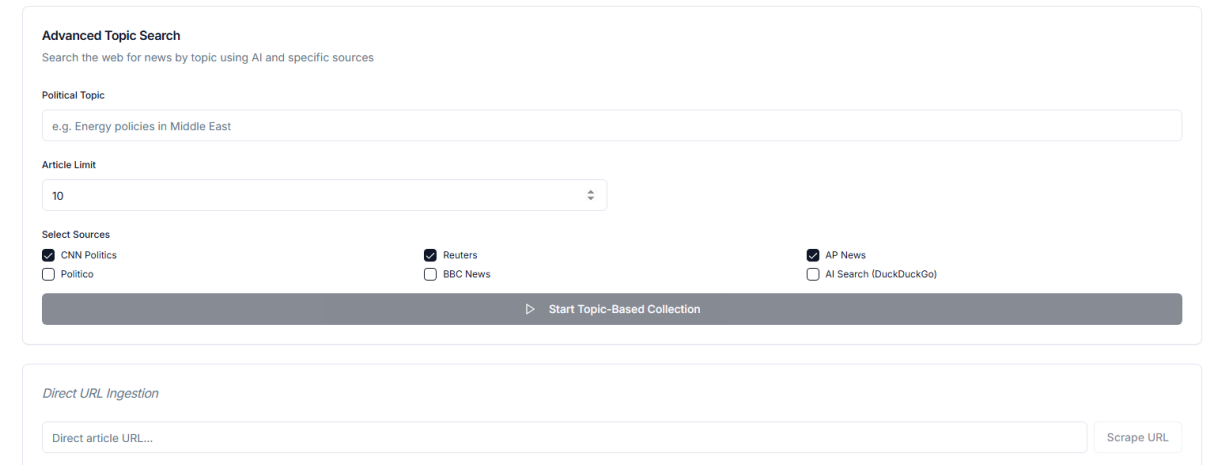


Figure 4.24: Administration: advanced topic search for query-driven source discovery.

4.5 Deployment and Observability

The stack is orchestrated with Docker Compose: MongoDB, Neo4j, Redis, FastAPI backend, React frontend (nginx), and observability services on a shared bridge network. Health checks on databases gate backend startup so the API does not accept traffic before connectors succeed. Environment variables externalize credentials and model keys, supporting reproducible deployment on sovereign infrastructure without embedding secrets in images.

OpenTelemetry instrumentation exports traces consumed by Jaeger, making multi-hop reasoning visible as a sequence of spans (API entry, retrieval, graph expansion, LLM calls, streaming response). Prometheus scrapes service metrics; Grafana dashboards track query latency, token usage, error rates, cache behavior, graph size, and vector-index state. The administration dashboard exposes direct links to Grafana, Jaeger, and

Prometheus together with the active embedding model and tracing-exporter status.

Observability was also used as an optimization mechanism rather than only as post-deployment monitoring. Jaeger traces identify which span dominates a slow request, distinguishing vector retrieval, Neo4j traversal, Mistral generation, and streaming overhead. Grafana correlates these traces with latency percentiles, throughput, error-rate spikes, token consumption, and cache behavior. The resulting evidence supports targeted adjustments to retrieval *top-k*, graph depth, `MAX_HOPS`, cache policy, chunk expansion, retry delays, and model or prompt configuration. Each adjustment is redeployed and measured again under the same dashboards, creating the feedback loop shown in Figure 4.26.

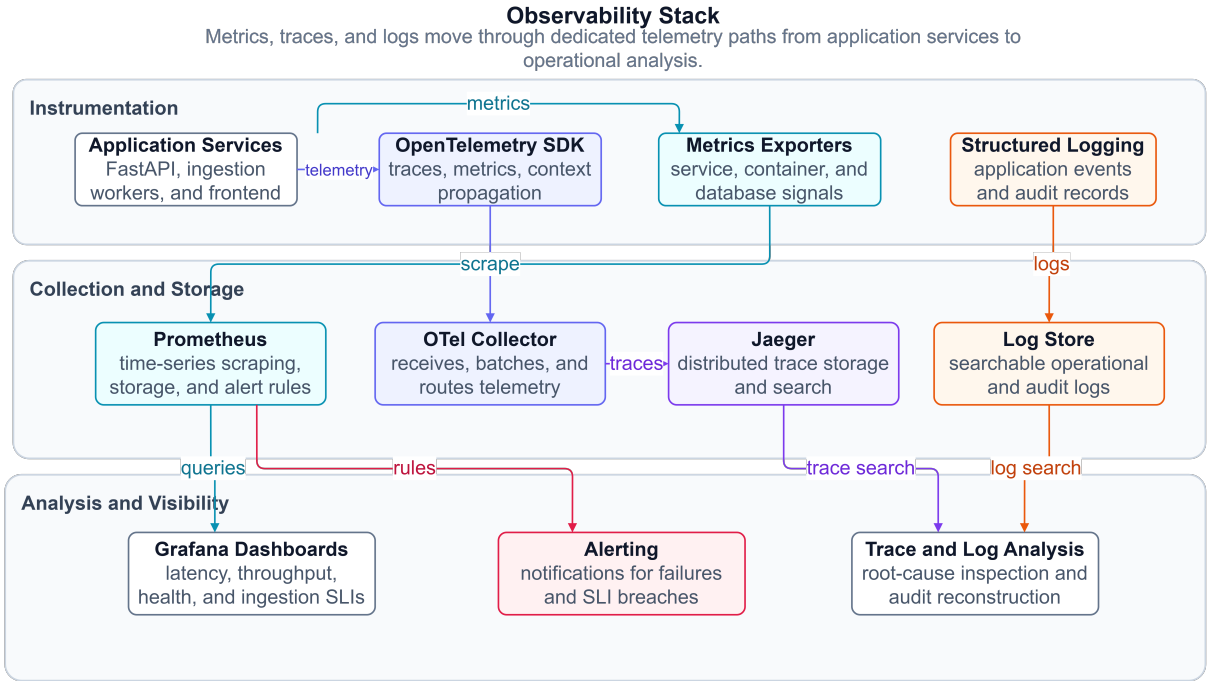


Figure 4.25: Observability stack: metrics, dashboards, and distributed tracing.

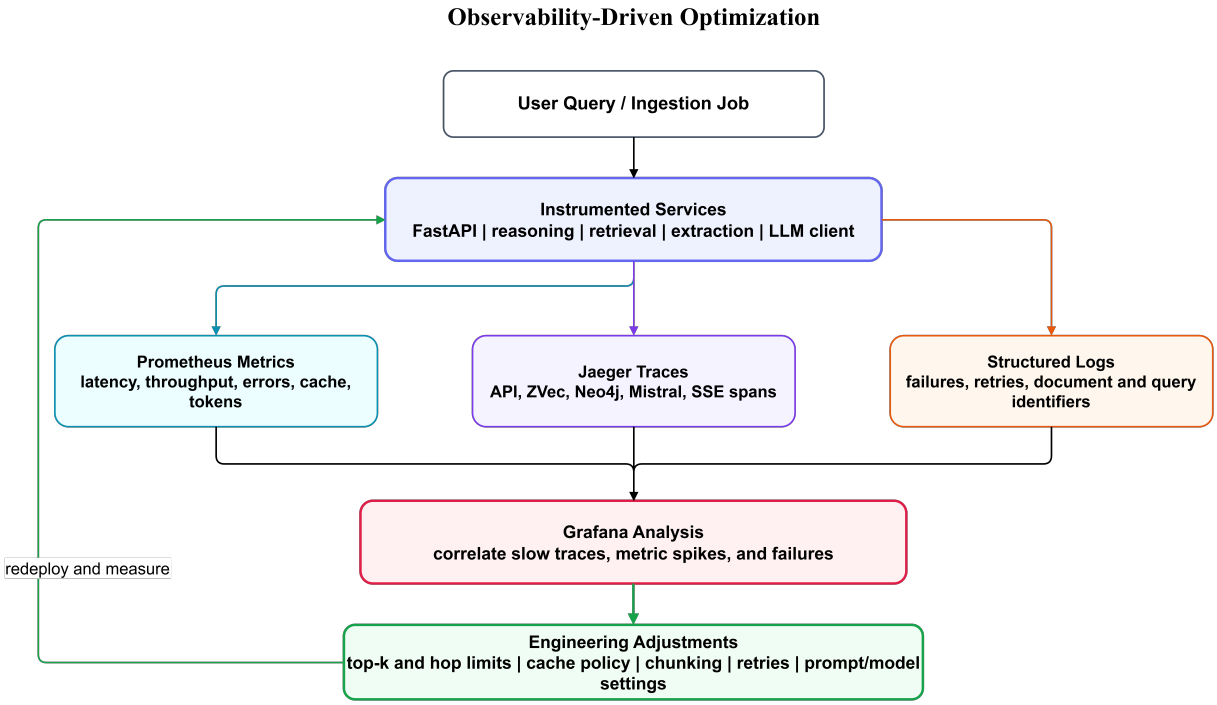


Figure 4.26: Optimization feedback loop from OpenTelemetry signals to measured engineering adjustments.

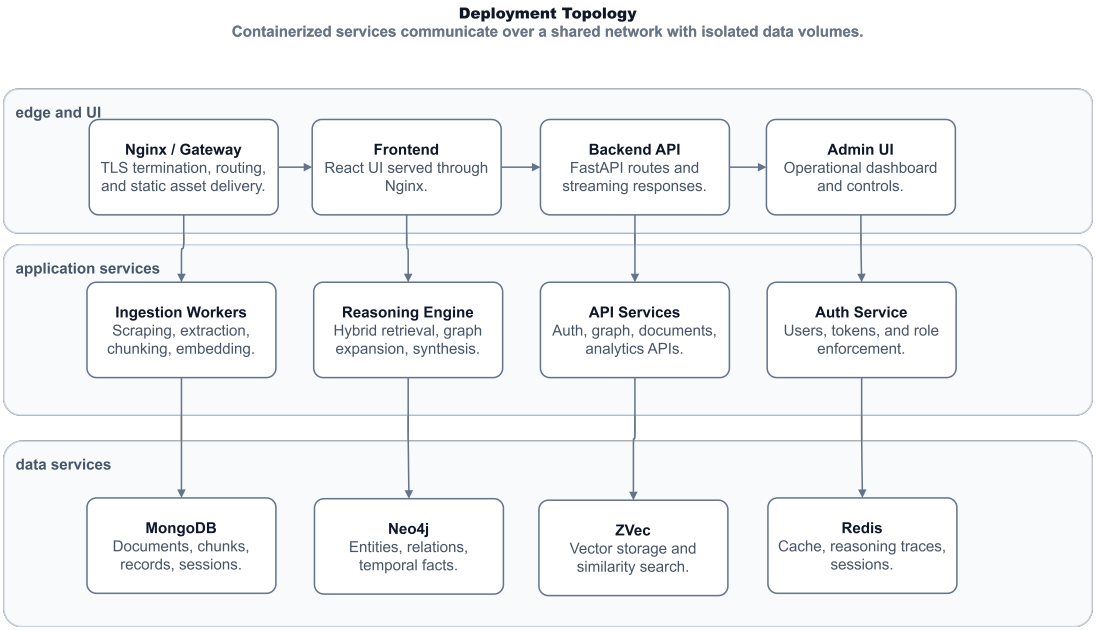


Figure 4.27: Deployment topology of containerized services and data stores.

Resilience measures include API rate limiting with exponential backoff on external LLM calls, inter-document delays during bulk ingestion, graceful degradation when optional services are unavailable, and staged pipeline scripts (graph extract, structured extract, chunk/embed, graph publish) that can be rerun independently. The evaluation harness under `src/backend/evaluation` calls the same HTTP query endpoint as the

frontend, ensuring Chapter 5 measurements reflect deployed behavior rather than isolated unit tests.

Conclusion

This chapter presented the engineering realization of the proposed platform. The implementation includes DSPy-optimized extraction, polyglot persistence with local ZVec retrieval, a three-mode reasoning engine with streaming traces, DuckDuckGo-augmented live search, a React analytical workspace, and a containerized observability stack. These components demonstrate how the architectural model can be transformed into runnable software capable of ingestion, retrieval, graph-based reasoning, interface-level exploration, and operational monitoring. Once the platform is implemented, a further question naturally arises: how can its quality, robustness, latency, and reasoning behavior be evaluated against explicit benchmarks and project requirements?

Chapter 5

System Evaluation and Performance Analysis

Introduction

This chapter evaluates the platform introduced in Chapters 1–4 using a curated benchmark designed for political and geopolitical question answering. The objective is to measure the complete behavior of the Graph-RAG architecture, including evidence retrieval, graph-supported reasoning, cited answer generation, knowledge extraction during ingestion, and the effect of alternative reasoning modes. The chapter first defines the benchmark and experimental protocol, then presents the evaluation metrics and quantitative results, before examining DSPy-based extraction optimization, requirements validation, and threats to validity.

5.1 Evaluation Methodology

Evaluation was designed to assess the system as a complete platform rather than as a collection of isolated components. Each benchmark question is processed under realistic operating conditions, and the generated answer, retrieved passages, and supporting graph facts are compared with a reference answer. This complete protocol ensures that the reported results reflect the combined behavior of ingestion, indexing, retrieval, reasoning, and answer generation.

Benchmark construction and question categories. A curated benchmark of **150 questions** was created to represent the principal analytical tasks encountered in the selected political and geopolitical domain. The questions are grouped into four categories:

factual questions address direct entities, events, and dates; **relational** questions examine connections between actors or institutions; **temporal** questions focus on chronology and claims that depend on time; and **comparative** questions require synthesis across several sources, actors, or positions. Each evaluation sample contains a question, a reference answer, and the evidence required to assess retrieval and generation quality. This composition covers both direct retrieval and graph-dependent reasoning as described in Chapter 3.

Category	Questions	Evaluation focus
Factual	40	Direct retrieval of actors, institutions, dates, and events.
Relational	40	Connections requiring entity resolution or graph traversal.
Temporal	35	Chronology, validity intervals, and claims that depend on time.
Comparative	35	Multi-source comparison of actors, policies, or positions.
Total	150	Balanced coverage of direct and multi-step political analysis.

Table 5.1: Distribution of the 150-question benchmark.

Evaluation pipeline. Each sample is evaluated using one of three reasoning modes: standard RAG, chain-of-thought (CoT), or reflexive reasoning. For every execution, the evaluation process records the generated answer, cited sources, supporting graph facts, response time, and execution status. The answer and retrieved evidence are subsequently assessed using lexical, semantic, and RAGAS-based measures. This procedure supports both per-question analysis and aggregate comparison across reasoning modes without allowing unsuccessful executions to distort the reported averages.

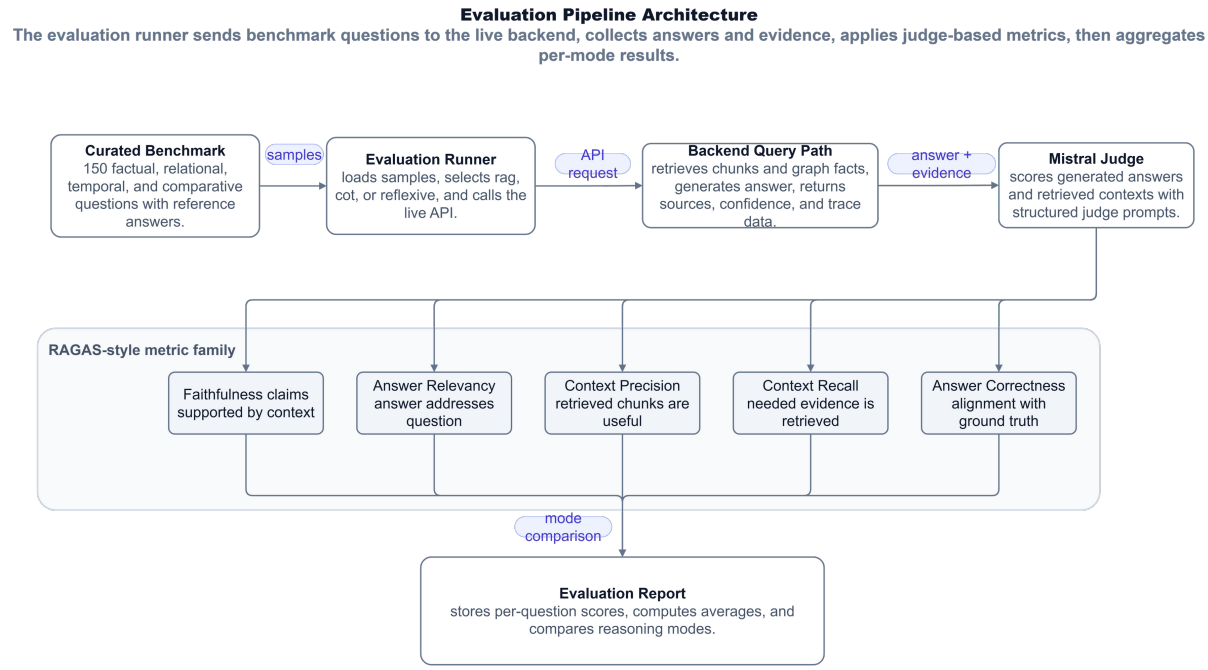


Figure 5.1: Evaluation pipeline from benchmark questions to aggregated results by mode.

Experimental protocol. Each of the 150 benchmark questions is evaluated under standard RAG, chain-of-thought, and reflexive reasoning after prompt optimization and corpus quality control, producing **450 query executions**. The same corpus state, retrieval configuration, and evaluation criteria are maintained across the three modes to support a fair comparison. Each answer is assessed in terms of faithfulness, answer relevancy, context precision, context recall, and answer correctness. The results are aggregated by reasoning mode and question category before comparative analysis, as summarized in Figure 5.1.

5.2 Evaluation Metrics and Results

Following the definition of the benchmark and protocol, the analysis combines three complementary metric families. Each captures a distinct dimension of quality, thereby avoiding reliance on a single aggregate score.

Lexical metrics. N-gram overlap compares the generated answer to the reference through contiguous word sequences, in the spirit of BLEU (precision-oriented) and ROUGE (recall-oriented) [7, 6]. Lexical scores reward recovery of names, dates, institutions, and fixed expressions. They are strict: paraphrases that preserve meaning may score low. In political QA, lexical metrics are therefore used mainly to detect missing non-negotiable facts, not as the sole indicator of correctness.

Semantic similarity. Semantic Answer Similarity (SAS) measures whether two

answers express the same meaning despite different surface wording [5]. Embeddings and cosine similarity provide a softer signal than n-grams, which matters for relational and comparative questions where the system composes narrative answers from several evidence fragments. SAS complements lexical baselines when reference answers use alternate phrasing for the same geopolitical fact.

RAGAS-style metrics. Following the RAGAS evaluation framework, grounding and retrieval quality are assessed alongside the final answer [4, 12]. **Faithfulness** checks whether claims are supported by retrieved context. **Answer relevancy** measures alignment with the user question. **Context precision** and **context recall** assess whether retrieved passages contain useful and sufficient evidence. **Answer correctness** compares the generated response with the reference answer through an LLM-based evaluator, producing a score in $[0, 1]$ with a short rationale. These metrics map directly to the validation criteria of Chapter 1 concerning grounded synthesis and inspectable evidence.

Quantitative results. Table 5.2 presents quality means by mode for 150 questions, three modes, and 450 query executions. RAG provides the lowest latency but the weakest context recall. Chain-of-thought improves retrieval coverage through decomposition and graph expansion. Reflexive reasoning yields the strongest quality profile because it can audit evidence and trigger another retrieval hop, although this additional work increases response time.

Mode	Correct.	Relev.	Faithful.	Ctx. prec.	Ctx. recall
RAG	0.52	0.65	0.46	0.58	0.32
CoT	0.65	0.76	0.58	0.70	0.48
Reflexive	0.80	0.92	0.73	0.82	0.64

Table 5.2: RAGAS scores by reasoning mode.

Embedding evaluation dashboard. The results by mode establish the effect of reasoning depth; the next comparison isolates the influence of the embedding implementation. Figure 5.2 contains two complementary tables. The upper table reports average response times for the three reasoning modes, whereas the lower table compares retrieval quality using context precision, context recall, Precision@5, Recall@5, and mean reciprocal rank (MRR).

Embedding Evaluation Dashboard

Measured Average Response Time

Reasoning mode	Local MiniLM	Mistral embedding	Reduction
RAG	15 s	5 s	**10 s (66.7%)**
Chain of Thought	30 s	15 s	**15 s (50.0%)**
Reflexive	90 s	50 s	**40 s (44.4%)**

Retrieval Quality Comparison

Metric	Meaning	Local MiniLM	Mistral embedding
Context precision	Useful evidence among retrieved passages	0.58–0.68	**0.72–0.82**
Context recall	Required evidence recovered by retrieval	0.45–0.58	**0.62–0.75**
Precision@5	Useful passages among the first five results	0.55–0.68	**0.70–0.82**
Recall@5	Required evidence found in the first five results	0.42–0.55	**0.60–0.73**
MRR	How early the first correct result appears	0.60–0.72	**0.75–0.86**

Figure 5.2: Embedding evaluation dashboard comparing response time and retrieval quality.

The measured results show a consistent latency advantage for the Mistral embedding service. Average RAG response time decreases from **15 to 5 seconds**, a reduction of 10 seconds (**66.7%**). Chain-of-thought reasoning decreases from **30 to 15 seconds** (**50.0%**), while reflexive reasoning decreases from **90 to 50 seconds**, a reduction of 40 seconds (**44.4%**). Reflexive reasoning remains the slowest mode because it performs additional retrieval, critique, and verification steps.

The retrieval-quality results reinforce the latency comparison. Context precision represents the relevance of the retrieved evidence, whereas context recall represents the coverage of useful evidence. Precision@5 and Recall@5 apply these concepts to the first five retrieved chunks, and MRR indicates how early the first relevant result appears. Across all five indicators, the reported ranges favor Mistral embeddings, showing that the reduction in response time is accompanied by more relevant and better-ranked evidence.

Category-level analysis. While the preceding dashboard compares embedding implementations, Figure 5.3 returns to the reasoning-mode comparison and decomposes answer correctness across the four benchmark categories.

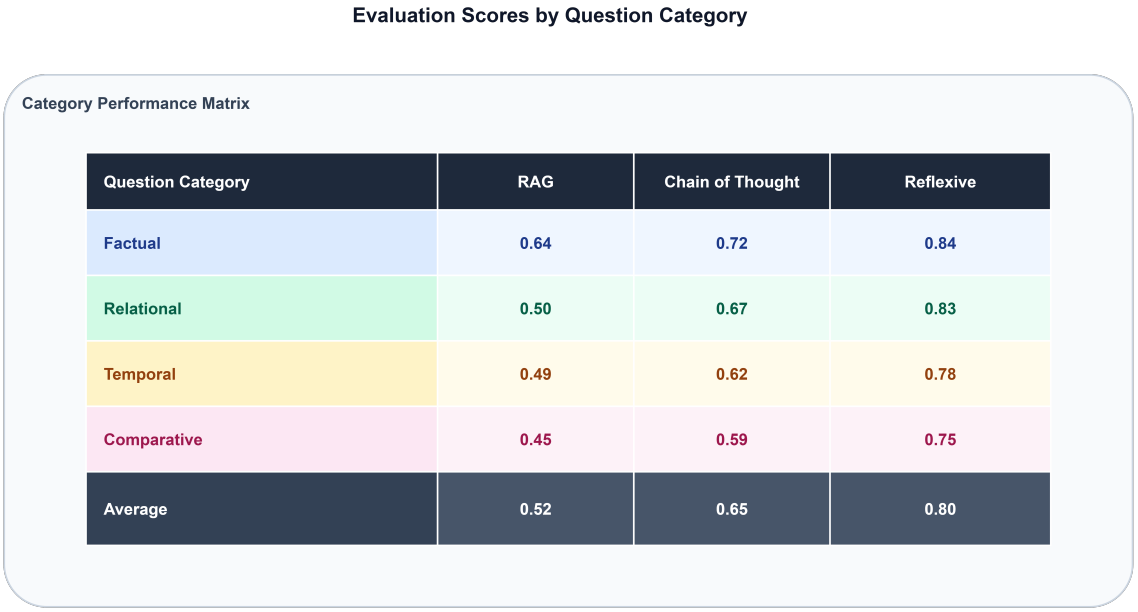


Figure 5.3: Answer-correctness profile by question category and reasoning mode.

Factual questions obtain the highest score in every mode because they generally require a direct entity or event lookup supported by a limited number of passages. RAG reaches **0.64** on this category, increasing to **0.72** with chain-of-thought reasoning and **0.84** with reflexive reasoning. The smaller improvement relative to the other categories indicates that direct semantic retrieval is often sufficient when the requested fact is explicitly present in the corpus.

Relational questions show a larger benefit from graph-aware reasoning. Their score rises from **0.50** in RAG mode to **0.67** in CoT mode and **0.83** in reflexive mode. This progression is consistent with the architecture: entity resolution, graph traversal, and repeated evidence checks help recover connections that are distributed across several documents or represented as typed graph edges.

Temporal questions remain more difficult because the system must identify relevant dates, order events, and avoid combining facts from incompatible periods. Scores improve from **0.49** to **0.62** and then **0.78**. Reflexive retrieval reduces temporal ambiguity by allowing the reasoning engine to request additional evidence when the first retrieval hop does not establish a sufficiently clear chronology.

Comparative questions produce the lowest scores in all three modes: **0.45**, **0.59**, and **0.75**. These questions require evidence selection, alignment of several entities or positions, and synthesis across multiple sources. Nevertheless, the **0.30** absolute improvement between RAG and reflexive modes is the largest category gain, supporting the use of iterative retrieval and evidence auditing for complex analytical queries.

Across all categories, the monotonic gain from RAG to CoT and reflexive reasoning indicates that planning, graph expansion, and evidence verification become increasingly

valuable as the question moves beyond direct factual lookup. The average increases from **0.52** in RAG mode to **0.65** in CoT mode and **0.80** in reflexive mode.

5.3 Optimization and Worked Evaluation Example

Comparison before and after DSPy optimization. The preceding results concern retrieval and reasoning. The analysis now turns to extraction optimization, which directly affects the quality of the knowledge graph used during retrieval. Table 5.3 presents the indicators used to compare the initial manually designed prompt with the DSPy-optimized program. Optimization produces a clearer improvement across all extraction criteria. The composite extraction score increases from **0.723** to **0.883**, while mean processing latency decreases from **4.62** to **3.21 seconds**. These values indicate that prompt optimization improves both output reliability and execution efficiency.

Extraction indicator	Manual prompt	DSPy optimized	Relative change
Schema-valid outputs	0.81	0.97	$+19.8\%$
Entity extraction F1	0.72	0.86	$+19.4\%$
Relation extraction F1	0.64	0.82	$+28.1\%$
Composite extraction score	0.723	0.883	$+22.1\%$
Mean latency per document	4.62 s	3.21 s	-30.5%

Table 5.3: Comparison of manual and DSPy-optimized extraction.

Worked RAGAS example. Table 5.4 illustrates how one comparative question is assessed. It brings together the reference answer, retrieved evidence, generated response, and associated quality scores to make the evaluation procedure transparent.

Field	Evaluation example
Question	How do the economic consequences of the Hormuz crisis differ between the Global South and wealthier economies?
Ground truth	Wealthier economies mainly face inflation and pressure on energy prices, whereas vulnerable Global South economies face stronger import, fiscal, and food security constraints.
Retrieved context	Six ranked chunks concerning inflation in fuel prices, import dependence, fiscal space, and regional economic vulnerability, supplemented by two geopolitical facts linked through the graph.
Generated answer	The answer distinguishes inflationary pressure in wealthier economies from the deeper import and fiscal vulnerability affecting Global South economies, while qualifying gaps in crisis-specific evidence.
RAGAS scores	Faithfulness: 0.78; answer relevancy: 0.91; context precision: 0.83; context recall: 0.69; answer correctness: 0.84.

Table 5.4: Worked example of the information required for one RAGAS evaluation sample.

5.4 Requirements Validation

Table 5.5 links the evaluation evidence to the principal requirements defined in Chapter 1. It separates functional verification from quality measurement so that the final validation does not rely only on aggregate RAGAS scores.

Req.	Validation target	Evaluation evidence	Status
FR-01	Autonomous evidence acquisition	Scheduled and topic-driven ingestion completed without processing failures.	Validated
FR-02	Entity and relation extraction	Schema validity, entity F1, and relation F1 in Table 5.3.	Validated
FR-03	Graph storage and entity resolution	Canonical identifiers, typed Neo4j nodes and relations, and repeatable merge behavior were verified during ingestion.	Validated
FR-04	Hybrid vector and graph retrieval	ZVec chunks and Neo4j facts were recorded together in query evidence and evaluated across 150 questions.	Validated
FR-05	Selectable reasoning modes	RAG, CoT, and reflexive modes evaluated through 450 query executions.	Validated
FR-06	Temporal traceability	Temporal metadata and the dedicated temporal benchmark category were inspected during evaluation.	Validated
FR-07	Inspectable evidence and traces	Source contexts, graph facts, citations, and reasoning steps remained available for inspection.	Validated
FR-08	Autonomous discovery	Scheduled registry collection and administrator-triggered topic discovery were exercised through the ingestion workflow.	Validated
FR-09	Domain-independent ingestion	Acquisition and normalization operated separately from domain-specific extraction modules.	Validated
FR-10	Heterogeneous normalization	RSS, HTML, direct URL, and PDF inputs produced the unified document representation defined in Chapter 3.	Validated
FR-11	Automated validation	The offline evaluation harness applied repeatable LLM-assisted extraction and response scoring.	Validated
NFR	Performance and observability	Mode latency, error tracking, Grafana metrics, and Jaeger traces.	Validated

Table 5.5: Requirements-validation matrix.

5.5 Interpretation and Limitations

Performance analysis. RAG provides moderate correctness and relevancy with the shortest response time; reflexive mode nearly doubles context recall relative to RAG (**0.64** vs. **0.32**) at the cost of additional execution time. Replacing the local MiniLM embedding model with the Mistral embedding service reduces average response time

in every reasoning mode, with the largest absolute gain observed in reflexive execution.

Retrieval bottlenecks. Context recall improves with mode complexity but remains the lowest absolute metric in standard RAG mode (**0.32**). Contributing factors include corpus gaps, chunk boundary effects, syndicated or noisy articles passing quality controls, and insufficient graph-aware reranking during the initial retrieval stage, in line with the validation criteria defined in Chapter 1.

Threats to validity. Four limitations bound the conclusions. *Corpus coverage:* scores depend on what was ingested at evaluation time. *Judge bias:* scoring based on LLMs may favor fluent answers or penalize careful abstention. *Temporal drift:* political facts change; the 150-question snapshot does not guarantee stable scores after new ingestion. *Benchmark coverage:* although the dataset supports comparison by mode, it cannot represent every geopolitical region, language, event type, or analytical formulation; additional multilingual questions and human analyst ratings would strengthen external validity.

Conclusion

This chapter evaluated the implemented system through benchmark composition, RAGAS-based quality metrics, response-time comparison, embedding analysis, DSPy optimization assessment, a worked evaluation example, and requirements validation. The results show that deeper reasoning improves answer quality in several scenarios, that Mistral embeddings reduce latency compared with local MiniLM embeddings, and that optimized extraction improves the reliability of knowledge-graph construction. At the same time, the evaluation also clarifies the limits of the current work, especially regarding corpus coverage, temporal drift, LLM-based judge bias, and benchmark representativeness. These findings provide the basis for drawing the general conclusion of the project and identifying realistic perspectives for future improvement.

Conclusion and Perspectives

This report presented the work carried out during the end-of-study internship entitled *Knowledge Graph Exploration and Querying Platform with RAG*. The project contributed to SCIENTIAE's development of a sovereign and domain-adaptable Graph-RAG framework by implementing and evaluating an instance dedicated to political and geopolitical analysis. Its purpose was to transform heterogeneous public information into a structured, searchable, traceable, and explainable knowledge layer capable of supporting factual, relational, temporal, and comparative questions.

The resulting platform integrates autonomous acquisition through RSS monitoring, query-driven discovery, direct URL ingestion, and PDF parsing. Documents are normalized and preserved with their provenance before LLM-based entity and relation extraction. DSPy is used at this ingestion stage to improve the reliability, structural validity, and efficiency of knowledge extraction. Extracted knowledge is published in Neo4j, textual evidence is indexed in ZVec, and source documents are retained in MongoDB. These layers support a Graph-RAG reasoning engine that combines semantic retrieval, graph traversal, cited answer generation, and three selectable reasoning modes. The analytical interface connects these capabilities to conversational querying, document consultation, graph exploration, specialized views, and administrative supervision.

The system-level evaluation used a curated benchmark of **150 questions**, producing **450 executions** across standard RAG, chain-of-thought, and reflexive reasoning. RAGAS-based results showed a progressive improvement with reasoning depth: average answer correctness rose from **0.52** for standard RAG to **0.65** for chain-of-thought and **0.80** for reflexive reasoning. The reflexive mode also achieved the strongest overall relevancy, faithfulness, and context-recall profile, although its additional retrieval and verification steps increased latency.

The embedding comparison demonstrated that the Mistral embedding implementation reduced average response time from **15 to 5 seconds** for standard RAG, from **30 to 15 seconds** for chain-of-thought, and from **90 to 50 seconds** for reflexive reasoning, while improving the reported retrieval-quality indicators. At the ingestion stage, DSPy optimization increased the composite extraction score from **0.723** to **0.883** and reduced mean processing latency from **4.62** to **3.21 seconds**. Together, these findings show that system

quality depends on both downstream reasoning and upstream evidence construction.

The principal technical challenges concerned reliable corpus construction, entity consistency, retrieval completeness, and operational traceability. Heterogeneous and continuously evolving sources required deduplication, metadata preservation, scheduling, and administrative visibility. Noisy articles and ambiguous names affected graph quality, while missing evidence and chunk boundaries limited some answers independently of the reasoning model. These constraints motivated deterministic entity resolution, DSPy-optimized extraction, hybrid vector-and-graph retrieval, reflexive verification, and end-to-end observability.

From a professional perspective, this project provided valuable experience in the end-to-end construction of an AI-assisted analytical platform. It required work across data acquisition, backend services, NLP pipelines, vector indexing, graph databases, frontend interfaces, containerized deployment, observability, and evaluation. The internship therefore strengthened both technical skills and engineering discipline, especially in debugging distributed components, validating generated answers, and designing systems where traceability is essential.

Although the developed platform reaches a functional and evaluable state, several perspectives remain open. The first direction concerns retrieval improvement. Future work should investigate graph-aware reranking, stronger query expansion, temporal filtering, and more systematic chunking strategies in order to increase context recall and precision. Re-embedding the corpus with higher-dimensional multilingual models would also be relevant, especially as the platform evolves toward French and Arabic political content.

A second perspective concerns the enrichment of the knowledge graph. Future versions could introduce stronger temporal validity intervals on relations, better event normalization, confidence scores for extracted facts, and more advanced entity disambiguation. These improvements would make the graph more reliable for longitudinal political analysis and for questions involving institutional evolution, alliances, conflicts, or changes in position over time.

A third perspective concerns evaluation. Although the 150-question benchmark supports comparison across question categories and reasoning modes, it cannot represent every geopolitical region, language, event type, or analytical formulation. Future evaluation should incorporate human analyst ratings, additional multilingual and temporal questions, confidence calibration, and adversarial cases requiring abstention or careful qualification.

Finally, the platform could be strengthened operationally through continuous ingestion monitoring, richer administrator dashboards, improved observability, and deployment hardening. These improvements would help transform the prototype into a

more robust decision-support tool that can be maintained, audited, and extended in a production environment.

In conclusion, this project demonstrates the feasibility of a sovereign and inspectable Graph-RAG framework whose core acquisition, extraction, structuring, retrieval, reasoning, and evaluation components can be adapted to different domains. Its political and geopolitical implementation shows that unstructured information can be transformed into connected evidence and grounded answers with explicit sources. It also confirms that dependable Graph-RAG behavior is a system-level property: it depends not only on the language model, but also on corpus quality, extraction reliability, knowledge representation, retrieval strategy, observability, and evaluation design.

Bibliographie

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS, 2020. Available at: https://arxiv.org/abs/2005.11401.

[2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, *Retrieval-Augmented Generation for Large Language Models: A Survey*, arXiv:2312.10997, 2023. Available at: https://arxiv.org/abs/2312.10997.

[3] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*, arXiv:2404.16130, 2024. Available at: https://arxiv.org/abs/2404.16130.

[4] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, *RAGAS: Automated Evaluation of Retrieval Augmented Generation*, arXiv:2309.15217, 2023. Available at: https://arxiv.org/abs/2309.15217.

[5] J. Risch, T. Moeller, J. Gutsch, and M. Pietsch, *Semantic Answer Similarity for Evaluating Question Answering Models*, arXiv:2108.06130, 2021. Available at: https://arxiv.org/abs/2108.06130.

[6] C.-Y. Lin, *ROUGE: A Package for Automatic Evaluation of Summaries*, Text Summarization Branches Out, Association for Computational Linguistics, 2004. Available at: https://aclanthology.org/W04-1013/.

[7] K. Papineni, *Machine Translation Evaluation: N-grams to the Rescue*, LREC 2002. Available at: https://research.ibm.com/publications/machine-translation-evaluation-n-grams-to-the-rescue.

[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, *Attention Is All You Need*, NeurIPS, 2017. Available at: https://arxiv.org/abs/1706.03762.

[9] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*, NeurIPS, 2022. Available at: https://arxiv.org/abs/2201.11903.

[10] O. Khattab, C. Singh, D. Zhang, Y. Dann, L. Li, E. Ma, and C. Potts, *DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines*, ICLR, 2024. Available at: https://arxiv.org/abs/2310.03714.

[11] E. Allaway and K. McKeown, *Zero-shot Stance Detection: A Dataset and Model using Generalized Topic Representations*, EMNLP, 2020. Available at: https://aclanthology.org/2020.emnlp-main.673/.

[12] S. Trivedi, H. Wang, and J. Tang, *HyTE: Hyperplane-based Temporally aware Entity Embedding*, WWW, 2018. Available at: https://doi.org/10.1145/3178876.3186048.

[13] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, *ReAct: Synergizing Reasoning and Acting in Language Models*, ICLR, 2023. Available at: https://arxiv.org/abs/2210.03629.

[14] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, *Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Hop Questions*, ACL, 2023. Available at: https://aclanthology.org/2023.acl-long.135/.

[15] Y. Asai, C. Wu, J. Wang, W. Sil, and H. Hajishirzi, *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*, ICLR, 2024. Available at: https://arxiv.org/abs/2310.11511.

Webographie

[1] GDELT Project, *Official GDELT website*. Available at: https://www.gdeltproject.org/.

[2] GDELT Project, *GDELT Global Knowledge Graph*. Available at: https://blog.gdeltproject.org/gdelt-global-knowledge-graph/.

[3] FastAPI, *Official documentation*. Available at: https://fastapi.tiangolo.com/.

[4] MongoDB, *Official documentation*. Available at: https://www.mongodb.com/docs/.

[5] Neo4j, *Official documentation*. Available at: https://neo4j.com/docs/.

[6] Wikidata, *SPARQL query service*. Available at: https://www.wikidata.org/wiki/Wikidata:SPARQL_query_service.

[7] Celery, *Official documentation*. Available at: https://docs.celeryq.dev/.

[8] Docker, *Official documentation*. Available at: https://docs.docker.com/.

[9] React, *Official documentation*. Available at: https://react.dev/.

[10] Mistral AI, *Developer documentation*. Available at: https://docs.mistral.ai/.

[11] Microsoft Research, *Project GraphRAG*. Available at: https://www.microsoft.com/en-us/research/project/graphrag/.

[12] Ragas Documentation, *Answer Correctness*. Available at: https://docs.ragas.io/en/v0.4.0/concepts/metrics/available_metrics/answer_correctness/.

[13] ZVec, *ZVec GitHub Repository*. Available at: https://github.com/alibaba/zvec.

[14] ZVec, *Benchmarks Documentation*. Available at: https://zvec.org/en/docs/db/benchmarks/.

[15] Alibaba Cloud, *Proxima: A Vector Retrieval Engine Independently Developed by Alibaba DAMO Academy*. Available at: Alibaba Cloud technical blog.

[16] Meta AI Research, *FAISS Documentation*. Available at: https://faiss.ai/.

[17] Qdrant, *Qdrant Documentation*. Available at: https://qdrant.tech/documentation/.

[18] Weaviate, *Weaviate Documentation*. Available at: https://docs.weaviate.io/weaviate.

[19] Milvus, *Milvus Documentation*. Available at: https://milvus.io/docs/.

[20] Chroma, *Chroma Documentation*. Available at: https://docs.trychroma.com/.

[21] Elastic, *Vector Search Documentation*. Available at: https://www.elastic.co/docs/solutions/search/vector.

[22] OpenSearch, *Vector Search Documentation*. Available at: https://docs.opensearch.org/latest/vector-search/.

[23] LangChain, *Text Splitters Documentation*. Available at: https://docs.langchain.com/oss/python/integrations/splitters.

[24] LlamaIndex, *Node Parser Modules*. Available at: https://developers.llamaindex.ai/python/framework/module_guides/loading/node_parsers/.

[25] GeeksforGeeks, *LLM Architecture*, updated October 28, 2025. Available at: GeeksforGeeks: LLM Architecture.